Discrete and Probabilistic Systems

ENGS 27 — Course Notes

Prof. Bijan Mazaheri
Thayer School of Engineering, Dartmouth College

Fall 2026

These notes were written by Bijan Mazaheri from his lectures. Claude, an AI assistant made by Anthropic, helped check for errors and edit; the content, examples, and structure are the author's.

Contents

0 About This Course **1**
- 0.1 What This Course Is About . . . 1
- 0.2 Seven Questions . . . 1
- 0.3 Problem Sets . . . 2
- 0.4 Seven Ideas That Run Through the Course . . . 2

1 Sets, Logic, Proofs, and Counting **4**
- 1.1 Set Theory . . . 4
  - 1.1.1 Sets and Notation . . . 4
  - 1.1.2 Operations on Sets . . . 5
  - 1.1.3 Cardinality . . . 6
  - 1.1.4 Power Sets . . . 6
  - 1.1.5 Indicator Functions . . . 7
- 1.2 Logic and Proof Techniques . . . 7
  - 1.2.1 Statements and Connectives . . . 8
  - 1.2.2 Implication is Containment . . . 8
  - 1.2.3 Quantifiers . . . 9
  - 1.2.4 Direct Proof and the Contrapositive . . . 10
  - 1.2.5 Proof by Contradiction . . . 10
  - 1.2.6 Induction . . . 10
  - 1.2.7 The Pigeonhole Principle . . . 11
- 1.3 Counting . . . 12
  - 1.3.1 The Product and Sum Rules . . . 12
  - 1.3.2 Factorials and Overcounting . . . 14
  - 1.3.3 Permutations and Combinations . . . 14
  - 1.3.4 Inclusion/Exclusion . . . 16
  - 1.3.5 A First Coding Bound . . . 17

2 Probability **19**
- 2.1 Probability . . . 19
  - 2.1.1 Sample Spaces and Events . . . 19
  - 2.1.2 Positivity, Unit Measure, Additivity . . . 19
  - 2.1.3 Interpreting Probability . . . 20
  - 2.1.4 Common Manipulations . . . 20
  - 2.1.5 Equally Likely Outcomes . . . 21
- 2.2 Conditioning . . . 23
  - 2.2.1 Conditional, Joint, and Marginal Probability . . . 24
  - 2.2.2 Independence . . . 25
  - 2.2.3 The Chain Rule . . . 27

2.2.4 The Law of Total Probability . . . 27
2.2.5 Bayes' Rule . . . 28
2.3 Random Variables and Expectation . . . 29
2.3.1 Random Variables . . . 29
2.3.2 Expectation . . . 30
2.3.3 Linearity of Expectation . . . 30
2.3.4 Indicator Random Variables . . . 31
2.3.5 Conditional Expectation and the Tower Rule . . . 32
2.3.6 Simpson's Paradox . . . 32
2.4 Discrete Distributions . . . 33
2.4.1 Bernoulli and Binomial . . . 33
2.4.2 Geometric . . . 34
2.4.3 Approximating Many Failures . . . 35
2.4.4 Negative Binomial . . . 35
2.4.5 Collecting Everything . . . 36
2.4.6 Summary . . . 38
2.5 Variance and Probability Concentration . . . 38
2.5.1 Variance and Standard Deviation . . . 39
2.5.2 Covariance and Sums . . . 39
2.5.3 The Law of Large Numbers . . . 41
2.5.4 Monte Carlo Estimation and $1/\sqrt{n}$ Error . . . 43

3 Information and Coding 45
3.1 Information Theory . . . 45
3.1.1 Twenty Questions . . . 45
3.1.2 Self-Information . . . 46
3.1.3 Entropy . . . 46
3.1.4 Conditional Entropy . . . 47
3.1.5 Mutual Information . . . 48
3.1.6 Back to Wordle . . . 50
3.2 Compression . . . 50
3.2.1 Decision Trees . . . 51
3.2.2 Variable-Length Codes . . . 52
3.2.3 Prefix-Free Codes . . . 52
3.2.4 Shannon–Fano Coding . . . 53
3.2.5 Huffman Coding . . . 54
3.2.6 Optimality of Huffman Codes . . . 55
3.2.7 Lossless and Lossy Compression . . . 56
3.3 Error Correction . . . 57
3.3.1 Noisy Channels . . . 57
3.3.2 Hamming Distance . . . 58
3.3.3 Parity Checks . . . 59
3.3.4 Hamming Codes . . . 59
3.3.5 Comments and Further Reading . . . 61

4 Graphs and Random Graphs 63
4.1 Introduction to Graphs . . . 63
4.1.1 The Bridges of Königsberg . . . 63
4.1.2 Graphs and Walks . . . 64
4.1.3 The Handshake Lemma . . . 65

4.1.4 Graphs as Connections . . . 65
4.1.5 Graphs as Restrictions . . . 66
4.1.6 Vocabulary . . . 71
4.2 Random Graphs and Thresholds . . . 71
4.2.1 From Expectations to Probabilities . . . 71
4.2.2 Birthdays and Collisions . . . 72
4.2.3 The Erdős–Rényi Random Graph . . . 72
4.2.4 Small Structures . . . 73
4.2.5 Isolated Vertices . . . 73
4.2.6 The Connectivity Threshold . . . 73
4.2.7 The Giant Component . . . 75
4.2.8 Summary . . . 75
4.2.9 Implications . . . 75
4.2.10 Growing Versus Breaking . . . 77
4.3 Branching Processes and Epidemics . . . 77
4.3.1 How Big Is a Component? . . . 77
4.3.2 Convolutions . . . 78
4.3.3 Generating Functions . . . 79
4.3.4 Sums of Random Length . . . 79
4.3.5 Branching Processes . . . 80
4.3.6 When the Giant Component Emerges . . . 81

5 Regular Languages and Automata 83
5.1 Finite-State Machines . . . 83
5.1.1 A Machine You Have Already Built . . . 83
5.1.2 Directed Graphs . . . 83
5.1.3 Strings and Languages . . . 83
5.1.4 Deterministic Finite Automata . . . 84
5.1.5 Simple Computations . . . 85
5.1.6 Waiting for a Pattern . . . 85
5.1.7 De Bruijn Graphs . . . 85
5.2 Regular Expressions . . . 87
5.2.1 Languages from Languages . . . 87
5.2.2 Regular Expressions . . . 87
5.2.3 From Regular Expressions to NFAs . . . 88
5.3 The Limits of Finite Memory . . . 90
5.3.1 From NFAs to DFAs . . . 90
5.3.2 Simplifying DFAs . . . 90
5.3.3 The Pumping Lemma . . . 93
5.3.4 A Language That Is Not Regular . . . 94
5.3.5 Why the Filter Fails . . . 94

6 Markov Chains 96
6.1 Adjacency Matrices . . . 96
6.1.1 The Adjacency Matrix . . . 96
6.1.2 Multiplying by the Adjacency Matrix . . . 96
6.1.3 Powers of the Adjacency Matrix . . . 98
6.1.4 Counting Writable Strings . . . 99
6.2 Intro to Markov Chains . . . 100
6.2.1 The Markov Property . . . 100

6.2.2 Examples . . . 101
6.2.3 The Transition Matrix . . . 102
6.2.4 Moving Probability Around . . . 103
6.2.5 Time until Absorption . . . 104
6.3 Linear Algebra and Eigenvalues . . . 105
6.3.1 What Is Linearity? . . . 105
6.3.2 Linear Transformations and Vectors . . . 105
6.3.3 Stretching . . . 106
6.3.4 Basis Changes . . . 106
6.3.5 The Power Method . . . 107
6.3.6 Rotations . . . 108
6.3.7 Useful Facts . . . 108
6.4 Convergence, Ergodicity, and Stationary Distributions . . . 109
6.4.1 When the Power Method Works . . . 109
6.4.2 Nothing Beats 1 . . . 110
6.4.3 Stationary Distributions as Left Eigenvectors . . . 110
6.4.4 Reducible Chains . . . 111
6.4.5 Periodic Chains . . . 111
6.4.6 Ergodicity . . . 112
6.4.7 A Non-Ergodic Example: Gambler's Ruin . . . 113
6.4.8 An Ergodic Example: PageRank . . . 115
6.5 Reversible Markov Chains and MCMC . . . 116
6.5.1 Reversibility and Detailed Balance . . . 116
6.5.2 The Ehrenfest Diffusion Model . . . 118
6.5.3 Metropolis–Hastings . . . 119
6.5.4 Bayesian Inference by Sampling . . . 121

7 Continuous Distributions and Monte Carlo 123
7.1 Poisson and Exponential Distributions . . . 123
7.1.1 Continuous Random Variables and Densities . . . 123
7.1.2 From Coin Flips to Continuous Time . . . 124
7.1.3 The Poisson Distribution . . . 126
7.1.4 The Exponential Distribution . . . 127
7.1.5 Exponentials Are Memoryless . . . 127
7.2 Random Walks and Normal Distributions . . . 129
7.2.1 Zooming In on a Random Walk . . . 129
7.2.2 Sums Smooth to Bells . . . 130
7.2.3 Only the Variance Survives . . . 131
7.2.4 Convolution as a Table . . . 132
7.2.5 Gaussians Are a Fixed Point . . . 133
7.2.6 The Central Limit Theorem . . . 134
7.2.7 Why Gaussians Are Everywhere . . . 135
7.3 Queueing Theory . . . 136
7.3.1 Queues as Birth–Death Chains . . . 136
7.3.2 How Long Is the Line? . . . 136
7.3.3 Little's Law . . . 137
7.3.4 Pooling . . . 137
7.3.5 Why Cloud Computing Won . . . 137
7.3.6 Ergodic Averages Along a Single Chain . . . 138
7.3.7 Computing PageRank . . . 138

7.3.8 Simulating a Queue . . . 139

8 Applications **140**
8.1 Bayesian Networks . . . 140
8.1.1 Motivation: The Cost of a Joint Distribution . . . 140
8.1.2 Bayesian Networks . . . 140
8.1.3 Factorization . . . 141
8.1.4 Three Structures . . . 142
8.1.5 Confounding: When to Condition . . . 142
8.1.6 Colliders: When Not to Condition . . . 143
8.2 Neural Networks and Cellular Automata . . . 144
8.2.1 From Bayesian Networks to Structural Equations . . . 144
8.2.2 Universal Approximation . . . 145
8.2.3 Cellular Automata . . . 146
8.2.4 Conway's Game of Life . . . 146
8.2.5 Universality . . . 146

Chapter 0

About This Course

0.1 What This Course Is About

This course is about two kinds of structure. The first is *discrete* structure: sets, logical statements, graphs, strings, and machines that read them. The second is *probabilistic* structure: what happens when the objects we are studying behave randomly, and what we can nevertheless say with certainty about them.

The two halves are not separate subjects. A graph with random edges tells you when an epidemic takes off. A machine with random transitions instead of fixed ones is a Markov chain, and a Markov chain is how a search engine ranks the web. Most of the interesting results in this course come from putting a discrete structure and a probabilistic question side by side.

0.2 Seven Questions

The course is organized around seven questions. By the end of the term you should have developed a deep intuition for the answer to each one, and be able to explain it in terms of the topics noted alongside.

1. *How does zipping a file make it smaller?*
   Compression (Lecture 10).

2. *How do scratched CDs still work?*
   Error correction and Hamming codes (Lecture 11).

3. *Why do diseases and social media posts go viral? Why do things shatter?*
   Phase transitions: percolation theory and random graphs (Lecture 13).

4. *Why is it impossible to write a filter that catches every attack when you only have limited memory?*
   Regular languages, automata, and the pumping lemma (Lecture 17).

5. *How does Google work?*
   Markov chains, stationary distributions, and PageRank (Lecture 21).

6. *Why did cloud computing take over as a better distribution of tasks than personal computers?*
   Probability concentration (Lecture 8) and queuing theory (Lecture 25).

7. *How did neural networks give rise to AI?*
   Universal approximation (Lecture 27).

The questions are listed in the order in which they are answered. Nothing is resolved before Lecture 10: the first three units build the tools, and after that a question closes out every few lectures.

A recurring theme, and the one that ties the middle of the course together, is **how much you can do with limited memory**. It appears first as a question about machines (Chapter 5), then as an assumption about random processes (the Markov property, Chapter 6), and finally as a question about what is computable at all (Chapter 8).

0.3 Problem Sets

Each problem set covers one unit of the course. Deadlines, grading, and course policies are on the syllabus.

Set 1. Sets, logic, proofs, and counting (Lectures 1–3).

Set 2. Probability foundations (Lectures 4–7).

Set 3. Concentration and information (Lectures 8–11).

Set 4. Graphs and random graphs (Lectures 12–14).

Set 5. Regular languages and automata (Lectures 15–17).

Set 6. Markov chains (Lectures 18–20).

Set 7. Convergence, PageRank, and normal distributions (Lectures 21–24).

Lectures 25 to 27 are not covered by a problem set.

0.4 Seven Ideas That Run Through the Course

The course is organized by topic, but a handful of ideas recur in every part of it, and in every branch of engineering. The table lists them, with examples the course works out and the lecture where each appears. Expect to meet each idea several times, in settings that look unrelated at first.

Idea	Where it appears
Counting expectations. The expected number of anything is a sum of probabilities, even when the events depend on one another.	*Computer:* hash collisions and the birthday problem (4, 6). *Biomedical:* how much sequencing it takes to cover a genome (7). *Energy and communications:* how many links each device needs so that none is cut off (13).
Averages concentrate. Sums and averages of many random pieces become predictable, at a rate that can be computed.	*Computer:* Monte Carlo estimation, and why its error shrinks like $1/\sqrt{n}$ (8). *Everyone:* random walks, and why bell curves are everywhere (24). *Computation:* error bars on Monte Carlo estimates (25).
Thresholds. Behavior can change abruptly at a critical value, and the critical value can be computed.	*Materials:* when a brittle material shatters (13). *Environmental:* whether fluid seeps through a porous medium (13). *Energy:* when a power grid stays connected as lines fail (13). *Biomedical:* when an epidemic takes off (14). *Electrical:* how much a noisy channel can carry (11).
Information and redundancy. Uncertainty can be measured in bits, redundancy added on purpose protects against noise, and both set hard limits.	*Computer:* what a compression program actually does (10). *Electrical:* error correction and channel capacity (11). *Biomedical:* assembling a genome from short reads, and storing data in DNA (15). *Chemical and materials:* Boltzmann's entropy is the same quantity (4, 9, 22). *Everyone:* twenty questions and Wordle (9).
State and memory. A system that remembers only a little can be drawn as a machine, and what it cannot do can be proved.	*Computer:* pattern filters, and why no filter can check nesting (15–17). *Security:* opening a garage door in eight seconds (15). *Electrical:* every digital circuit with memory (15). *Mechanical:* controllers such as an elevator's (15). *Biomedical:* finding where an enzyme cuts DNA (15), and sequences that remember their last few letters (19).
Fixed points. Many processes settle where applying them again changes nothing, and the settling point can be computed without simulating them.	*Chemical:* diffusion and the second law (22). *Materials:* simulating how solids melt, where Markov chain Monte Carlo began (22). *Operations:* queues, and why cloud computing won (25). *AI and search:* PageRank (21). *Biomedical:* whether an outbreak dies out (14). *Statistics:* the normal distribution, the shape that adding leaves unchanged (24).
Graphs as a common language. Many systems are dots and connections, and the same few facts about graphs apply to all of them.	*Operations:* scheduling and register allocation as coloring (12). *Networks:* ranking runners from race results, and random graphs with and without hubs (13). *Biomedical:* assembling a genome by walking a de Bruijn graph (15). *Counting:* walks counted by powers of a matrix (18). *AI and search:* the web as a graph for PageRank (21), causes as arrows between variables (26), and what neural networks, layered graphs of simple units, can represent (27).

Chapter 1

Sets, Logic, Proofs, and Counting

This chapter builds the vocabulary and the reasoning habits that the rest of the course assumes. It is only three lectures long, and almost everything later depends on it. Two items in particular come back in ways that are easy to miss: the power set construction of Section 1.1.4 reappears when we convert nondeterministic machines to deterministic ones, and the pigeonhole principle of Section 1.2.7 is the engine behind the central impossibility result of Chapter 5.

1.1 Set Theory

1.1.1 Sets and Notation

A *set* is a collection of distinct objects, considered without regard to order or repetition. The objects are its *elements*, and we write $x \in A$ to mean that x is an element of A, and $x \notin A$ otherwise.

Two ways to specify a set:

- By *roster*, listing the elements: $D = \{1, 2, 3, 4, 5, 6\}$.
- By *set-builder* notation, giving a rule: $E = \{\, x \in D : x \text{ is even} \,\}$, read "the set of x in D such that x is even." Here $E = \{2, 4, 6\}$.

Because a set ignores order and repetition, $\{1, 2, 2, 3\} = \{3, 1, 2\}$. This is the difference between a set and the data structures you are used to from programming, where both order and multiplicity usually matter.

The *empty set* $\emptyset$ has no elements. We often fix a *universe* U containing everything under discussion; for a die roll, $U = \{1, \ldots, 6\}$.

The die example illustrates where this chapter is headed. A question about the roll, such as "was it even?", picks out a subset of U. In Chapter 2 those subsets are called *events*, and a probability is a function that assigns a number to each of them. So the sets defined in this lecture are the objects probability is about, and the operations defined in the next subsection are how probabilistic questions get combined.

Definition 1.1.1 (Subset and equality)**.** $A \subseteq B$ means every element of A is an element of B. If $A \subseteq B$ and $B \subseteq A$, then $A = B$.

That last sentence is more useful than it looks. It says that to prove two sets are equal, you prove two containments: take an arbitrary element of one side and show it belongs to the other, then do the reverse. This is the first proof technique in the course.

1.1.2 Operations on Sets

Definition 1.1.2 (Union, intersection, complement, difference)**.** For $A, B \subseteq U$:

$$\begin{aligned} A \cup B &= \{\, x : x \in A \text{ or } x \in B \,\} && \text{(union)} \\ A \cap B &= \{\, x : x \in A \text{ and } x \in B \,\} && \text{(intersection)} \\ A^c &= \{\, x \in U : x \notin A \,\} && \text{(complement)} \\ A \setminus B &= \{\, x : x \in A \text{ and } x \notin B \,\} && \text{(difference)} \end{aligned}$$

Sets A and B are *disjoint* if $A \cap B = \emptyset$.

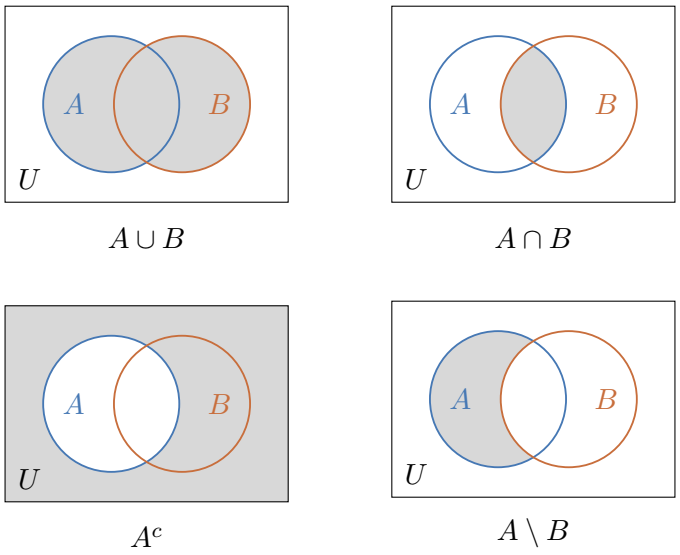


Figure 1.1: The four operations, with the resulting set shaded in each case. The rectangle is the universe U. Complement is the only one of the four that depends on what U is: shrink the universe and A^c shrinks with it, while $A \cup B$, $A \cap B$ and $A \setminus B$ do not change.

Figure 1.1 shows all four. In mathematics "or" is inclusive: $x \in A \cup B$ is satisfied when x lies in both.

Take the die again, with $U = \{1, \ldots, 6\}$, and let $E = \{2, 4, 6\}$ be the even rolls and $G = \{4, 5, 6\}$ the rolls greater than 3. Then

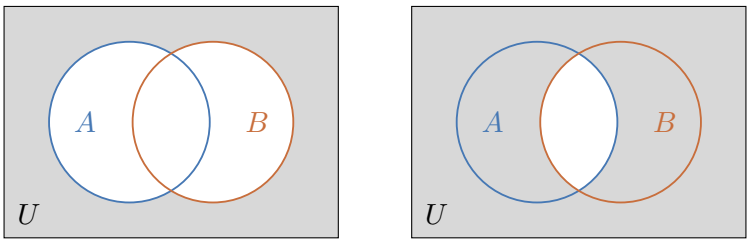
$$E \cap G = \{4, 6\}, \qquad E \cup G = \{2, 4, 5, 6\}, \qquad E^c = \{1, 3, 5\}, \qquad E \setminus G = \{2\},$$

which read as "even and greater than 3," "even or greater than 3," "not even," and "even but not greater than 3." So $\cup$ and $\cap$ correspond exactly to "or" and "and," and complement to "not." That correspondence between set operations and logical connectives is the bridge to the next lecture, and it is why the axioms of probability in Section 2.1 are stated in terms of unions and complements.

Theorem 1.1.3 (De Morgan's laws)**.** *For any* $A, B \subseteq U$,

$$(A \cup B)^c = A^c \cap B^c \qquad \textit{and} \qquad (A \cap B)^c = A^c \cup B^c.$$

The identities are easy to believe and we prove them in Section 1.2, once we have the vocabulary for negating a compound statement, which is what they really are.



$(A \cup B)^c = A^c \cap B^c$ $(A \cap B)^c = A^c \cup B^c$

Figure 1.2: De Morgan's laws. In each panel the shaded region is the set named underneath. On the left, being outside $A \cup B$ means being outside both, so the shaded region is everything beyond the two circles. On the right, being outside $A \cap B$ means failing to be in at least one of them, so only the lens where they overlap is excluded.

In words: the complement of "A or B" is "not A and not B." You will use this constantly in probability, where the easiest way to compute the chance that *something* happens is often to compute (one minus) the chance that *nothing* does.

Figure 1.2 shows both identities as Venn diagrams. The picture is an aid to memory, not a proof. A Venn diagram with two sets shows four regions, and one can check the identity region by region, but that argument does not generalize, and with four or more sets no planar diagram shows all the regions at all.

1.1.3 Cardinality

Definition 1.1.4 (Cardinality)**.** For a finite set A, the *cardinality* $|A|$ is the number of elements in A.

Proposition 1.1.5. *For finite sets A and B, $|A \cup B| \leq |A| + |B|$, with equality if and only if A and B are disjoint.*

The inequality is loose precisely because elements of $A \cap B$ get counted twice on the right-hand side. Fixing that bookkeeping is the content of the inclusion/exclusion principle, which we take up in Section 1.3 once we have a reason to care about counting.

Looking ahead

Proposition 1.1.5 is the set-theoretic ancestor of the *union bound* in probability, which will be one of the most heavily used tools in the second half of the course. The statement $\Pr[A \cup B] \leq \Pr[A] + \Pr[B]$ is the same observation with counting replaced by measure.

Remark 1.1.6. Cardinality also makes sense for infinite sets, and the theory there is surprising: writing $\mathbb{N} = \{1, 2, 3, \dots\}$ for the natural numbers, $\mathbb{Z}$ for the integers, $\mathbb{Q}$ for the rationals and $\mathbb{R}$ for the reals, it turns out that $\mathbb{Z}$ and $\mathbb{Q}$ have the same size while $\mathbb{R}$ is strictly larger. We will not need any of it. Every set we count in this course is finite.

1.1.4 Power Sets

Definition 1.1.7 (Power set)**.** The *power set* of S, written $\mathcal{P}(S)$, is the set of all subsets of S, including $\emptyset$ and S itself.

Example 1.1.8. For $S = \{a, b, c\}$,

$$\mathcal{P}(S) = \{\, \emptyset,\ \{a\},\ \{b\},\ \{c\},\ \{a, b\},\ \{a, c\},\ \{b, c\},\ \{a, b, c\} \,\},$$

so $|\mathcal{P}(S)| = 8$. The elements of $\mathcal{P}(S)$ are themselves sets, which confuses people at first.

Theorem 1.1.9. *If $|S| = n$ then $|\mathcal{P}(S)| = 2^n$.*

Proof. Label the elements of S as $s_1, \ldots, s_n$. To each subset $A \subseteq S$ associate the string of bits $b_1 b_2 \cdots b_n$ where $b_i = 1$ if $s_i \in A$ and $b_i = 0$ otherwise.

This association loses no information: from the string we can read off exactly which elements are in A, so different subsets give different strings, and every string of length n arises from some subset. Subsets of S and bit strings of length n are therefore two descriptions of the same collection of objects.

There are n positions and two choices at each, so there are 2^n such strings, and hence 2^n subsets. □

That proof technique of matching two collections element for element, so that a count of one is a count of the other, is called a *correspondence*. This is a very useful tool: instead of counting the objects you were asked about, you can find a second collection of objects that they correspond to exactly, and count that instead.

1.1.5 Indicator Functions

The bit string in that proof deserves a name of its own, because it comes back repeatedly.

Definition 1.1.10 (Indicator function)**.** The *indicator function* of a subset $A \subseteq S$ is the function

$$\mathbb{1}_A(x) = \begin{cases} 1 & \text{if } x \in A, \\ 0 & \text{if } x \notin A. \end{cases}$$

A subset and its indicator function carry exactly the same information, so we may use whichever is more convenient. Sets are easier to talk about; indicators are easier to compute with, because they turn membership into arithmetic. Note for instance that

$$\mathbb{1}_{A \cap B} = \mathbb{1}_A \cdot \mathbb{1}_B, \qquad \mathbb{1}_{A^c} = 1 - \mathbb{1}_A,$$

so multiplication behaves like "and" and subtraction from one behaves like "not." Logic becomes algebra. We use this twice more: in Section 2.3 to count things on average, and in Section 6.1 to turn an entire graph into a matrix.

Looking ahead

In Section 5.2 we will convert a nondeterministic machine into a deterministic one by building a new machine whose states are *sets* of the original states, which is the power set construction. The 2^n of Theorem 1.1.9 is why that conversion can blow up in size.

1.2 Logic and Proof Techniques

A proof is an argument that would convince a skeptical reader who grants you only the definitions and what has already been established. This lecture is about the standard shapes such arguments take. Two of those shapes, contradiction and induction, will be doing real work later in the course.

1.2.1 Statements and Connectives

A *statement* is an assertion that is either true or false. From statements P and Q we build others: the negation $\neg P$, the conjunction $P \wedge Q$, the disjunction $P \vee Q$, the implication $P \Rightarrow Q$, and the biconditional $P \Leftrightarrow Q$.

The implication is the one that causes trouble. $P \Rightarrow Q$ asserts only that Q holds *whenever* P does; it says nothing about the case where P is false, and in that case the implication is considered true. It is useful to keep three related statements straight:

Implication	$P \Rightarrow Q$
Converse	$Q \Rightarrow P$
Contrapositive	$\neg Q \Rightarrow \neg P$

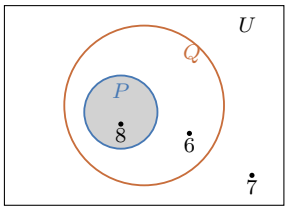
The contrapositive is logically equivalent to the original implication. The converse is not, and conflating the two is the single most common error in informal reasoning.

The distinction makes implication worth being careful about in design work. A guarantee that a system in a safe state stays safe is an implication, and it says nothing whatever about a system that is already unsafe.

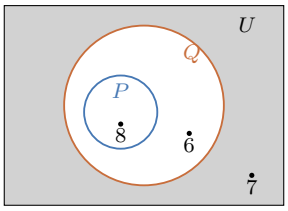
1.2.2 Implication is Containment

Why is the contrapositive equivalent, and the converse not? The clearest answer is a picture, and it uses the correspondence from Section 1.1: if $\cap$ is "and" and complement is "not," then $\subseteq$ is "implies." Write P for the set of things satisfying P and Q for the set satisfying Q. Then

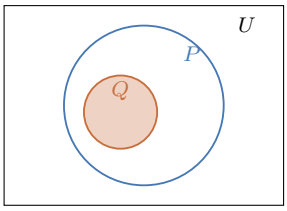
$$P \Rightarrow Q \quad \text{means} \quad P \subseteq Q.$$



(a) $P \Rightarrow Q$
every point of P is in Q



(b) $\neg Q \Rightarrow \neg P$
every point outside Q is outside P



(c) $Q \Rightarrow P$
every point of Q is in P

Figure 1.3: Implication as containment. In (a) and (b), P is the multiples of 4 and Q the even numbers, and the two panels are the same picture: (a) reads the containment forwards, (b) reads it from outside, which is the contrapositive. Panel (c) is what $Q \Rightarrow P$ would require, the containment the other way around.

Complementing both sides of a containment reverses it,

$$P \subseteq Q \quad \Longleftrightarrow \quad Q^c \subseteq P^c,$$

so proving the contrapositive is not a weaker substitute for proving the original: it is the same statement, approached from the other side. The converse asserts $Q \subseteq P$, which is a different containment altogether, and holds only when the two sets are equal.

1.2.3 Quantifiers

Most mathematical statements are about all or some members of a set. We write

$$\forall x \in U,\ P(x) \qquad \text{and} \qquad \exists x \in U,\ P(x)$$

for "every element of U satisfies P" and "some element of U satisfies P."

Negation exchanges the two quantifiers and negates the predicate:

$$\neg\big(\forall x \in U,\ P(x)\big) \equiv \exists x \in U,\ \neg P(x),$$
$$\neg\big(\exists x \in U,\ P(x)\big) \equiv \forall x \in U,\ \neg P(x).$$

The denial of "every swan is white" is that some swan is not white; it is not that every swan is non-white, and it says nothing at all about things that are not swans. To refute the statement you produce one swan of the wrong color, which is why a single counterexample settles a universal claim.

A universal statement is a long conjunction and an existential one is a long disjunction: if $U = \{u_1, \ldots, u_n\}$ then

$$\forall x \in U,\ P(x) \quad \text{is} \quad P(u_1) \wedge P(u_2) \wedge \cdots \wedge P(u_n),$$
$$\exists x \in U,\ P(x) \quad \text{is} \quad P(u_1) \vee P(u_2) \vee \cdots \vee P(u_n).$$

So $\forall$ behaves like "and" and $\exists$ like "or."

Negation also moves through those connectives, and the two rules for doing so are the statements we met in Section 1.1:

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q, \qquad \neg(P \wedge Q) \equiv \neg P \vee \neg Q.$$

The first says that for neither P nor Q to hold, both must fail; the second that for the pair not to hold together, at least one must fail. Notice that these are the quantifier rules at a different size. Negating a long conjunction turns it into a long disjunction, which is exactly $\neg\forall$ becoming $\exists\neg$. This also explains the remark above about counterexamples: to defeat a conjunction you need only one false term.

This also settles Theorem 1.1.3. Membership in $A \cup B$ is the statement $(x \in A) \vee (x \in B)$, so

$$x \in (A \cup B)^c \iff \neg\big((x \in A) \vee (x \in B)\big) \iff (x \notin A) \wedge (x \notin B) \iff x \in A^c \cap B^c,$$

where the middle step is the first rule above. The second identity is the same argument with the second rule. A set operation is a connective applied to the statement "x belongs," and complementing a set is negating that statement, so the set identity and the logical one are the same.

When quantifiers nest, *order matters*, and the most reliable way to read a nested statement is as a game between you and an adversary. In

$$\forall x \in X,\ \exists y \in Y,\ P(x, y),$$

the adversary moves first, choosing some $x \in X$; then you choose $y \in Y$, knowing what they picked. The statement is true if you have a winning reply to every opening move. Reversing it to $\exists y \in Y,\ \forall x \in X,\ P(x, y)$ changes the game entirely: now you must commit to a single y that works against every x.

Looking ahead

The central result of Chapter 5, which answers our question about why a filter with limited memory can always be beaten, is a statement with three nested quantifiers, and there the domains are as constrained as the order: the adversary's move is restricted to strings above a certain length, and that restriction makes the statement true.

1.2.4 Direct Proof and the Contrapositive

A *direct proof* of $P \Rightarrow Q$ assumes P and derives Q. Sometimes the reverse direction is easier, and then we prove the contrapositive instead, which by Figure 1.3 is the same statement. The following needs one of each.

Proposition 1.2.1. *An integer n is even if and only if n^2 is even.*

Proof. Forwards, directly: if $n = 2k$ then $n^2 = 4k^2 = 2(2k^2)$, which is even.

Backwards, by the contrapositive, so we show that if n is odd then n^2 is odd: if $n = 2k+1$ then $n^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$, which is odd. A direct proof of this direction would have to begin with $n^2 = 2m$ and extract information about n, which is harder. □

1.2.5 Proof by Contradiction

To prove P by contradiction, assume $\neg P$ and derive something impossible. The assumption must then have been false.

Theorem 1.2.2. *$\sqrt{2}$ is irrational.*

Proof. Suppose not, so $\sqrt{2} = a/b$ for integers a, b with $b \neq 0$, and assume the fraction is in lowest terms, so a and b are not both even.

Squaring gives $2b^2 = a^2$, so a^2 is even, and a is even by Proposition 1.2.1. Write $a = 2c$. Then $2b^2 = 4c^2$, so $b^2 = 2c^2$, so b^2 is even and therefore b is even.

But now a and b are both even, contradicting the assumption that the fraction was in lowest terms. Hence no such a, b exist. □

Contradiction is the natural shape for proving that something is *impossible*, which is why it will be the form of the strongest negative result in this course. The next subsection gives an impossibility result of direct engineering interest, and Chapter 5 gives the one this course is built toward.

1.2.6 Induction

Theorem 1.2.3 (Principle of mathematical induction)**.** *Let $P(n)$ be a statement about positive integers. If $P(1)$ holds, and if $P(n) \Rightarrow P(n+1)$ for every $n \geq 1$, then $P(n)$ holds for all $n \geq 1$.*

The template has three parts: the *base case*, the *inductive hypothesis* (assume $P(n)$), and the *inductive step* (deduce $P(n+1)$). The following is the standard first example, and it is also the one we use most often later in the course.

Proposition 1.2.4 (Finite geometric sum)**.** *For any $r \neq 1$ and any $n \geq 0$,*

$$\sum_{i=0}^{n} r^i = \frac{r^{n+1} - 1}{r - 1}.$$

Proof. Base case. For $n = 0$ the left side is $r^0 = 1$ and the right side is $(r-1)/(r-1) = 1$.

Inductive step. Assume the formula for n. Then

$$\sum_{i=0}^{n+1} r^i = \left(\sum_{i=0}^{n} r^i\right) + r^{n+1} = \frac{r^{n+1}-1}{r-1} + r^{n+1} = \frac{r^{n+1}-1+r^{n+2}-r^{n+1}}{r-1} = \frac{r^{n+2}-1}{r-1},$$

which is the claim for $n+1$. By induction it holds for all $n \geq 0$. □

The case $r = 2$ says $1 + 2 + 4 + \cdots + 2^n = 2^{n+1} - 1$, which we use in our next proof! The case $|r| < 1$, where the sum of the infinite series is $1/(1-r)$, lets the random graph argument of Section 4.2 and the queue length of Section 7.3 come out in closed form.

When $P(n+1)$ needs more than $P(n)$, assume $P(1), \ldots, P(n)$ all hold instead; this is called *strong induction*, and it is the same principle applied to the statement "$P(k)$ holds for all $k \leq n$."

1.2.7 The Pigeonhole Principle

There is one very common proof by contradiction technique that will recur a few times in this course.

Theorem 1.2.5 (Pigeonhole principle)**.** *If n objects are placed into m boxes and $n > m$, then some box contains at least two objects. More generally, some box contains at least $\lceil n/m \rceil$ of them.*

Proof. If every box contained at most one object, the total number of objects would be at most $m < n$, a contradiction. For the general statement, if every box held fewer than $\lceil n/m \rceil$ objects the total would be less than $m\lceil n/m \rceil$, which is at least n. □

Figure 1.4 says the same thing as a statement about functions: assigning objects to boxes is a function from the objects to the boxes, and with more objects than boxes no such function can be one-to-one.

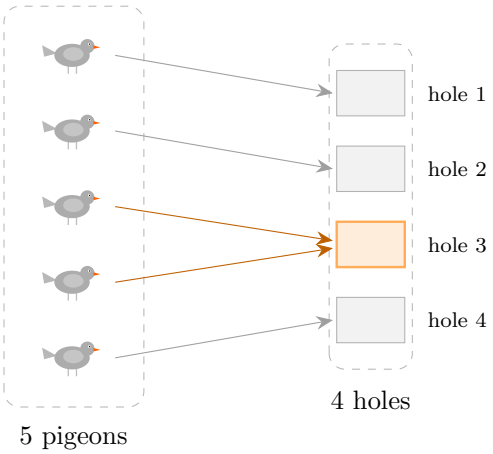


Figure 1.4: The pigeonhole principle as a statement about functions. Assigning pigeons to holes is a function from a five-element set to a four-element set, drawn here with an arrow from each pigeon to its hole. No such function can be one-to-one: some hole receives two arrows, here hole 3. The proof is the counting argument, that four holes holding at most one pigeon each can account for at most four pigeons.

Let's finish with a result that previews data compression. A *lossless compressor* turns any file into a shorter one and lets the original be recovered.

Theorem 1.2.6. *No rule maps every bit string of length n to a strictly shorter string in a way that allows the input to be recovered.*

Proof. Recovering the input requires that no two inputs be sent to the same output. The inputs are the 2^n strings of length n, by Theorem 1.1.9. The available outputs are the strings of length at most $n-1$, and there are $\sum_{i=0}^{n-1} 2^i = 2^n - 1$ of those by Proposition 1.2.4, the $i = 0$ term counting the empty string. With more inputs than outputs, pigeonhole gives two inputs sharing an output, and those two files cannot both be recovered. □

So compression cannot win on every input: a scheme that shortens the files people actually have must lengthen some others. *This reframes the engineering problem because it tells us we need to focus on what we are compressing, not the algorithm itself.* Which inputs to favor, and by how much, is the subject of Section 3.2.

Looking ahead

In Section 3.2 we prove that Huffman codes are optimal, and the argument is an induction on the number of symbols in the alphabet. Several of the counting identities in the next lecture also admit inductive proofs, though we will usually prefer a counting argument because it explains more. Finally, we will use induction to prove a useful fact about adjacency matrices in Section 6.1.

1.3 Counting

Counting is the engine of discrete probability. If every outcome in a finite universe is equally likely, then the probability of an event is the number of outcomes in it divided by the total, and both numbers have to be counted. Correspondence, which we have already used once, is the other thread here: together with double counting and pigeonhole it gives reasoning tools that recur for the rest of the term.

1.3.1 The Product and Sum Rules

In the 1630s, René Descartes watched a fly cross his ceiling and looked for a way to describe its path. Naming a position by a pair of numbers, its distance along one wall and its distance along the other, turned a geometric question into an algebraic one, and the set of all such pairs is still called the Cartesian plane. The construction below is that idea with the numbers replaced by the elements of any two sets.

Definition 1.3.1 (Cartesian product)**.** The *Cartesian product* of A and B is the set of ordered pairs

$$A \times B = \{\, (a, b) : a \in A,\ b \in B \,\}.$$

More generally $A_1 \times \cdots \times A_k$ is the set of ordered k-tuples $(a_1, \ldots, a_k)$ with $a_i \in A_i$.

A Cartesian product is itself a set, but unlike the power set its elements are ordered: $(a, b) \neq (b, a)$ unless $a = b$.

Theorem 1.3.2 (Product rule)**.** $|A_1 \times \cdots \times A_k| = |A_1| \cdots |A_k|$.

Example 1.3.3. A phone comes in four colors and its maker sells five cases. A choice of color and a choice of case are independent of one another, so the pairings are the elements of a Cartesian product and there are $4 \cdot 5 = 20$ of them, as in Figure 1.5.

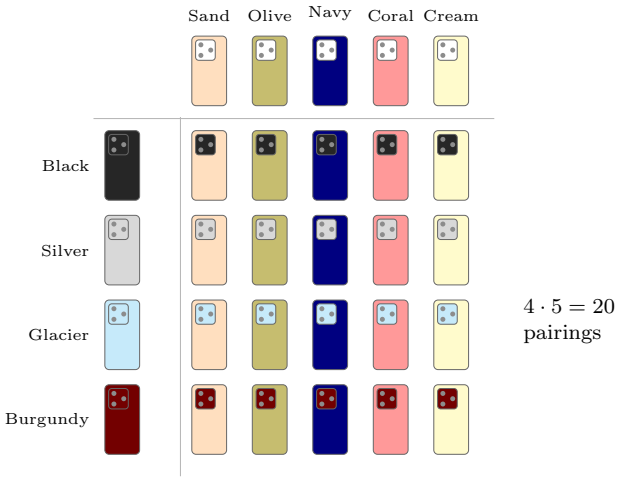


Figure 1.5: The product rule, with the phone color showing through the cutout in the case. Four colors and five cases give one phone per pairing, so the pairings fill a rectangle and there are $4 \cdot 5 = 20$ of them. For k sets of options the picture is a k-dimensional box.

The product rule says that if a process consists of k successive choices, and the number of options at step i is n_i no matter what was chosen earlier, then there are $n_1 n_2 \cdots n_k$ outcomes. The proviso allows that count to change from step to step, as in the shuffle below, but it forbids the count depending on *which* earlier choices were made.

Theorem 1.3.4 (Sum rule)**.** *If $A_1, \ldots, A_k$ are pairwise disjoint, then $|A_1 \cup \cdots \cup A_k| = |A_1| + \cdots + |A_k|$.*

The two rules correspond to the two set constructions from Section 1.1: product for "and then," disjoint union for "or." Nearly every counting problem is an exercise in decomposing a set until only these two rules are needed, and there are only these two ways to do it. Either describe an element by a series of choices, which makes the set a Cartesian product and the count a product of the options at each step; or split the set into pieces, which makes the count a sum over the pieces.

Both decompositions can be done loosely, and each has its own way to fix the "overcounting". Describe a choice that does not really distinguish anything and you count every element several times, once per way of making that spurious choice, so you divide the surplus out. Split into pieces that overlap and you count the shared part more than once, so you subtract it off. Which correction applies is settled by which decomposition you used.

	decompose into choices	decompose into pieces
the count	multiply	add
the repair	divide	subtract

The two columns are the two halves of this lecture. Multiplying and dividing occupy Sections 1.3.2 and 1.3.3; adding and subtracting occupy Section 1.3.4.

Example 1.3.5. A New Hampshire license plate is three letters followed by four digits. By the product rule there are $26^3 \cdot 10^4 = 175{,}760{,}000$ of them.

The state also issues a moose conservation plate, which carries four characters, each of which may be a letter or a digit, so there are $36^4 = 1{,}679{,}616$ of those. A vehicle has one plate or the other, so the two collections are disjoint and the sum rule applies: the state can issue

$$26^3 \cdot 10^4 + 36^4 = 177{,}439{,}616$$

distinct plates in total. Note how little the second option adds: the moose plates are fewer than 1% of the standard ones, because four characters is three fewer than seven and each character is worth a factor of 26 or 10.

1.3.2 Factorials and Overcounting

Example 1.3.6 (Shuffling a deck)**.** How many orderings does a deck of 52 cards have? Apply the product rule to the positions: 52 cards could come first, then 51, then 50, and so on, giving

$$52 \cdot 51 \cdots 2 \cdot 1 = 52! \approx 8.07 \times 10^{67}.$$

Definition 1.3.7 (Factorial)**.** $n! = n(n-1)\cdots 2 \cdot 1$, with $0! = 1$.

Now change the question. Suppose we care only about the *ranks* of the cards in order and not their suits, so that the four twos are interchangeable. How many orderings are distinguishable now?

The count 52! is wrong, but not by an unknown amount, and the reason is a Cartesian product. Specifying a full ordering of the deck amounts to two independent choices:

- an ordering of the 52 *ranks*, four twos and four threes and so on, which is what we now care about; and
- an allocation of suits to the positions holding each rank, which is what we have decided to ignore.

By the product rule, 52! is the size of that product, so

$$52! \;=\; (\text{rank orderings}) \times (\text{suit allocations}).$$

The second factor is the same for every rank ordering. Within each of the 13 ranks the four suits can be attached to that rank's four positions in 4! ways, and the ranks are handled independently, so there are $(4!)^{13}$ allocations. That is the factor by which each rank ordering has been counted, so dividing it out leaves

$$\frac{52!}{(4!)^{13}} \approx 9.20 \times 10^{49}.$$

Figure 1.6 shows the whole picture on a deck of two ranks, where the classes hold $(4!)^2 = 576$ orderings each and there are 70 of them. At full size the classes hold $(4!)^{13} \approx 8.8 \times 10^{17}$ orderings each, which is how much too long the list of 52! orderings is.

The idea to keep

Overcount deliberately, then divide by the size of the overcount. It works whenever every object of interest is counted the same number of times, which is worth checking rather than assuming: the next subsection uses it again to get the binomial coefficient, and Section 1.3.4 is what to do when it fails.

1.3.3 Permutations and Combinations

Definition 1.3.8 (k-permutation)**.** A *k-permutation* of an n-element set is an ordered selection of k distinct elements. The number of these is

$$P(n,k) = n(n-1)\cdots(n-k+1) = \frac{n!}{(n-k)!}.$$

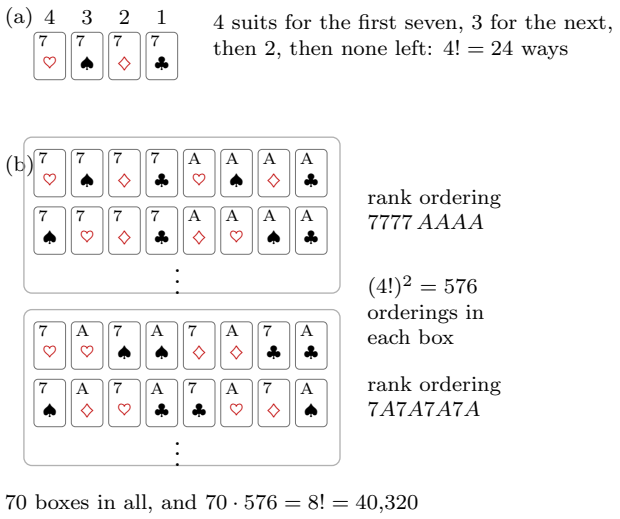


Figure 1.6: A deck of two ranks, four sevens and four aces. In (a) the four suits are attached to the four sevens one position at a time, with the pool shrinking, so a single rank admits 4! allocations; the aces admit another 4! independently, giving $(4!)^2 = 576$. In (b) each box collects the orderings sharing one rank ordering, and every box holds those same 576. The boxes partition the 8! orderings, so there are $8!/(4!)^2 = 70$ rank orderings. A real deck has 13 ranks rather than two, so its boxes hold $(4!)^{13}$ orderings each.

The formula is the product rule with a shrinking pool: n choices for the first position, $n-1$ for the second, and so on. Taking $k = n$ gives $n!$ orderings of the whole set.

Definition 1.3.9 (Combination)**.** A *k-combination* is an unordered selection of k distinct elements, i.e. a k-element subset. The number of these is written $\binom{n}{k}$.

Theorem 1.3.10. $\displaystyle\binom{n}{k} = \frac{n!}{k!\,(n-k)!}.$

Proof. Count k-permutations in two stages: first choose which k elements to use, then order them. Every k-permutation arises exactly once this way, so $P(n,k) = \binom{n}{k} \cdot k!$. Solving for $\binom{n}{k}$ and substituting $P(n,k) = n!/(n-k)!$ gives the formula. □

The proof did not count the combinations directly. It counted the ordered selections, which are easier, then noticed that each unordered selection had been counted $k!$ times, once for each way of ordering it, and divided. This is often a very useful strategy!

Remark 1.3.11 (A quick upper bound). The proof also gives a bound that is often all we need. Since $\binom{n}{k} = P(n,k)/k!$, and $P(n,k) = n(n-1)\cdots(n-k+1)$ is a product of k factors each at most n,

$$\binom{n}{k} \;\le\; \frac{n^k}{k!} \;\le\; n^k.$$

When k is small compared with n, every factor is nearly n, so the first bound is close to exact. Section 4.2 uses this to count the possible copies of a small structure inside a large graph.

Looking ahead

Double counting has a direct descendant in Section 4.1. Counting the endpoints of the edges of a graph in two ways, once by edges and once by vertices, gives $\sum_v \deg(v) = 2\,|E|$, the handshake lemma. It has another in Section 2.3, where *linearity of expectation* lets us count a random quantity by summing indicator variables, one per object, rather than enumerating outcomes.

1.3.4 Inclusion/Exclusion

This is the other column of the table in Section 1.3.1. The overcount here comes from splitting a set into pieces that overlap rather than from inventing a spurious choice, so the repair is subtraction rather than division, and it cannot be otherwise: an element of two of the sets is counted twice while an element of three is counted three times, so there is no single factor to divide by. The correction has to be made case by case, and that is what this subsection is.

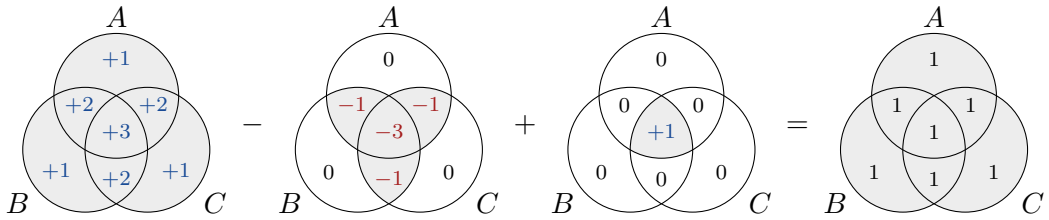
Proposition 1.1.5 told us $|A \cup B|$ can be smaller than $|A| + |B|$, because the overlap is counted twice. Correcting for it is straightforward.

Example 1.3.12. This year's phone comes in black, silver, glacier and burgundy; last year's came in silver, orange and blue. How many colors has the maker offered across the two years? Not $4 + 3$: silver is on both lists and would be counted twice, so the answer is $4 + 3 - 1 = 6$.

Theorem 1.3.13 (Inclusion/exclusion, two sets)**.** $|A \cup B| = |A| + |B| - |A \cap B|$.

Theorem 1.3.14 (Three sets)**.**

$$\begin{aligned} |A \cup B \cup C| = {} & |A| + |B| + |C| \\ & - |A \cap B| - |A \cap C| - |B \cap C| \\ & + |A \cap B \cap C|\,. \end{aligned}$$



$|A| + |B| + |C|$ $\quad$ $|A \cap B| + |A \cap C| + |B \cap C|$ $\quad$ $|A \cap B \cap C|$ $\quad$ $|A \cup B \cup C|$

Figure 1.7: The three terms of inclusion/exclusion, and what each contributes to each region. A region left white receives nothing from that term. Adding across the four panels gives 1 in every region of the union, which is the goal: each element counted exactly once.

Figure 1.7 splits the formula into its three terms and records what each term contributes to each of the seven regions.

The first term reaches every region. A point in one circle is counted once, a point in exactly two circles is counted twice, and the point in all three is counted three times.

The second term reaches only the overlaps. A point in exactly two circles belongs to exactly one of $A \cap B$, $A \cap C$, $B \cap C$, so it loses one count. The center belongs to all three pairwise intersections, so it loses three.

The third term reaches only the center, and returns one count to it.

Now add down each region. The single regions get $1 + 0 + 0$. The pairwise regions get $2 - 1 + 0$. The center gets $3 - 3 + 1$. Every one of them lands on 1, and the alternating signs are the record of that over-correction and repair.

Double counting. That verification is an instance of a technique in its own right: count one set in two ways and equate the answers. Here the two ways are by region and by term, and forcing them to agree is what fixes the signs. Elsewhere the two counts are of different shapes, and equating them yields an identity; sorting the subsets of an n-element set by size and comparing with Theorem 1.1.9 is the example to try, and it is on the problem set.

The pattern continues: add the sizes of all single sets, subtract all pairwise intersections, add all triples, and so on with alternating signs.

Theorem 1.3.15 (Inclusion/exclusion, n sets)**.** *For finite sets $A_1, \dots, A_n$,*

$$\Big|\bigcup_{i=1}^{n} A_i\Big| = \sum_{\emptyset \neq J \subseteq \{1,\dots,n\}} (-1)^{|J|+1} \Big|\bigcap_{i \in J} A_i\Big|.$$

Equivalently, grouping the terms by the size of J,

$$\Big|\bigcup_{i=1}^{n} A_i\Big| = \sum_{k=1}^{n} (-1)^{k+1} \sum_{|J|=k} \Big|\bigcap_{i \in J} A_i\Big|,$$

so the kth group has $\binom{n}{k}$ terms and carries the sign $(-1)^{k+1}$.

We will not prove this. The argument is the region-by-region check of Figure 1.7 carried out for n sets at once: fix an element lying in exactly k of the A_i, count how many terms include it, and check that the signed total is 1.

Looking ahead

The probability version is the same statement with $|\cdot|$ replaced by $\Pr[\,\cdot\,]$, and we will state it in Section 2.1 without a new proof. Dropping all the correction terms gives the union bound, which is loose but requires no knowledge of the intersections at all, a trade we will make often, and which is important for the percolation argument of Section 4.2.

1.3.5 A First Coding Bound

Counting can prove that an engineering target is unreachable, and the following is the example this course will use twice.

A *code* of length n is a set $C \subseteq \{0,1\}^n$ of strings, called codewords, of which we send one to convey a message.

Example 1.3.16. Take $n = 3$ and $C = \{000, 111\}$, so one of two messages is sent as three bits. If 010 arrives, it is not a codeword, and it differs from 000 in one position and from 111 in two, so the receiver corrects it to 000. Any single flip can be undone this way, because the two codewords are three flips apart and a single flip leaves a string nearer to the one that was sent.

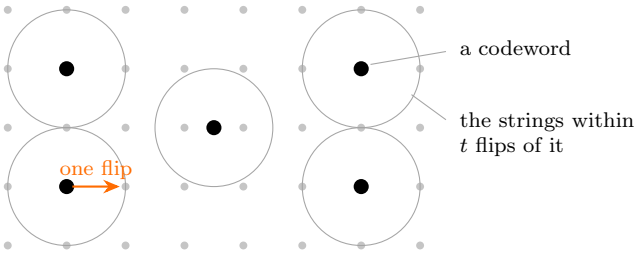


Figure 1.8: The bound as a packing problem. Every string of length n is a point, each codeword claims the ball of strings within t flips of it, and those balls must not overlap or a received string would have two equally good explanations. The number of codewords is therefore at most the number of points divided by the size of one ball.

In general, if the channel flips at most t of the n bits, the receiver can still identify what was sent provided no two codewords are within $2t$ flips of each other: any two codewords differ in at least $2t + 1$ positions.

Theorem 1.3.17 (Sphere-packing bound)**.** *If any two codewords of $C \subseteq \{0,1\}^n$ differ in at least $2t + 1$ positions, then*

$$|C| \cdot \sum_{i=0}^{t} \binom{n}{i} \;\leq\; 2^n.$$

Proof. Fix a codeword c and count the strings that differ from it in at most t positions. A string differing in exactly i positions is obtained by choosing which i of the n positions to flip, so there are $\binom{n}{i}$ of them, and the choices of i are disjoint. By the sum rule the strings within t flips of c number $\sum_{i=0}^{t} \binom{n}{i}$.

These collections, one per codeword, are disjoint. Suppose some string x lay within t flips of two codewords $c \neq c'$. Changing c into x takes at most t flips and changing x into c' takes at most t more, so c and c' differ in at most $2t$ positions, contrary to assumption.

So we have $|C|$ disjoint collections of equal size $\sum_{i=0}^{t} \binom{n}{i}$, all inside the set of 2^n strings of length n. The sum rule gives the inequality. □

This can be interpreted as a "budget" that no clever coding construction can beat, and Figure 1.8 is the picture of it: the balls claim disjoint territory inside a space of fixed size.

Example 1.3.18. Take $n = 7$ and $t = 1$, a block of seven bits that survives any single flip. A ball has $\binom{7}{0} + \binom{7}{1} = 8$ strings, so

$$|C| \leq \frac{2^7}{8} = 16.$$

At most 16 messages, which is 4 bits of data in a 7-bit block. No cleverness can beat that, and Section 3.3 constructs a code that attains it exactly.

Chapter 2

Probability

Probability begins where Chapter 1 left off. The objects we assign probabilities to are sets, the rules for combining them are the set operations of Section 1.1, and in the simplest and most common case the computation of a probability reduces to a counting problem of the kind we solved in Section 1.3. What is new is a single function, Pr, that takes a set and returns a number.

2.1 Probability

2.1.1 Sample Spaces and Events

Definition 2.1.1 (Sample space, outcome, event)**.** An experiment has a *sample space* Ω, the set of all possible outcomes. An individual outcome is written $\omega \in \Omega$. An *event* is a subset $A \subseteq \Omega$. We say A *occurs* if the outcome that actually happens belongs to A.

This is the universe U of Section 1.1 under a new name. The vocabulary changes because the questions change, but the algebra is identical: $A \cup B$ is the event that at least one of the two occurs, $A \cap B$ that both do, and A^c that A does not.

Example 2.1.2. Flip a coin twice. Then $\Omega = \{HH, HT, TH, TT\}$. The event "at least one head" is $A = \{HH, HT, TH\}$, and the event "the two flips agree" is $B = \{HH, TT\}$. Here $A \cap B = \{HH\}$ and $A^c = \{TT\}$.

Outcomes and events come with a second pair of names, borrowed from statistical mechanics, which we use whenever the outcomes are configurations of something. An outcome ω is a complete description of what happened, down to which face each die showed and which side each coin landed on; call it a *microstate*. An event is usually the set of all the ways some coarser description could be realized, such as that the sum was seven or that three of the coins came up heads; call that a *macrostate*. The words are only a relabelling of ω and A, and they carry no assumption about which outcomes are likely (though a macrostate is always at least as likely as any microstate it contains).

2.1.2 Positivity, Unit Measure, Additivity

Definition 2.1.3 (Probability measure)**.** A *probability measure* Pr assigns a number $\Pr[A]$ to each event $A \subseteq \Omega$ subject to three requirements:

(i) **Positivity.** $\Pr[A] \geq 0$ for every event A.

(ii) **Unit measure.** $\Pr[\Omega] = 1$.

(iii) **Additivity.** If $A_1, A_2, \ldots$ are pairwise disjoint, then $\Pr[\bigcup_i A_i] = \sum_i \Pr[A_i]$.

Remark 2.1.4 (Why $\Pr[\,\cdot\,]$ and not $P(\cdot)$). Many texts write $P(A)$ or $\mathbb{P}(A)$ for a probability. We write $\Pr[A]$, upright and with square brackets, because the letter P is already spoken for twice in this course: $P(n,k)$ counts k-permutations, as in Section 1.3, and P is the transition matrix of a Markov chain throughout Chapter 6, where expressions such as πP^n appear on nearly every line. Keeping probability visually distinct from P avoids a collision in the places it would do the most damage. Expectation is $\mathbb{E}[\,\cdot\,]$ and variance $\mathrm{Var}(\cdot)$.

2.1.3 Interpreting Probability

The axioms say how probabilities must behave together but not what any one of them means, and there are two standard answers.

The *frequentist* reading is that $\Pr[A] = 0.3$ describes a repeatable experiment: run it many times under the same conditions and A happens in about three tenths of the runs. Probability is then a property of the setup, like a resistance or a melting point, and it is measured by repetition. On this reading there is no such thing as the probability that a particular bridge is sound, because that bridge is not a population of bridges; it is sound or it is not.

The *Bayesian* reading is that $\Pr[A] = 0.3$ describes your information: among the ways the world might be, given what you know, A holds in the 0.3 portion of them. The randomness is in your knowledge rather than in the experiment, so the one-off questions are allowed, and the sample space becomes a set of possible worlds rather than a set of repetitions.

Both readings obey the same three axioms, so the mathematics below is the same either way, and each reading is natural somewhere. A channel that flips each bit with probability p, as in Section 3.3, is repeatable, and the reading there is frequentist. A positive test result that should make you believe you are ill, as in Section 2.2, concerns an illness you either have or do not, and the reading there is Bayesian, which is also why the graphical models of Section 8.1 are called "Bayesian Networks."

The shorthand worth remembering is that a frequentist is describing data and a Bayesian is describing a model: one asks how often the world does something, the other how sure we are about a world we cannot see.

2.1.4 Common Manipulations

The following consequences get used often enough that we derive them here and quote them freely afterwards.

Proposition 2.1.5. *For any events $A, B \subseteq \Omega$:*

(a) $\Pr[\emptyset] = 0$.

(b) $\Pr[A^c] = 1 - \Pr[A]$.

(c) If $A \subseteq B$ then $\Pr[A] \le \Pr[B]$, and $\Pr[A] \le 1$ for every A.

(d) $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$.

(e) $\Pr[A \cup B] \le \Pr[A] + \Pr[B]$.

Proof. For (b), the events A and A^c are disjoint with union Ω, so additivity and unit measure give $\Pr[A] + \Pr[A^c] = 1$. Taking $A = \Omega$ gives (a).

For (c), write $B = A \cup (B \setminus A)$, a union of disjoint sets, so $\Pr[B] = \Pr[A] + \Pr[B \setminus A] \ge \Pr[A]$ by positivity. Since every A satisfies $A \subseteq \Omega$, we get $\Pr[A] \le 1$.

For (d), split $A \cup B$ into the three disjoint pieces $A \setminus B$, $A \cap B$, and $B \setminus A$, and observe that $\Pr[A] + \Pr[B]$ counts the middle piece twice. Part (e) then follows from positivity of $\Pr[A \cap B]$. □

Part (b) deserves a note of its own. A great many probability computations are easiest to perform by finding the chance that the thing in question *fails* to happen and subtracting from one, and combined with De Morgan's laws it becomes the identity that does more work than any other in applied probability. Since $(A_1 \cup \cdots \cup A_n)^c = A_1^c \cap \cdots \cap A_n^c$ by Theorem 1.1.3,

$$\Pr[\text{at least one } A_i \text{ occurs}] = 1 - \Pr[\text{no } A_i \text{ occurs}].$$

Unions are awkward because their pieces overlap, while the intersection on the right is often a single easy quantity. We use this on the birthday problem below, and again in Section 2.2 where independence turns the intersection into a product.

Part (d) is the two-set inclusion/exclusion of Section 1.3 with cardinality replaced by probability, and the general version transfers in exactly the same way:

$$\Pr\left[\bigcup_{i=1}^{n} A_i\right] = \sum_{\emptyset \neq J \subseteq \{1,\ldots,n\}} (-1)^{|J|+1} \Pr\left[\bigcap_{i \in J} A_i\right].$$

Part (e) is the *union bound*, which we obtain by discarding all the correction terms. It is usually loose, and it asks nothing at all about how the events overlap. That combination makes it one of the most used inequalities in the course.

2.1.5 Equally Likely Outcomes

Proposition 2.1.6. *If Ω is finite and every outcome is equally likely, then for any event A,*

$$\Pr[A] = \frac{|A|}{|\Omega|}.$$

Proof. Let $|\Omega| = N$ and let each outcome have probability q. The singleton events are disjoint with union Ω, so additivity and unit measure give $Nq = 1$, hence $q = 1/N$. Applying additivity to A, which contains $|A|$ singletons, gives $\Pr[A] = |A|/N$. □

Under this assumption, every probability question becomes two counting questions. This is why I forced all of you to learn to count.

Example 2.1.7. Roll two fair dice, so $|\Omega| = 36$. The event "the sum is at least 10" consists of $\{(4,6),(5,5),(6,4),(5,6),(6,5),(6,6)\}$, so its probability is $6/36 = 1/6$.

In the language of microstates and macrostates, the proposition says that a macrostate's probability is the fraction of the microstates that realize it. In physics the macrostates one actually observes are the ones with many microstates, which is what high entropy means. In fact, this behavior dominates in *high temperature* systems, where the Boltzmann distribution becomes nearly equally probable. So a macrostate is likely exactly when there are many ways to realize it, and Figure 2.1 shows the smallest interesting case. Without the equally likely assumption a macrostate is still a set of microstates, but their probabilities have to be added rather than counted.

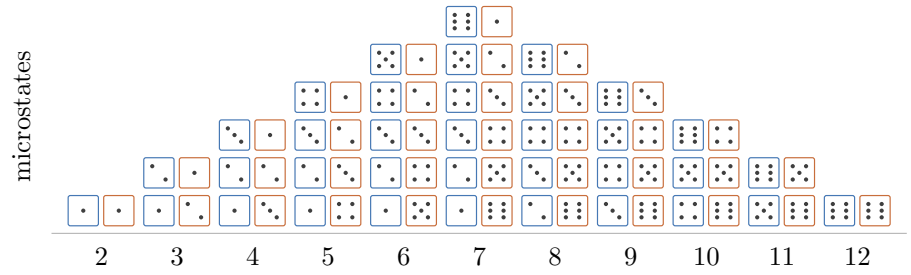


Figure 2.1: Two dice, with every microstate drawn. Each cell is one ordered pair, the first die in blue and the second in orange, and the cells are stacked over the sum they produce, so the height of a column *is* the number of microstates in that macrostate. A sum of 7 happens in six ways and a sum of 2 in one, so 7 is six times as likely. Note that $(2, 5)$ and $(5, 2)$ are different microstates, which is why the middle columns are tall. The shape of this picture is the reason the dice game is interesting and, in a form we meet in Section 7.2, the reason sums of random things tend to look bell-shaped.

Looking ahead

This distinction runs through three later lectures. In Section 3.1 we measure the uncertainty of a macrostate by the logarithm of the number of microstates it contains, which is Boltzmann's $S = k \log W$ and, in the version we will use, entropy. In Section 6.5 a diffusing gas is modeled by a chain whose macrostate is the number of molecules on one side. With N molecules there are 2^N microstates and $\binom{N}{k}$ of them put k on the left, and the equilibrium distribution turns out to be exactly that: the gas is found in the macrostate with the most microstates. And the concentration results of Section 2.5 are what make "overwhelmingly more likely" quantitative.

Example 2.1.8 (The birthday problem)**.** Suppose n people have birthdays drawn independently and uniformly from 365 days. What is the chance that two of them share a birthday?

The event is a union over pairs, so compute the complement instead: by Theorem 1.1.3, a shared birthday somewhere fails exactly when every pair differs. There are 365^n equally likely assignments of birthdays, and the number in which all n birthdays differ is the number of n-permutations of 365 days, namely $P(365, n)$ from Section 1.3. So

$$\Pr[\text{all distinct}] = \frac{P(365, n)}{365^n} = \frac{365}{365} \cdot \frac{364}{365} \cdots \frac{365 - n + 1}{365},$$

and the probability of a shared birthday is one minus this. At $n = 23$ the product falls just below $1/2$, so with 23 people a match is already more likely than not. At $n = 50$ the chance of a match is about 97%.

The same calculation governs hash collisions, with 2^k hash values in place of 365 days, which is the first thing the problem set asks for.

The birthday problem surprises people because they substitute a different question: the chance that someone shares *their* birthday, which grows like $n/365$. The chance of some shared birthday grows with the number of *pairs*, which is $\binom{n}{2}$ and therefore quadratic in n. We return to this in Section 2.3 with a method that makes the $\binom{n}{2}$ visible from the start.

The next example is the argument that bundling mortgages makes them safe. It is worth following carefully, because the reasoning is correct and the conclusion was wrong.

Example 2.1.9 (Why bundle mortgages?)**.** A bank holds n mortgages, each of which either defaults or does not. Take every default pattern to be equally likely, so there are 2^n microstates and each mortgage defaults with probability $1/2$. The bank fails if 70% or more of its mortgages default.

The macrostate "exactly k defaults" contains $\binom{n}{k}$ microstates, so

$$\Pr[\text{bank fails}] = \frac{1}{2^n} \sum_{k \geq 0.7n} \binom{n}{k}.$$

With $n = 10$ the bank fails on seven or more defaults, and

$$\binom{10}{7} + \binom{10}{8} + \binom{10}{9} + \binom{10}{10} = 120 + 45 + 10 + 1 = 176,$$

so the probability is $176/1024 = 0.172$. With $n = 100$ the same sum from $k = 70$ upwards gives 3.9×10^{-5}, and with $n = 1000$ it is 9×10^{-38}.

Nothing about the individual mortgages improved. Each is still a coin flip. What changed is that there are far more ways to achieve default rates near 50% and hardly any microstates stray as far as 70%. Bundling makes the *average* predictable.

Every step of that argument is sound, and the conclusion was still wrong. The weak point is the phrase "every default pattern equally likely," which says nothing about any individual mortgage and everything about how they move together. What that phrase is doing, and what happens without it, is the subject of the next lecture.

2.2 Conditioning

Lecture 4 left a calculation hanging. Bundling a hundred mortgages, each a fair coin, drove the chance of losing 70% of them down to 3.9×10^{-5}, and the whole of that conclusion rested on treating every default pattern as equally likely. Here is the same bundle with that assumption removed.

Example 2.2.1 (The same mortgages, bundled badly)**.** Keep n mortgages each defaulting with probability $1/2$, and now suppose they default *together*: with probability $1/2$ every mortgage defaults, and with probability $1/2$ none does. Each individual mortgage is exactly as before, a fair coin, and any single one of them would look identical to a lender examining it on its own.

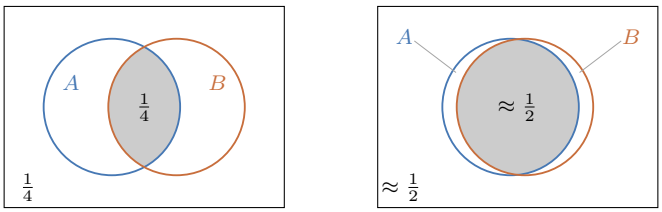
But now only two microstates have any probability at all, and one of them is total default, so

$$\Pr[\text{bank fails}] = \tfrac{1}{2}$$

for every n. Bundling accomplishes nothing whatever. At $n = 100$ this is roughly 13,000 times the figure computed above.

Figure 2.2 shows the two situations for a pair of mortgages. The marginal probabilities, the areas of the two circles, are the same in both pictures. What differs is the overlap, which entirely determines the probability of co-occurring defaults.

Whether the overlap is $\frac{1}{2}$ or not is a question about *dependence*, and that is the subject of this lecture. The mortgages of 2006 were rated as though they were independent events. They were bundled by the thousand, and the bundles were rated on the strength of a similar calculation to the one in Example 2.1.9. When house prices fell they fell everywhere at once, which is the right picture, and the bundles behaved like Example 2.2.1.



(a) independent
$\Pr[A \cap B] = \frac{1}{4}$

(b) moving together
$\Pr[A \cap B] \approx \frac{1}{2}$

Figure 2.2: Two mortgages, A and B, each defaulting with probability $\frac{1}{2}$, so each circle covers half the square in both pictures. On the left they are independent and the overlap is a quarter, leaving a quarter of the square outside both. On the right they nearly coincide: both default together about half the time and neither defaults the other half, with almost nothing in between. No examination of A alone, or of B alone, distinguishes these two worlds.

Why failures arrive together rather than separately is a question in its own right, and Section 4.2 answers it in one context: when failures can spread from one unit to its neighbours, there is a threshold below which they stay local and above which they span the whole system. The chain reaction of mortgage defaults that sent the 2009 economy into a spiral is not dissimilar from the mathematics of shattering and pandemics that we study later in the course.

For now, the key feature we want to study is how learning that one event occurred changes the chances of the others. This lecture defines that mechanism and turns it into a technique: first the case where conditioning changes nothing, then the three results we use to build complicated probabilities out of simple conditional pieces.

2.2.1 Conditional, Joint, and Marginal Probability

Learning that event A occurred changes our universe $\Omega \to A$, since all other possible worlds are now impossibilities. This, in turn, can shift other probabilities to the new fraction of this reduced universe that they occupy.

Definition 2.2.2 (Conditional probability)**.** For events A and B with $\Pr[B] > 0$, the *conditional probability* of A given B is

$$\Pr[A \mid B] = \frac{\Pr[A \cap B]}{\Pr[B]}.$$

Figure 2.3 shows what the definition does. Conditioning keeps the part of A that lies inside B and then renormalizes so that B itself has probability 1. One can check directly that for fixed B, the function $A \mapsto \Pr[A \mid B]$ satisfies all three axioms, so a conditional probability is a probability measure in its own right and everything proved above applies to it.

Example 2.2.3. Roll two fair dice. Let A be the event that the sum is at least 10 and B the event that the first die shows 6. Then $\Pr[A] = 1/6$ and $\Pr[B] = 1/6$, while $A \cap B = \{(6,4),(6,5),(6,6)\}$ has probability $3/36$. So

$$\Pr[A \mid B] = \frac{3/36}{6/36} = \frac{1}{2},$$

considerably larger than $\Pr[A]$. Knowing the first die is a six helps.

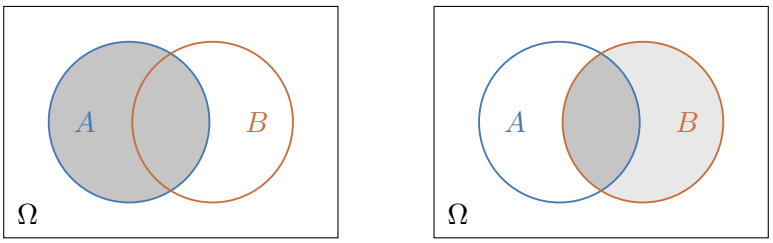


Figure 2.3: Conditioning on B. On the left, $\Pr[A]$ measures A against the whole sample space. On the right, we are told that B occurred, so B becomes the sample space: the part of A outside B is discarded, and what remains is measured against B rather than against Ω. Dividing by $\Pr[B]$ is what rescales the smaller universe back to total probability 1. The simplest check on the definition is $\Pr[B \mid B] = \Pr[B]/\Pr[B] = 1$: once B is known, B is certain.

When several quantities are involved, the probability of a full specification, such as $\Pr[\text{first} = 6, \text{ second} = 4]$, is called a *joint* probability. The probability of a partial specification, obtained by summing the joint probabilities over the unspecified parts, is called a *marginal* probability. Conditional probabilities relate the two, as the definition above makes explicit.

The word comes from the habit of writing a joint distribution as a table and recording the row and column sums along its edges:

	A	A^c	
B	$\Pr[A \cap B]$	$\Pr[A^c \cap B]$	$\Pr[B]$
B^c	$\Pr[A \cap B^c]$	$\Pr[A^c \cap B^c]$	$\Pr[B^c]$
	$\Pr[A]$	$\Pr[A^c]$	1

The four interior entries are the joint probabilities and the numbers in the margins are $\Pr[A]$ and $\Pr[B]$, each obtained by summing a row or a column. Marginalizing means collapsing the table onto one of its edges.

2.2.2 Independence

Definition 2.2.4 (Independence)**.** Events A and B are *independent* if $\Pr[A \cap B] = \Pr[A]\Pr[B]$, written $A \perp B$.

Figure 2.4 is the condition as a picture. When $\Pr[B] > 0$, dividing both sides by $\Pr[B]$ shows that independence is equivalent to $\Pr[A \mid B] = \Pr[A]$: learning B tells you nothing about A. The product form is the official definition because it is symmetric in A and B and stays meaningful when one of them has probability zero.

> *Remark* 2.2.5. Independent and disjoint are unrelated conditions, and confusing them is the most common early error. If A and B are disjoint and both have positive probability, then $\Pr[A \mid B] = 0$ while $\Pr[A] > 0$, so they are as far from independent as events can be. Disjointness says the two cannot happen together; independence says that whether one happens carries no information about the other.

Definition 2.2.6 (Mutual independence)**.** Events $A_1, \ldots, A_n$ are *mutually independent* if for every subset $J \subseteq \{1, \ldots, n\}$,

$$\Pr\left[\bigcap_{i \in J} A_i\right] = \prod_{i \in J} \Pr[A_i].$$

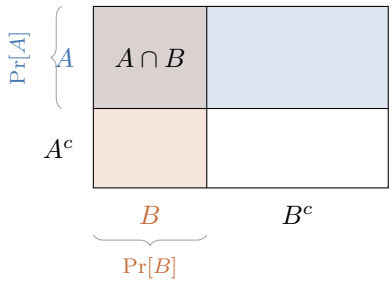


Figure 2.4: Independence as a picture. Cut the sample space into the column B and the row A. The two events are independent exactly when the shaded corner takes up the same fraction of the B column as the A row takes up of the whole square, which is what $\Pr[A \cap B] = \Pr[A]\Pr[B]$ says. Reading it down the column gives $\Pr[A \mid B] = \Pr[A]$: restricting attention to B changes nothing about A.

The definition asks for every subset because nothing weaker implies the rest: three events can be independent in every pair and still fail the condition for the triple. This is a worthwhile exercise to think about if you have some extra time!

Independence is also a tool, not only a hypothesis to be checked. The following trick is due to von Neumann.

Example 2.2.7 (A fair bit from an unfair coin)**.** You have a coin with an unknown bias (maybe a rock with two sides that you picked up off the ground!). How do you get a perfectly fair coin out of it?

Flip it twice, and treat the two flips as independent. Report 1 on HT, report 0 on TH, and on HH or TT report nothing and start over. By independence $\Pr[HT] = p(1-p)$ and $\Pr[TH] = (1-p)p$, and these are equal whatever p is, so

$$\Pr[HT \mid HT \cup TH] = \Pr[TH \mid HT \cup TH] = \tfrac{1}{2}.$$

The bias cancels because the two orderings are built from the same two factors, and p never has to be known or even estimated. This is how physical randomness gets turned into usable bits: a noisy diode or a thermal source has an unknown and drifting bias, and a debiasing step of this kind sits between it and anything that needs fair coins, cryptographic keys most of all.

The cost is the discarded flips, which Section 2.4 lets us count, and how much of the coin's randomness the procedure wastes is a question for Section 3.1.

Independence is also what lets the set identities of Lecture 1 be used on probabilities.

Example 2.2.8 (How good must a part be?)**.** A car has $n = 10{,}000$ parts, all of which must work. Suppose each part is defective with probability p, independently of the others, and let D_i be the event that part i is defective. The car has a defect on $D_1 \cup \cdots \cup D_n$, a union of ten thousand events with every intersection to worry about.

De Morgan turns it into something manageable. The complement of the union is the intersection of the complements, and independence factorizes that:

$$\Pr[\text{no defect}] = \Pr\left[\bigcap_i D_i^c\right] = \prod_i \Pr[D_i^c] = (1-p)^n.$$

At $p = 10^{-4}$, which is one defect in ten thousand and sounds like a well-run factory, $(1-10^{-4})^{10000} = 0.368$: nearly 63% of cars leave with a defect. Keeping 99% of cars clean

needs p below 10^{-6}, one defect per million parts, which is why automotive suppliers are held to rates quoted in parts per million rather than percentages.

The union bound of Proposition 2.1.5 gives $\Pr[\text{defect}] \le np$, which requires no independence at all. Equivalently $\Pr[\text{no defect}] = (1-p)^n \ge 1 - np$, which is Bernoulli's inequality, so the union bound and that algebraic fact are the same statement. At $p = 10^{-4}$ it reads $np = 1$ and says nothing. At $p = 10^{-5}$ it gives 0.10 against an exact 0.0952, and at $p = 10^{-6}$ it gives 0.0100 against 0.00995. The bound is worth using exactly when np, the expected number of defects, is small, and that is the regime a designer has to be in anyway.

2.2.3 The Chain Rule

Rearranging the definition of conditional probability gives

$$\Pr[A \cap B] = \Pr[B]\,\Pr[A \mid B],$$

which is more useful than the original form, because in practice the conditional probabilities are often the quantities we know.

Theorem 2.2.9 (Chain rule)**.** *For any events $A_1, \dots, A_n$ with $\Pr[A_1 \cap \cdots \cap A_{n-1}] > 0$,*

$$\Pr[A_1 \cap \cdots \cap A_n] = \Pr[A_1]\,\Pr[A_2 \mid A_1]\,\Pr[A_3 \mid A_1 \cap A_2] \cdots \Pr[A_n \mid A_1 \cap \cdots \cap A_{n-1}].$$

Proof. Each factor is a ratio of consecutive joint probabilities, $\Pr[A_1 \cap \cdots \cap A_k] / \Pr[A_1 \cap \cdots \cap A_{k-1}]$. The product telescopes, leaving $\Pr[A_1 \cap \cdots \cap A_n]$. □

Example 2.2.10 (A bursty channel)**.** Bits travel down a link that corrupts them, and corruption comes in bursts: a bit is corrupted with probability 0.01 normally, but with probability 0.3 if its predecessor was corrupted. Those two conditional numbers are the whole model; there is no uniform collection of outcomes to count.

The chain rule gives the probability that the first three bits are all corrupted:

$$\Pr[C_1 \cap C_2 \cap C_3] = \Pr[C_1]\,\Pr[C_2 \mid C_1]\,\Pr[C_3 \mid C_1 \cap C_2] = (0.01)(0.3)(0.3) = 9 \times 10^{-4},$$

where the last factor drops the condition on C_1 because only the immediately preceding bit matters. Had we assumed the bits were corrupted independently, we would have computed $(0.01)^3 = 10^{-6}$ and understated the risk by a factor of 900.

That last simplification, where conditioning on the whole past reduces to conditioning on the most recent event, is the *Markov property*, and Chapter 6 is about processes that have it. The chain rule turns such a process into a product, and it is also how the graphical models of Section 8.1 factorize.

2.2.4 The Law of Total Probability

Definition 2.2.11 (Partition)**.** Events $B_1, \dots, B_k$ *partition* Ω if they are pairwise disjoint and their union is Ω.

Theorem 2.2.12 (Law of total probability)**.** *If $B_1, \dots, B_k$ partition Ω and each has positive probability, then for any event A,*

$$\Pr[A] = \sum_{i=1}^{k} \Pr[A \cap B_i] = \sum_{i=1}^{k} \Pr[A \mid B_i]\,\Pr[B_i].$$

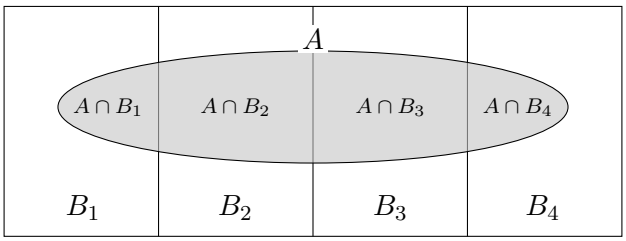


Figure 2.5: The law of total probability. The four strips partition the universe, so the event A is cut into the four disjoint slices $A \cap B_i$. Additivity says the probability of A is the sum of the slices, and the chain rule rewrites each slice as $\Pr[A \mid B_i] \Pr[B_i]$.

This is marginalization, which we introduced in Section 2.2.1: the first sum collapses the joint distribution of A and the B_i onto its A margin, and the second expands each joint term by the definition of conditional probability. The two readings are worth keeping apart in your head. Marginalizing is what you do when the joint probabilities are what you have; total probability is what you do when the conditionals are what you have, which in practice is more often.

Proof. The sets $A \cap B_i$ are pairwise disjoint, and their union is A because the B_i cover Ω. Additivity gives the first equality and the chain rule gives the second. □

Figure 2.5 is the whole proof in one picture. The practical reading is that a hard unconditional probability can be assembled from easy conditional ones, provided we can find a partition on which the conditional behavior is simple. Choosing that partition well can take some practice.

2.2.5 Bayes' Rule

The chain rule can be applied to $A \cap B$ in either order, which gives $\Pr[A \mid B] \Pr[B] = \Pr[B \mid A] \Pr[A]$. Solving for one conditional probability in terms of the other is Bayes' rule.

Theorem 2.2.13 (Bayes' rule)**.** *If* $\Pr[A] > 0$ *and* $\Pr[B] > 0$*, then*

$$\Pr[A \mid B] = \frac{\Pr[B \mid A] \Pr[A]}{\Pr[B]},$$

and if $B_1, \ldots, B_k$ *partition* Ω*,*

$$\Pr[B_j \mid A] = \frac{\Pr[A \mid B_j] \Pr[B_j]}{\sum_{i=1}^{k} \Pr[A \mid B_i] \Pr[B_i]}.$$

The second form is the first with the law of total probability substituted into the denominator, which is usually how the denominator gets computed. In the standard terminology $\Pr[B_j]$ is the *prior*, $\Pr[A \mid B_j]$ the *likelihood*, $\Pr[B_j \mid A]$ the *posterior*, and the denominator $\Pr[A]$ the *evidence*. Those names come from reading the rule as

$$\Pr[\text{model} \mid \text{data}] = \frac{\Pr[\text{data} \mid \text{model}] \Pr[\text{model}]}{\Pr[\text{data}]},$$

which is the Bayesian interpretation of Section 2.1.3: a belief about the model, held before the data arrived, updated by how well the model accounts for what was seen. If you are wondering how to compute the "evidence," which is the probability of the data under no

particular model, you can use the law of total probability: summing the numerator over every model gives exactly $\Pr[A]$.

Bayes' rule is the machinery for reversing a conditional probability, which is what we want whenever the thing we can observe is a consequence and the thing we want to know is its cause.

Example 2.2.14 (A rapid COVID test)**.** A rapid antigen test detects SARS-CoV-2 in about 73% of infected people with symptoms, and correctly returns a negative for about 99.1% of people without the virus. Suppose 10% of people with those symptoms are infected. Someone tests positive. Are they infected?

Write C for infection and $+$ for a positive test. The law of total probability gives us the denominator,

$$\Pr[+] = \underbrace{(0.73)}_{\Pr[+|C]}\underbrace{(0.10)}_{\Pr[C]} + \underbrace{(0.009)}_{\Pr[+|C^c]}\underbrace{(0.90)}_{\Pr[C^c]} = 0.073 + 0.0081 = 0.0811,$$

and Bayes' rule gives

$$\Pr[C \mid +] = \frac{\overbrace{(0.73)}^{\text{likelihood}}\ \overbrace{(0.10)}^{\text{prior}}}{\underbrace{0.0811}_{\text{evidence}}} \approx 0.90.$$

Now change only the prior. If instead 0.5% of those tested are infected, the same test gives

$$\Pr[C \mid +] = \frac{(0.73)(0.005)}{(0.73)(0.005) + (0.009)(0.995)} \approx 0.29.$$

The test has not changed, and a specificity of 99.1% still looks excellent, but the answer has moved from 90% to 29%. That sensitivity to the prior is the character of Bayesian reasoning, and the reason testing policy distinguished diagnosis from mass screening.

The error this example corrects is systematic rather than random. People asked for $\Pr[D \mid +]$ tend to report something close to $\Pr[+ \mid D]$, and the two differ by a factor of ten here.

Looking ahead

Bayes' rule scales up. When many quantities depend on one another, writing out all the conditional probabilities becomes hopeless, and the way forward is to record which quantities depend on which and exploit the resulting structure. That is the subject of Section 8.1, where the dependencies are drawn as a graph. You don't know what a graph is right now, but you will soon learn about them in Chapter 4!

One further conditioning identity, the *tower rule*, belongs with this group. It requires expectation, so we state and use it in the next lecture.

2.3 Random Variables and Expectation

2.3.1 Random Variables

Definition 2.3.1 (Random variable)**.** A *random variable* is a function $X : \Omega \to \mathbb{R}$. Its *probability mass function*, or pmf, is

$$p_X(x) = \Pr[X = x] = \Pr[\{\omega \in \Omega : X(\omega) = x\}],$$

and the set of x with $p_X(x) > 0$ is the *support* of X.

A random variable is neither random nor a variable: it is a deterministic function that attaches a number to each outcome. The randomness lives in Ω, and X reports a numerical feature of whatever outcome occurs. Each number now carries a probability, because $X = x$ is an event like any other: the set of outcomes on which X takes that value. This lecture treats *discrete* random variables, meaning ones with countable support, so that the pmf carries all the information. Continuous random variables need different tools and wait until Section 7.2.

Example 2.3.2. Flip three fair coins and let X be the number of heads. Then $X(HHT) = 2$, and the pmf is $p_X(0) = 1/8$, $p_X(1) = 3/8$, $p_X(2) = 3/8$, $p_X(3) = 1/8$. The numerators are the binomial coefficients $\binom{3}{k}$ from Section 1.3.

Since the events $\{X = x\}$ for distinct x are disjoint and cover Ω, every pmf satisfies $\sum_x p_X(x) = 1$.

2.3.2 Expectation

Definition 2.3.3 (Expectation)**.** The *expectation* of a discrete random variable X is

$$\mathbb{E}[X] = \sum_x x\, p_X(x),$$

summed over the support, provided the sum converges absolutely (convergence is a math technicality...don't worry about it).

Equivalently, summing over outcomes instead of values,

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} X(\omega)\, \Pr[\omega], \tag{2.1}$$

which is the form that shows expectation is *linear* (which we will discuss next). Mechanically, $\mathbb{E}[X]$ is the average of the values of X weighted by how likely they are, or in physical terms the center of mass of the pmf.

Example 2.3.4. For one fair die, $\mathbb{E}[X] = (1 + 2 + 3 + 4 + 5 + 6)/6 = 3.5$. No roll ever produces 3.5; an expectation need not lie in the support.

2.3.3 Linearity of Expectation

Theorem 2.3.5 (Linearity)**.** *For random variables X and Y on the same sample space and constants a, b,*

$$\mathbb{E}[aX + bY] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y].$$

Proof. Using (2.1),

$$\mathbb{E}[aX + bY] = \sum_{\omega} \big(aX(\omega) + bY(\omega)\big) \Pr[\omega] = a \sum_{\omega} X(\omega) \Pr[\omega] + b \sum_{\omega} Y(\omega) \Pr[\omega],$$

which is $a\mathbb{E}[X] + b\mathbb{E}[Y]$. □

The proof is three lines, and the result is the most useful tool in elementary probability, for one reason: nothing was assumed about how X and Y relate. They may be dependent in any way whatsoever, they need not even be distinct, and the identity still holds. Compare this with variance in Section 2.5, which adds only under independence.

2.3.4 Indicator Random Variables

The indicator function of Section 1.1 becomes a random variable as soon as the set it describes is an event.

Definition 2.3.6 (Indicator)**.** For an event A, the *indicator* $\mathbb{1}_A$ is the random variable equal to 1 when A occurs and 0 otherwise.

Proposition 2.3.7. $\mathbb{E}[\mathbb{1}_A] = \Pr[A]$.

Proof. The support is $\{0, 1\}$ with $p(1) = \Pr[A]$, so $\mathbb{E}[\mathbb{1}_A] = 1 \cdot \Pr[A] + 0 \cdot \Pr[A^c] = \Pr[A]$. □

Proposition 2.3.7 combined with linearity gives a method. Whenever a quantity of interest counts how many things of some kind occur, write it as a sum of indicators, one per thing, and add up their probabilities:

$$X = \sum_{i=1}^{n} \mathbb{1}_{A_i} \qquad \Longrightarrow \qquad \mathbb{E}[X] = \sum_{i=1}^{n} \Pr[A_i].$$

The events A_i are usually heavily dependent, and linearity does not care.

Set that beside the union bound of Proposition 2.1.5, which has the same right-hand side:

$$\mathbb{E}\Big[\sum_i \mathbb{1}_{A_i}\Big] = \sum_i \Pr[A_i], \qquad \Pr\Big[\bigcup_i A_i\Big] \le \sum_i \Pr[A_i].$$

The sum counts the events that occur, and at least one occurs whenever the count is positive, so the union bound is the statement that a probability cannot exceed an expected count. Section 2.5 turns that observation into Markov's inequality and Section 4.2 uses it to show that random graphs have no isolated vertices.

Example 2.3.8 (Shared birthdays, revisited)**.** With n people, let X count the pairs who share a birthday. Index the pairs by the $\binom{n}{2}$ two-element subsets, and let A_{ij} be the event that persons i and j match, so $\Pr[A_{ij}] = 1/365$. Then

$$\mathbb{E}[X] = \binom{n}{2} \frac{1}{365}.$$

At $n = 23$ this is $253/365 \approx 0.69$, and at $n = 50$ it is $1225/365 \approx 3.36$. The quadratic growth that made the birthday problem surprising in Section 2.1 is now the first thing one writes down. Note how much easier this is than the exact probability: the A_{ij} are not independent, since a match between persons $1, 2$ and between $2, 3$ forces one between $1, 3$, and none of that interferes.

Example 2.3.9 (Hat check)**.** n people leave hats at a coat check and receive them back in a uniformly random order. Let X count the people who get their own hat. Person i has chance $1/n$ of a match, so

$$\mathbb{E}[X] = n \cdot \frac{1}{n} = 1,$$

independent of n. Computing the distribution of X is a much harder problem; its expectation takes one line.

The same count governs any assignment in which nobody may be given their own item: shuffling n papers among n reviewers leaves one self-assignment on average, and at least one about 63% of the time, whether n is ten or ten thousand.

2.3.5 Conditional Expectation and the Tower Rule

Definition 2.3.10 (Conditional expectation)**.** For a discrete random variable X and an event B with $\Pr[B] > 0$,

$$\mathbb{E}[X \mid B] = \sum_x x \Pr[X = x \mid B],$$

summed over the support of X. If Y is another random variable, $\mathbb{E}[X \mid Y]$ denotes the random variable whose value is $\mathbb{E}[X \mid Y = y]$ whenever $Y = y$.

The second half of that definition is the subtle part. $\mathbb{E}[X \mid Y]$ is a function of Y, and since Y is random, so is $\mathbb{E}[X \mid Y]$. Being a random variable, it has an expectation of its own.

Theorem 2.3.11 (Tower rule)**.** $\mathbb{E}\big[\mathbb{E}[X \mid Y]\big] = \mathbb{E}[X]$.

Proof. The events $\{Y = y\}$ partition Ω, so

$$\mathbb{E}\big[\mathbb{E}[X \mid Y]\big] = \sum_y \mathbb{E}[X \mid Y = y] \Pr[Y = y] = \sum_y \sum_x x \Pr[X = x \mid Y = y] \Pr[Y = y].$$

By the chain rule the inner product is $\Pr[X = x, Y = y]$, and summing over y leaves $\Pr[X = x]$ by the law of total probability. What remains is $\sum_x x \Pr[X = x] = \mathbb{E}[X]$. □

The tower rule is the law of total probability with probabilities replaced by expectations, and the proof above is that statement plus one extra summation. Used in practice it reads: to find an average, average over cases and then average the case averages, weighted by how likely the cases are.

Example 2.3.12 (A withheld admission rate)**.** Dartmouth admits through two rounds. For the class entering in Fall 2025 it received 3,552 early decision applications and 24,678 in all other rounds, admitted 6.02% of applicants overall, and admitted 3.77% of the applicants in the later rounds. It did not publish the early rate.

Let X indicate admission and Y the round. The rounds partition the applicants, so the tower rule reads

$$\underbrace{0.0602}_{\mathbb{E}[X]} = \underbrace{\left(\tfrac{3552}{28230}\right)}_{\Pr[\text{early}]} \underbrace{(\,?\,)}_{\mathbb{E}[X\mid\text{early}]} + \underbrace{\left(\tfrac{24678}{28230}\right)}_{\Pr[\text{other}]} \underbrace{(0.0377)}_{\mathbb{E}[X\mid\text{other}]},$$

one equation in one unknown. Solving gives an early rate of 21.6%, nearly six times the rate in the later rounds. A blended average does not conceal a round once the other round is known.

Looking ahead

Indicators plus linearity is the engine of the first moment method. In Section 4.2 we bound the expected number of isolated vertices in a random graph exactly this way, and in Section 3.1 entropy itself is defined as an expectation, of the quantity $-\log p$.

2.3.6 Simpson's Paradox

The weights in the tower rule can do more than shift an average. They can reverse it.

In 1973 the University of California, Berkeley was sued for discriminating against women in graduate admissions. The aggregate figures looked damning: 44% of male applicants were admitted against 35% of female applicants. Broken down by department, however, most departments were admitting women at a *higher* rate than men.

Table 2.1 shows how both can hold at once. Women applied mostly to the selective department and men mostly to the unselective one, so each group's overall rate is dominated by the department it favored. Every conditional rate points one way and the blended rate points the other.

Department	Men	Women
Unselective	100 applied, 90% admitted	10 applied, 100% admitted
Selective	10 applied, 10% admitted	100 applied, 20% admitted
Overall	110 applied, 82.7% admitted	110 applied, 27.3% admitted

Table 2.1: A simplified version of the Berkeley figures. Women are admitted at a higher rate in each department and at a far lower rate overall, because the two groups apply in different proportions to departments with very different rates.

This is called *Simpson's paradox*, and it is not a paradox so much as a warning about which average answers which question. Nothing is wrong with either calculation: the conditional rates describe how each department treated its applicants, and the blended rate describes what happened to the applicant pool as a whole. They differ because the weights differ, and the weights are themselves a fact about the world that may need explaining – here, why the two groups applied to different departments at all.

Which one to use is therefore a question about what you are asking, not about arithmetic, and Section 8.1 returns to it with the machinery to say when conditioning on a variable clarifies a comparison and when it corrupts one. This question is the subject of an entire class at Dartmouth, ENGS 105.1, which I teach in the spring.

2.4 Discrete Distributions

A handful of pmfs come up very frequently. All four in this lecture are built from a sequence of independent trials (coin flips) each succeeding with probability p, and their difference stems from what question is asked.

2.4.1 Bernoulli and Binomial

Definition 2.4.1 (Bernoulli)**.** $X \sim \text{Bernoulli}(p)$ if X takes the value 1 with probability p and 0 with probability $1 - p$.

A Bernoulli random variable is an indicator, so $\mathbb{E}[X] = p$ by Proposition 2.3.7. Every distribution below is assembled from Bernoulli trials.

Definition 2.4.2 (Binomial)**.** $X \sim \text{Binomial}(n, p)$ if X counts the successes in n independent trials each of success probability p.

Proposition 2.4.3. *For $X \sim \text{Binomial}(n, p)$ and $0 \le k \le n$,*

$$\Pr[X = k] = \binom{n}{k} p^k (1-p)^{n-k}, \qquad \mathbb{E}[X] = np.$$

Proof. Any particular sequence with k successes and $n-k$ failures has probability $p^k(1-p)^{n-k}$ by independence, and there are $\binom{n}{k}$ such sequences, one for each choice of which trials succeed. Additivity gives the pmf.

For the mean, write $X = \mathbb{1}_1 + \cdots + \mathbb{1}_n$ where $\mathbb{1}_i$ indicates success on trial i. Linearity gives $\mathbb{E}[X] = np$. □

Look at the second half of that proof. Computing $\mathbb{E}[X]$ directly from the pmf means evaluating $\sum_k k\binom{n}{k}p^k(1-p)^{n-k}$, which takes some manipulation of binomial coefficients. Decomposing into indicators avoids the computation entirely.

2.4.2 Geometric

Definition 2.4.4 (Geometric)**.** $X \sim \text{Geometric}(p)$ if X is the number of independent trials up to and including the first success.

Proposition 2.4.5. *For* $X \sim \text{Geometric}(p)$ *and* $k \geq 1$*,*

$$\Pr[X = k] = (1-p)^{k-1}p, \qquad \Pr[X > k] = (1-p)^k, \qquad \mathbb{E}[X] = \frac{1}{p}.$$

Proof. The event $X = k$ requires $k-1$ failures followed by a success, which has probability $(1-p)^{k-1}p$ by independence. The event $X > k$ requires the first k trials all to fail.

For the mean, condition on the first trial. With probability p we finish in one trial. With probability $1-p$ we have used one trial and face the same problem again, so by the tower rule

$$\mathbb{E}[X] = p \cdot 1 + (1-p)\big(1 + \mathbb{E}[X]\big).$$

Solving gives $\mathbb{E}[X] = 1/p$. □

That derivation uses the structure of the problem rather than the pmf, and it generalizes: conditioning on the first step is the standard technique for processes that "restart".

Proposition 2.4.6 (Memorylessness)**.** *For* $X \sim \text{Geometric}(p)$ *and* $m, k \geq 0$*,*

$$\Pr[X > m + k \mid X > m] = \Pr[X > k].$$

Proof. $\Pr[X > m+k \mid X > m] = (1-p)^{m+k}/(1-p)^m = (1-p)^k$. □

A geometric random variable does not age. Ten failures in a row leave the distribution of the remaining wait exactly as it was at the start, and this is the reason the Markov chains of Chapter 6 can be described by a transition rule that ignores history.

Example 2.4.7 (The cost of von Neumann's trick)**.** Return to Example 2.2.7. Each attempt uses two flips and reports a bit when the pair is HT or TH, which happens with probability $2p(1-p)$. The attempts are independent, so the number of attempts until one succeeds is $\text{Geometric}(2p(1-p))$ with mean $1/(2p(1-p))$, and the expected number of flips per fair bit is

$$\underbrace{2}_{\text{flips per attempt}} \times \underbrace{\frac{1}{2p(1-p)}}_{\text{attempts}} = \frac{1}{p(1-p)},$$

where the two flips are spent whether or not the attempt reports anything. A fair coin, $p = 1/2$, still costs four flips per output bit: half of all attempts are discarded, so two attempts on average at two flips each. A coin with $p = 0.1$ costs eleven flips on average. A coin with $p = 0.01$ costs a hundred.

2.4.3 Approximating Many Failures

Tails like $(1-p)^n$ are hard to read off as they stand, and the standard move is to rewrite them as exponentials. Let's look at the following Taylor expansion,

$$e^{-p} = 1 - p + \frac{p^2}{2} - \cdots,$$

whose first two terms are exactly $1-p$. The terms after them alternate in sign and get smaller ($p \le 1$), so they add up to something positive, and

$$1 - p \le e^{-p}, \qquad \text{hence} \qquad (1-p)^n \le e^{-np}.$$

The useful reading of np is the expected number of successes, so a tail of $(1-p)^n$ says that avoiding all n trials costs less than $e^{-(\text{expected successes})}$.

The extra terms are also $\le p^2/2$ because the series is getting smaller and oscillating, so for small p the bound is nearly an equality. Raising to the nth power compounds that gap to a relative discrepancy of roughly $np^2/2$, and the bound becomes an approximation:

$$(1-p)^n \approx e^{-np}.$$

This is good when np^2 is small rather than merely when p is. At $p = 0.01$ and $n = 5000$ the two differ by almost thirty percent even though p is tiny. The condition is easiest to check written as

$$np^2 = (np) \cdot p,$$

the expected number of successes times p. So the approximation is safe whenever the expected count is moderate and p is small, however large n becomes.

There is a bound from below as well, which we've already seen. Bernoulli's inequality gives $(1-p)^n \ge 1 - np$. This is the union bound in disguise: rearranged, it says $1-(1-p)^n \le np$, which is Proposition 2.1.5(e) applied to n independent events of probability p each. It is informative only while $np < 1$, which is the same regime in which the union bound is worth using.

All together, we now have both an upper and a lower bound for the probability.

Proposition 2.4.8 (The exponential approximation)**.** *For $0 \le p \le 1$ and every $n \ge 1$,*

$$1 - np \;\le\; (1-p)^n \;\le\; e^{-np},$$

and $(1-p)^n \approx e^{-np}$ whenever np^2 is small.

We use this constantly: in Section 4.2 to see when a random graph has no isolated vertex.

2.4.4 Negative Binomial

Definition 2.4.9 (Negative binomial)**.** $X \sim \text{NegBinomial}(r, p)$ if X is the number of independent trials up to and including the rth success.

Split that wait at each success:

$$X = G_1 + G_2 + \cdots + G_r,$$

where G_i is the number of trials from the $(i-1)$st success to the ith. Each G_i is Geometric(p), and they are independent, because by memorylessness the trials after a success know nothing about the ones before it. This mirrors the binomial, which is a sum of n Bernoulli trials, and it gives both the mean below and the variance in Proposition 2.5.9.

Proposition 2.4.10. *For $k \geq r$,*

$$\Pr[X = k] = \binom{k-1}{r-1} p^r (1-p)^{k-r}, \qquad \mathbb{E}[X] = \frac{r}{p}.$$

Proof. The kth trial must be a success, and exactly $r-1$ of the preceding $k-1$ trials must be successes, which can happen in $\binom{k-1}{r-1}$ ways, each of probability $p^r(1-p)^{k-r}$.

For the mean, apply linearity to the decomposition above: $\mathbb{E}[X] = r\,\mathbb{E}[G_1] = r/p$. □

The negative binomial also turns up constantly in biology, where it is the standard model for how many times each gene's sequence appears among the reads of a sequencing run. The reason needs the Poisson distribution of Section 7.1, so we only note it here.

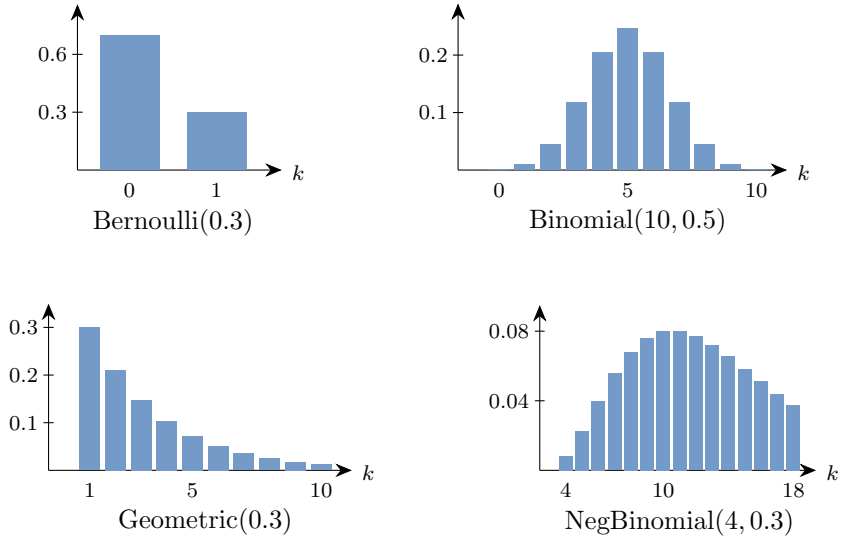


Figure 2.6: The four pmfs of this lecture. The Bernoulli describes one trial. The binomial lives on $\{0, \ldots, n\}$, is symmetric when $p = 1/2$, and keeps its mass near np. The geometric lives on $\{1, 2, \ldots\}$ and falls by the constant factor $1-p$ at every step, which is memorylessness seen as a picture. The negative binomial waits for four successes instead of one, so it cannot start before $k = 4$ and its mass sits near $r/p \approx 13$; being a sum of four independent geometric waits, it has lost the geometric's spike at the left edge and acquired a hump.

2.4.5 Collecting Everything

The negative binomial adds up geometric waits that all share one parameter. Often the stages of a longer job each have their properties, and the total is then a sum of geometric variables with different parameters.

Example 2.4.11 (The coupon collector)**.** A collection has n distinct items and each purchase brings one of them uniformly at random, sampling with replacement (this allows repeats). How many purchases does a full set take? This is the *coupon collector* problem, and it covers stickers for a soccer album, toys in a cereal box, or the toys in a fast-food kids' meal: each packet is drawn from the whole set, so the ones you already have keep coming back.

Split the wait by how much of the album is already done. With i distinct stickers in hand, a new sticker is one of the $n-i$ you lack, so each purchase is a success with probability

$(n-i)/n$ and the wait for the next new one is Geometric$((n-i)/n)$, with mean $n/(n-i)$. The total is the sum of those waits, so by linearity of expectation

$$\mathbb{E}[\text{stickers bought}] = \sum_{i=0}^{n-1} \frac{n}{n-i} = n\left(1 + \tfrac{1}{2} + \cdots + \tfrac{1}{n}\right) = nH_n \approx n \ln n.$$

For the 682 stickers of a World Cup album this is about 4840 stickers, some 970 packets of five, against the 136 packets you would need if every sticker were new. The factor $H_n \approx 7$ is the cost of randomness, and most of it is spent at the end: the last sticker alone takes n purchases on average, a seventh of the whole effort. It is the same reason the last cup in a game of beer pong takes so long to hit.

Sequencing a genome asks the coupon collector's question with a twist, and the twist changes how it has to be answered. The following approximate model of the sequencing process is called the Lander–Waterman model, after the standard analysis of shotgun sequencing.

Example 2.4.12 (The Lander–Waterman model, 1988)**.** A genome is a string of G bases drawn from A, C, G, T. The machine cannot read it end to end, so the sample is chopped into fragments and each fragment is read separately. A *read* is one such fragment: a run of L consecutive bases whose starting position is approximately uniform along the genome, and the N reads in a run are independent of one another because they come from identical, but separately chopped, strands.

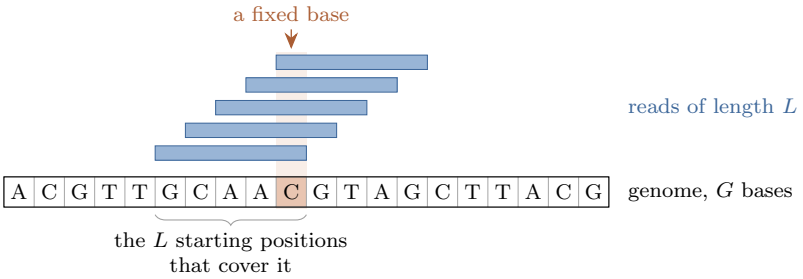


Figure 2.7: Why a single read covers a given base with probability L/G. A read is a run of L consecutive bases starting at a uniformly random position, here with $L = 5$. The shaded base is covered exactly when the read starts in one of the L positions ending at it, which are the five reads drawn above. Out of roughly G equally likely starting positions, L cover the base, so a read covers it with probability L/G.

The question is similar to coupon collector's: how many reads until every base has been covered? The sticker argument does not carry over, though. The trouble is that a read covers a *contiguous* stretch. With i stickers in hand the next one is new with probability $(n-i)/n$ whichever i they are, but whether a read reaches new bases depends on where the gaps are, not just on how many bases remain uncovered. The waits are no longer geometric with a known parameter, so we instead count what is still missing after N reads.

A given base is covered by a particular read exactly when that read starts in one of the L positions ending at the base, which has probability L/G, as Figure 2.7 shows. So the base is missed by every read with probability

$$\left(1 - \frac{L}{G}\right)^N \approx e^{-NL/G} = e^{-c}, \qquad c = \frac{NL}{G},$$

where c, the total number of bases read divided by the genome length, is called the *coverage* and is the average number of reads covering a base. The approximation is safe here even though N is enormous, because

$$N\Big(\frac{L}{G}\Big)^2 = \frac{NL}{G}\cdot\frac{L}{G} = c\,\frac{L}{G},$$

a moderate coverage times a tiny probability: at 30× on the human genome it is about 10^{-6}. One indicator per base and linearity then give Ge^{-c} for the expected number of bases never seen. The indicators for neighboring bases are strongly dependent, since adjacent bases are usually covered by the very same reads, which as usual does not matter.

Getting that expectation below one is the union bound: the probability that *some* base is missed is at most the expected number missed, and like every use of the union bound it ignores the strong dependence between neighboring bases. That needs $c > \ln G$, which in terms of reads answers the question:

$$N > \frac{G}{L}\ln G.$$

This looks like the coupon collector's $n \ln n$, but with one change. Each read collects L bases at once, so it takes roughly L times fewer draws than collecting the bases one at a time would. The logarithm is the same, we just reached this time by counting what is missing rather than by adding up waits.

For the human genome $G = 3 \times 10^9$ and $\ln G = 21.8$, so at $L = 150$ bases a read that is about 4.4×10^8 reads. That is why resequencing is sold at "30× coverage": not because thirty reads are needed anywhere in particular, but because covering everything costs a factor of $\ln G$ more than covering a typical base. At that read length 30× means about 6×10^8 reads and leaves an expected 3×10^{-4} bases unseen, while 10× would leave 136,000 of them.

> *Remark* 2.4.13 (The model's name). The analysis is due to Eric Lander and Michael Waterman, in 1988. Lander went on to become one of the principal leaders of the Human Genome Project, which ran from 1990 to 2003; his center at MIT was its largest single contributor, sequencing about a third of the genome. He then founded the Broad Institute.

2.4.6 Summary

Figure 2.6 plots all four distributions.

Distribution	Counts	pmf	Mean
Bernoulli(p)	success on one trial	$p,\quad 1-p$	p
Binomial(n, p)	successes in n trials	$\binom{n}{k}p^k(1-p)^{n-k}$	np
Geometric(p)	trials until first success	$(1-p)^{k-1}p$	$1/p$
NegBinomial(r, p)	trials until rth success	$\binom{k-1}{r-1}p^r(1-p)^{k-r}$	r/p

Variances are deferred to Proposition 2.5.9 in Section 2.5, which needs the rule for the variance of a sum. One further named distribution, the Poisson, arises as a limit of the binomial when n grows and p shrinks with np held fixed; Section 7.1 derives it.

2.5 Variance and Probability Concentration

Expectation says where a random variable sits on average, but we often also want to understand how close a distribution stays to its average. A bet with expected value zero is

very different depending on whether the outcomes are ± 1 or $\pm 10^6$. This lecture measures spread as variance, and then links that spread to the probability that an extreme event happens.

You already know the phenomenon this lecture makes precise. Seven heads in ten tosses of a fair coin is luck: it happens with probability $176/1024 = 0.172$, the same calculation as the mortgages of Section 2.1. Seven thousand heads in ten thousand tosses is a reason to inspect the coin. Sums and averages *concentrate* around their expected value as the number of trials grows, and that concentration is the reason insurance works.

2.5.1 Variance and Standard Deviation

Definition 2.5.1 (Variance)**.** For a random variable X with $\mathbb{E}[X] = \mu$,

$$\operatorname{Var}(X) = \mathbb{E}\big[(X - \mu)^2\big],$$

and the *standard deviation* is $\sigma_X = \sqrt{\operatorname{Var}(X)}$.

Squaring serves two purposes: it prevents positive and negative deviations from canceling, and it penalizes large deviations more than small ones. The standard deviation returns the answer to the units of X.

> *Remark* 2.5.2 (Higher moments). The quantity $\mathbb{E}[X^k]$ is the kth *moment* of X, so the mean is the first, and $\mathbb{E}[(X-\mu)^k]$ is the kth moment *about the mean*, so the variance is the second of those. Nothing stops us from going further. The third *divided by* σ^3 is the *skewness* and measures how lopsided a distribution is; the fourth divided by σ^4 is the *kurtosis* and measures how much of the probability sits far out in the tails. Those powers are what make the two quantities dimensionless: the kth moment carries the units of X^k, so dividing by σ^k cancels them and the answer describes the shape of the distribution rather than its scale.
>
> Think of it like a Taylor expansion. A linear approximation captures where a function is and which way it is heading, the quadratic term adds its curvature, and each further term adds a finer feature. The parallel is closer than an analogy, since the moments $\mathbb{E}[X^k]$ are exactly the Taylor coefficients of $\mathbb{E}[e^{tX}]$ about $t = 0$, the transform mentioned in Section 7.2.
>
> This course uses only the mean and the variance. They add up nicely over sums, which is what the rest of this lecture needs.

Proposition 2.5.3. *For any X with finite variance and constants a, b:*

$$\operatorname{Var}(X) = \mathbb{E}[X^2] - \big(\mathbb{E}[X]\big)^2, \qquad \operatorname{Var}(aX + b) = a^2 \operatorname{Var}(X).$$

Proof. Expand the square and use linearity: $\mathbb{E}[(X-\mu)^2] = \mathbb{E}[X^2] - 2\mu\mathbb{E}[X] + \mu^2 = \mathbb{E}[X^2] - \mu^2$. For the second, adding b shifts both X and its mean by b, leaving the deviation unchanged, and scaling by a scales the deviation by a, so the squared deviation scales by a^2. □

Example 2.5.4. For one fair die, $\mathbb{E}[X^2] = (1 + 4 + 9 + 16 + 25 + 36)/6 = 91/6$, so $\operatorname{Var}(X) = 91/6 - (7/2)^2 = 35/12 \approx 2.92$ and $\sigma \approx 1.71$.

2.5.2 Covariance and Sums

Definition 2.5.5 (Covariance)**.** $\operatorname{Cov}(X, Y) = \mathbb{E}\big[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\big] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$.

Taking $Y = X$ gives $\mathrm{Cov}(X, X) = \mathrm{Var}(X)$, so covariance extends variance from one variable to a pair.

Definition 2.5.6 (Independent random variables)**.** Random variables X and Y are *independent*, written $X \perp Y$, when every event $\{X = x\}$ is independent of every event $\{Y = y\}$:

$$\Pr[X = x,\ Y = y] = \Pr[X = x]\ \Pr[Y = y] \quad \text{for all } x, y.$$

Proposition 2.5.7. *If $X \perp Y$, then $\mathbb{E}[XY] = \mathbb{E}[X]\,\mathbb{E}[Y]$, and so $\mathrm{Cov}(X, Y) = 0$.*

Proof. Sum over all pairs of values and factor the joint probability:

$$\begin{aligned}\mathbb{E}[XY] &= \sum_x \sum_y xy \Pr[X = x, Y = y] = \sum_x \sum_y xy \Pr[X = x] \Pr[Y = y] \\ &= \Big(\sum_x x \Pr[X = x]\Big)\Big(\sum_y y \Pr[Y = y]\Big) = \mathbb{E}[X]\,\mathbb{E}[Y].\end{aligned}$$

Then $\mathrm{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] = 0$. □

Proposition 2.5.8. $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X, Y)$*, and more generally*

$$\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} \mathrm{Var}(X_i) + 2\sum_{i<j} \mathrm{Cov}(X_i, X_j).$$

So variance adds exactly when all the pairwise covariances vanish.

Here is the asymmetry with expectation announced in Section 2.3. Expectation adds for any random variables at all; variance adds only when the cross terms disappear.

Independent variables have $\mathrm{Cov} = 0$ by Proposition 2.5.7, so the cross terms vanish and variance adds. The converse fails, and independence is more than the identity needs: variables with $\mathrm{Cov}(X_i, X_j) = 0$ for every pair are called *uncorrelated*, i.e., they have no linear dependence.

We can now use these properties of variance to give the variances of the four distributions from last lecture.

Proposition 2.5.9. *For X with each of the distributions of Section 2.4,*

$$\begin{aligned} X \sim \mathrm{Bernoulli}(p): &\quad \mathrm{Var}(X) = p(1-p), \\ X \sim \mathrm{Binomial}(n, p): &\quad \mathrm{Var}(X) = np(1-p), \\ X \sim \mathrm{Geometric}(p): &\quad \mathrm{Var}(X) = \frac{1-p}{p^2}, \\ X \sim \mathrm{NegBinomial}(r, p): &\quad \mathrm{Var}(X) = \frac{r(1-p)}{p^2}. \end{aligned}$$

Proof. A Bernoulli variable satisfies $X^2 = X$, so $\mathrm{Var}(X) = \mathbb{E}[X] - \mathbb{E}[X]^2 = p - p^2$. A binomial is a sum of n *independent* Bernoulli trials, so the covariances vanish and the variances add, giving $np(1-p)$; a negative binomial is a sum of r independent geometric waits, giving r times the geometric variance. For the geometric itself, conditioning on the first trial as in Section 2.4 gives $\mathbb{E}[X^2] = p + (1-p)\,\mathbb{E}[(1+X)^2]$, and solving with $\mathbb{E}[X] = 1/p$ leaves $\mathbb{E}[X^2] = (2-p)/p^2$, so $\mathrm{Var}(X) = (2-p)/p^2 - 1/p^2$. □

2.5.3 The Law of Large Numbers

Let $X_1, \dots, X_n$ be independent with the same mean μ and the same variance σ^2, and let

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^{n} X_i$$

be the sample mean. Linearity gives $\mathbb{E}[\bar{X}_n] = \mu$. For the variance, Proposition 2.5.3 contributes the factor $1/n^2$ and Proposition 2.5.8 the sum of n equal terms:

$$\mathrm{Var}(\bar{X}_n) = \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}. \tag{2.2}$$

Averaging n independent measurements leaves the center alone and divides the variance by n, so the standard deviation falls like $\sigma/\sqrt{n}$. Equation (2.2) is the quantitative heart of this lecture and of much of the rest of the course.

Averaging makes the distribution less spread out, and it is tempting to conclude that large deviations of $\bar{X}_n$ from μ become unlikely. It turns out that this is not immediately obvious: "small variance" is a statement about average squared distance, while an extreme event is a statement about probability, and less variance does not by itself make extreme events less likely. Figure 2.8 shows the gap: of the two distributions there, the one with less variance is the one with extreme events.

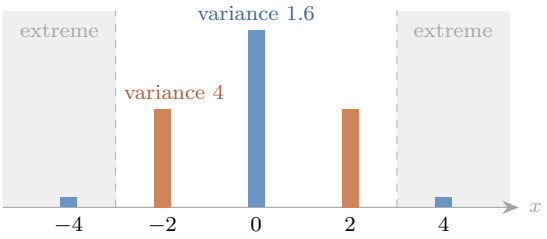


Figure 2.8: Less variance does not mean fewer extreme events. The blue distribution has variance 1.6 and the orange has variance 4, yet only the blue one puts any probability beyond the dashed lines. Variance averages the squared distance over all the mass, so a little mass far out can hide behind a lot of mass at the center; the inequalities below limit how much can hide.

What we want is a way to turn a small variance into a small probability of extreme events. Two inequalities do it, and they are often useful outside of this setting as well.

Theorem 2.5.10 (Markov's inequality)**.** *If $X \geq 0$ and $a > 0$, then $\Pr[X \geq a] \leq \mathbb{E}[X]/a$.*

Proof. Split the expectation at a:

$$\mathbb{E}[X] = \sum_x x\, p_X(x) = \underbrace{\sum_{x<a} x\, p_X(x)}_{\geq 0 \text{ since } X \geq 0} + \sum_{x \geq a} x\, p_X(x) \;\geq\; \sum_{x \geq a} x\, p_X(x).$$

Every value left in that sum is at least a, so replacing each x by a only decreases it further:

$$\sum_{x \geq a} x\, p_X(x) \;\geq\; a \sum_{x \geq a} p_X(x) = a\, \Pr[X \geq a].$$

Combining the two lines gives $\mathbb{E}[X] \geq a \Pr[X \geq a]$, and dividing by a is the claim. □

The bound is loose because information was discarded twice, once in each step: the first threw away everything X does below a, and the second pretended that all the mass above a sits exactly at a.

Figure 2.9 shows both steps as a picture. The mean is the balance point of the distribution, as in Section 2.3, and mass further from it pulls harder, like weight on a lever. The question is how much mass can be pushed out to a and beyond while the balance point stays at $\mathbb{E}[X]$.

So we want the mass on the left doing as much work as possible: we cannot push it below 0, so we pack it all in at 0, as far from the fulcrum as it will go. And we want the mass on the right pulling as little as possible, so we bring it in as close as we can, assuming everything $\geq a$ sits exactly at a.

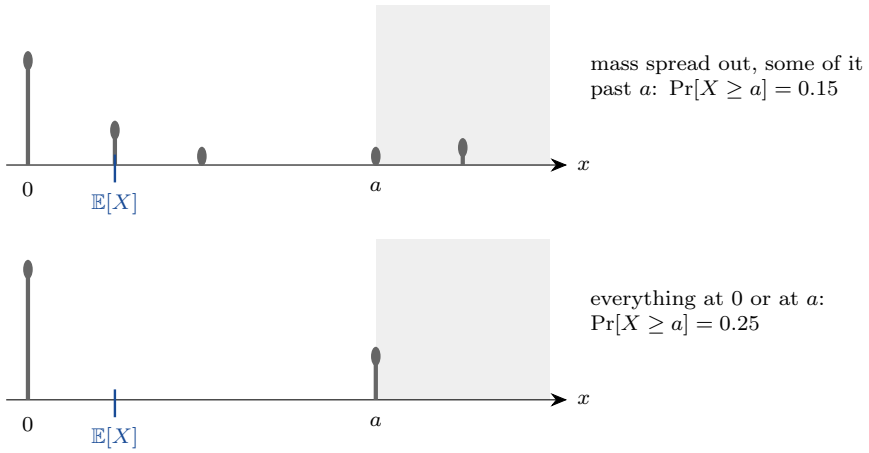


Figure 2.9: Why Markov's inequality holds, with $\mathbb{E}[X] = 1$ and $a = 4$. The goal is to see how much mass can be put $\geq a$ while balancing the mean. Both distributions have the same mean; the shaded region is $x \geq a$. The upper picture wastes its mass to the left of a because it would lower the mean if all packed in at 0, allowing more mass to the right of a. The mass to the right of a is also wasteful, because it would pull the mean less if it were all packed in at a.

Theorem 2.5.11 (Chebyshev's inequality)**.** *If X has mean μ and standard deviation σ, then for any $k > 0$,*

$$\Pr\big[|X - \mu| \geq k\sigma\big] \leq \frac{1}{k^2},$$

and equivalently $\Pr[|X - \mu| \geq \epsilon] \leq \sigma^2/\epsilon^2$.

Proof. The random variable $(X - \mu)^2$ is nonnegative with expectation σ^2, so Markov's inequality at level $k^2\sigma^2$ gives $\Pr[(X - \mu)^2 \geq k^2\sigma^2] \leq \sigma^2/(k^2\sigma^2)$, and the event on the left is the event $|X - \mu| \geq k\sigma$. □

Neither result needs to know the distribution, so both are usable without assumptions. The cost of this is that the bound can be a little loose: at $k = 2$ Chebyshev permits a quarter of the mass beyond two standard deviations, where for a bell-shaped distribution the true figure is nearer a twentieth.

But that twentieth assumes a bell shape, and some quantities that matter are far from bell-shaped. On 19 October 1987 the S&P 500 fell 20.5% in a single day, at least twenty standard deviations of its usual daily moves. A bell curve, the Gaussian distribution of Section 7.2, gives a fall that large a probability of about 10^{-89}, which is to say never;

Chebyshev, which assumes nothing about the shape, allows up to 1/400. Financial returns are *fat-tailed*, and for them the loose bound is the honest one.

With Chebyshev in hand, the law of large numbers takes one line.

Theorem 2.5.12 (Weak law of large numbers)**.** *Let $X_1, X_2, \dots$ be independent with common mean μ and common finite variance σ^2. Then for every $\epsilon > 0$,*

$$\Pr\big[\big|\bar{X}_n - \mu\big| \geq \epsilon\big] \leq \frac{\sigma^2}{n\epsilon^2} \xrightarrow[n\to\infty]{} 0.$$

Proof. Apply the second form of Chebyshev's inequality to $\bar{X}_n$, whose mean is μ and whose variance is σ^2/n by (2.2). □

Small variance really does force concentration.

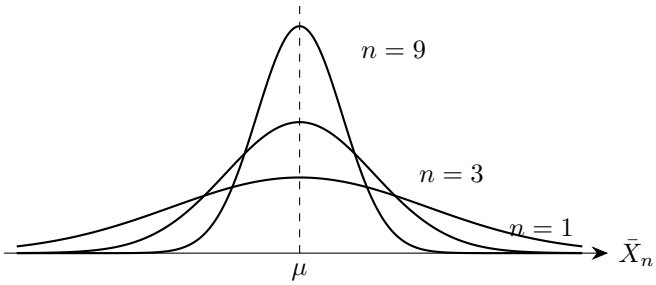


Figure 2.10: Concentration of the sample mean. Each curve is the distribution of $\bar{X}_n$ for a different number of samples. The center stays at μ while the width shrinks like $1/\sqrt{n}$, so the mass piles up over μ. The law of large numbers is the statement that the area outside any fixed window around μ tends to zero.

Figure 2.10 shows what the theorem describes. Averaging does not move the target and does not eliminate randomness; it narrows the distribution around the right answer at a rate we can compute.

2.5.4 Monte Carlo Estimation and $1/\sqrt{n}$ Error

The law of large numbers is also a computational method. Suppose we want a quantity that is hard to calculate exactly but easy to express as an expectation. Draw independent samples, average them, and use (2.2) to say how good the answer is. This is *Monte Carlo estimation*.

The most common case is estimating a probability. To estimate $p = \Pr[A]$, run the experiment n times and let $\hat{p} = \bar{X}_n$ where $X_i = \mathbb{1}_{A \text{ on trial } i}$. Then $\mathbb{E}[\hat{p}] = p$, and since each X_i is Bernoulli with variance $p(1-p) \leq 1/4$,

$$\mathrm{Var}(\hat{p}) = \frac{p(1-p)}{n} \leq \frac{1}{4n}, \qquad \sigma_{\hat{p}} \leq \frac{1}{2\sqrt{n}}.$$

Accuracy improves like $1/\sqrt{n}$, so buying one extra decimal digit costs a hundredfold increase in samples. Notice also that the bound does not mention what the experiment was: Monte Carlo works the same way for a quantity nobody can compute analytically as for one we could do by hand.

Example 2.5.13 (Estimating π)**.** Draw x and y independently and uniformly from $[0, 1]$, and ask whether $x^2 + y^2 < 1$. That event is the quarter disc inside the unit square of Figure 2.11, so its probability is $\pi/4$, and

$$\hat{\pi} = 4 \cdot \frac{\text{number of points inside}}{n}$$

estimates π without anyone ever computing an area. The indicator has variance $\frac{\pi}{4}(1 - \frac{\pi}{4}) = 0.169$, so

$$\sigma_{\hat{\pi}} = 4\sqrt{\frac{0.169}{n}} = \frac{1.64}{\sqrt{n}},$$

which is 0.16 at $n = 100$, 0.016 at $n = 10^4$ and 0.0016 at $n = 10^6$. Three correct decimal places cost a million samples, so as a way of computing π this is terrible.

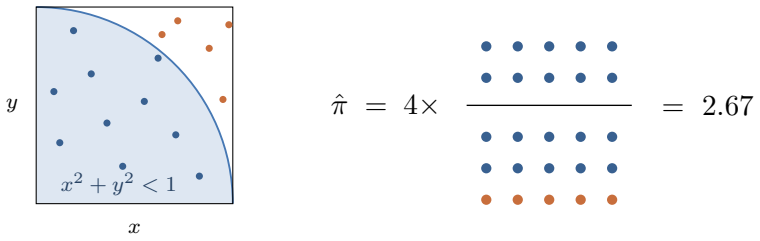


Figure 2.11: Estimating π by counting. Points are thrown uniformly at the unit square; the shaded quarter disc has area $\pi/4$, so the fraction landing inside it estimates $\pi/4$ and four times that fraction estimates π. Here 10 of the 15 points are inside, giving $\hat{\pi} = 2.67$, which shows how slowly this converges.

There are two reasons to care about a method this inefficient.

The first is that many probabilities cannot be written down at all. Nobody has a formula for the chance that a hurricane hits a given county, but a weather simulator can be run many times, and the fraction of runs that make landfall estimates it.

The second is dimension. Computing an integral on a grid of spacing ϵ takes $(1/\epsilon)^d$ evaluations in d dimensions, which grows exponentially and is called the *curse of dimensionality* (Figure 2.12).

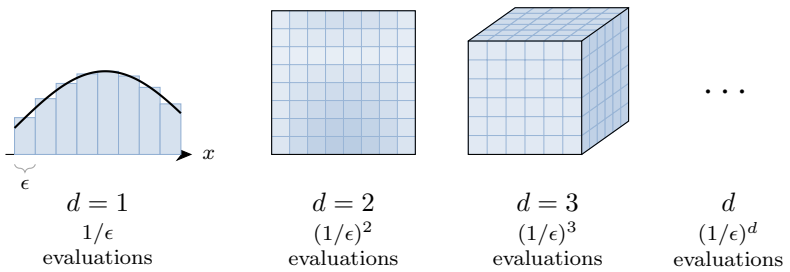


Figure 2.12: The curse of dimensionality. Approximating an integral on a grid of spacing ϵ costs one evaluation per cell, and the number of cells is $(1/\epsilon)^d$. Halving ϵ multiplies the work by 2^d, so accuracy that is routine in one or two dimensions is unreachable in ten. A Monte Carlo estimate, by contrast, has error $\sigma/\sqrt{n}$ no matter what d is.

Monte Carlo does not care: its error is $\sigma/\sqrt{n}$ in any dimension, since nothing in equation (2.2) mentioned d. So grids win in low dimensions and Monte Carlo wins once d is more than three.

Chapter 3

Information and Coding

Chapter 2 gave us tools for describing randomness. This chapter asks two questions about what we can do with it. First, how much does it cost to write a random thing down, and what is the cheapest possible cost? Second, if the writing gets damaged in transit, how do we recover it? The two questions turn out to be related and built on a quantity called entropy, and a single 1948 paper of Claude Shannon established the limits of both.

3.1 Information Theory

3.1.1 Twenty Questions

Let's start with a puzzle. In Wordle the most commonly played opening guess is ADIEU, on the reasonable-sounding theory that vowels are worth finding early. It is one of the worst openers available. Better choices are CRANE or SLATE. By the end of this lecture we will be able to say why, and the reason has nothing to do with vowels in particular.

Consider a simpler game. One player thinks of a word, say *horse*, and the others must identify it using yes/no questions such as "is it a living thing?" Suppose for concreteness that both sides know the word is one of these sixteen, all equally likely:

Living		*Inanimate*	
dog	horse	pen	house
cat	doctor	burger	shoe
Beilock	mushroom	car	bat
tree	grass	phone	ball

A yes to "is it a living thing?" eliminates the whole right-hand side at once.

A question splits the sixteen possibilities into two groups, and the answer tells us which group holds the word. If each question splits the remaining candidates exactly in half, then after x questions the number of candidates is $16 \cdot (1/2)^x$, and we have identified the word when this reaches 1:

$$\left(\tfrac{1}{2}\right)^x = \tfrac{1}{16} \qquad \Longrightarrow \qquad x = -\log_2\left(\tfrac{1}{16}\right) = 4.$$

Four questions, and the four answers form a bit string such as 0110 that names the word uniquely. Each question, answered, is worth one *bit*.

The key point to take from this is that information is a statement about uncertainty. The more uncertain we are to begin with, the more answers we need to collect to isolate the answer. A question about something we already know tells us nothing.

3.1.2 Self-Information

Not all answers are equally valuable. Suppose instead of a balanced question we ask "is it used for transportation?" Most of the time the answer is no and we have barely narrowed anything, from sixteen candidates to fourteen. Occasionally the answer is yes and we are left with just *horse* and *car*, which is a large gain. A surprising answer carries more information than an unsurprising one.

Definition 3.1.1 (Self-information)**.** The *self-information*, or *surprisal*, of an outcome m is

$$I(m) = -\log_2 \Pr[m].$$

For the transportation question, a yes has probability $2/16$ and carries $-\log_2(2/16) = 3$ bits, while a no carries only $-\log_2(14/16) \approx 0.19$ bits.

Why a logarithm? The game above already suggests one reason. An outcome of probability $1/16$ took four balanced questions to reach, since each answer halved the candidates and $1/16 = (1/2)^4$; an outcome of probability 2^{-k} takes k such questions, and $k = -\log_2 \Pr[m]$.

The second reason is the same one in disguise: information from independent sources should add. If M_1 and M_2 are independent, then

$$I(M_1 \cap M_2) = -\log_2 \Pr[M_1 \cap M_2] = -\log_2 \Pr[M_1] - \log_2 \Pr[M_2] = I(M_1) + I(M_2),$$

using independence in the middle step. The four questions worked precisely because their answers were independent. Each was a fair coin whatever the earlier answers had been, so any particular string of answers had probability $(1/2)^4$ and was worth $1 + 1 + 1 + 1$ bits. A redundant question, whose answer the earlier ones already settle, would add nothing, and later in this lecture we will see that no imbalanced yes/no question can do better than a balanced one. When $\Pr[m]$ is not a power of $1/2$, $-\log_2 \Pr[m]$ is not a whole number of questions, which we will dig into a bit more next lecture.

3.1.3 Entropy

Self-information describes a single outcome. To describe a whole distribution we average it.

Definition 3.1.2 (Entropy)**.** The *entropy* of a discrete random variable X is

$$H(X) = -\sum_x \Pr[X = x] \log_2 \Pr[X = x] = \mathbb{E}\big[I(X)\big].$$

Entropy is an expectation, so everything from Section 2.3 applies to it. If self-information counts the questions needed to pin down one particular outcome, then entropy is the average number of questions needed to pin down a draw from the distribution. The units are bits.

Example 3.1.3 (Equally likely outcomes)**.** If X is uniform on W outcomes then every term is $-\frac{1}{W}\log_2 \frac{1}{W}$, and there are W of them, so

$$H(X) = \log_2 W.$$

This links entropy to the microstate counting of Section 2.1: the entropy of a system with W equally likely microstates is the number of bits needed to say which one it is in.

> *Remark* 3.1.4 (Text). ASCII has 128 characters, and UTF-8 stores each in an 8-bit byte, though $\log_2 128 = 7$ bits would already do if every character were equally likely. They are

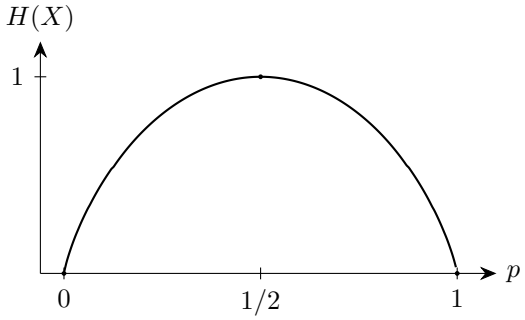


Figure 3.1: Entropy of $X \sim \text{Bernoulli}(p)$. The entropy is largest at $p = 1/2$, where the outcome is hardest to guess and one question is needed. It falls to zero at $p = 0$ and $p = 1$, where there is nothing to ask about.

> not: in English text e is far commoner than q, so the entropy per character is lower still. Compression exploits this gap...stay tuned!

Example 3.1.5. For $X \sim \text{Bernoulli}(p)$,

$$H(X) = -p \log_2 p - (1-p) \log_2 (1-p),$$

plotted in Figure 3.1. A fair coin has entropy 1 and a two-headed coin has entropy 0.

Definition 3.1.6 (Joint entropy)**.** The *joint entropy* of a pair of random variables is

$$H(X, Y) = -\sum_{x,y} \Pr[X = x, Y = y] \log_2 \Pr[X = x, Y = y].$$

3.1.4 Conditional Entropy

How many questions are left after an answer? Before anything is asked the word has entropy 4 bits. An answer leaves us with a conditional distribution over the words still possible, and the uncertainty that remains is the entropy of that distribution. How much remains can depend on which answer came back, so we average over the answers we might get.

Definition 3.1.7 (Conditional entropy)**.** For a particular value y,

$$H(X \mid Y = y) = -\sum_{x} \Pr[X = x \mid Y = y] \log_2 \Pr[X = x \mid Y = y],$$

and the conditional entropy of X given Y is the average of these over Y:

$$H(X \mid Y) = \sum_{y} \Pr[Y = y]\, H(X \mid Y = y) = \mathbb{E}_Y \big[H(X \mid Y = y) \big].$$

Note the convention: a lower-case y means one specific answer, a capital Y means we have averaged over the answers we might get. The averaging is the tower rule of Section 2.3 applied to self-information.

Example 3.1.8. Return to the sixteen equally likely words, so $H(X) = \log_2 16 = 4$ bits.

Ask $Y_1 =$ "is it a living thing?", which splits the sixteen into eight and eight. Whichever answer comes back, eight equally likely candidates remain, with entropy 3. So

$$H(X \mid Y_1) = \tfrac{1}{2}(3) + \tfrac{1}{2}(3) = 3 \text{ bits}.$$

Ask instead Y_2 = "is it used for transportation?", which splits them into two, *horse* and *car*, and fourteen. With probability 2/16 we are left with two candidates, entropy 1; with probability 14/16 we are left with fourteen, entropy $\log_2 14 \approx 3.807$. So

$$H(X \mid Y_2) = \tfrac{2}{16}(1) + \tfrac{14}{16}(3.807) \approx 3.456 \text{ bits.}$$

The balanced question leaves less randomness behind.

3.1.5 Mutual Information

Conditional entropy measures the uncertainty an answer leaves behind, so the natural measure of what a question is worth is how much uncertainty it removes.

Definition 3.1.9 (Mutual information)**.** The *mutual information* between X and Y is the reduction in the entropy of X from conditioning on Y,

$$I(X : Y) = H(X) - H(X \mid Y).$$

Example 3.1.10. Continuing Example 3.1.8, the balanced question gains $I(X : Y_1) = 4 - 3 = 1$ bit and the pointed question gains $I(X : Y_2) \approx 4 - 3.456 = 0.544$ bits.

Notice where those numbers come from: they are the entropies of the answers themselves. The living question is a fair coin, so $H(Y_1) = 1$, and the transportation question is answered yes two times in sixteen, so $H(Y_2) \approx 0.544$, the Bernoulli entropy of Figure 3.1. The uncertainty each question removes is exactly the randomness in its answer. That is why a yes/no question can remove at most one bit and it does so when it eliminates half of the potential answers.

This works because each word fixes the answer, and it fails for a question whose answer is still random once the word is known. Consider Y_3 = "is it raining?" Its answer is random, so $H(Y_3) > 0$, but it is independent of the word, so $H(X \mid Y_3) = H(X)$ and $I(X : Y_3) = 0$. Flipping it around makes the point: even knowing the word is "horse" leaves the weather just as uncertain, so $H(Y_3 \mid X) = H(Y_3)$.

An answer's randomness therefore has two parts: the part that narrows down the word, which is the mutual information, and the part that has nothing to do with it, the conditional entropy $H(Y \mid X)$ of the answer given the word. Figure 3.2 draws the decomposition.

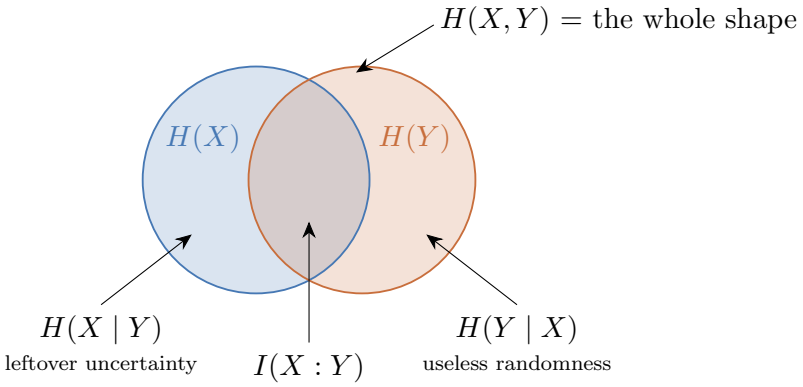


Figure 3.2: An information diagram. The left circle is the uncertainty $H(X)$ about the word and the right circle is the randomness $H(Y)$ of the answer. Their overlap, the mutual information, is the part of the answer's randomness that narrows down the word. The left crescent is the uncertainty left over once the answer is known, and the right crescent is randomness in the answer that does nothing to isolate the word. A question whose answer is fixed by the word has an empty right crescent, and the raining question has no overlap at all. Every identity in Proposition 3.1.12 can be read off the picture as a statement about areas.

Example 3.1.11 (A lying answerer)**.** Between these extremes is a question whose answer is partly about the word. A popular way to get people to share sensitive information in surveys, called *randomized response*, is to have them roll a die and deliberately lie if it comes up one, which gives them plausible deniability if they admit to something illegal. Suppose the answerer lies in this way about whether the word is a living thing, one time in six. The answer is still yes half the time, so $H(Y) = 1$ bit, just as for the honest question. But a yes now leaves the word living only with probability $5/6$, and the conditional entropy rises to $3 + H(1/6) \approx 3.650$ bits, so $I(X:Y) \approx 4 - 3.650 = 0.350$ bits. Of the answer's one bit, only 0.350 is about the word. The other 0.650 is the lie, the right crescent of Figure 3.2.

Proposition 3.1.12. *For discrete random variables X and Y:*

(a) $H(X \mid Y) = H(X,Y) - H(Y)$.

(b) $I(X:Y) = H(X) + H(Y) - H(X,Y)$.

(c) $I(X:Y) = I(Y:X)$.

(d) $I(X:Y) = 0$ *if and only if* X *and* Y *are independent.*

Proof. For (a), expand the definition and use $\Pr[x \mid y] = \Pr[x,y]/\Pr[y]$:

$$\begin{aligned} H(X \mid Y) &= -\sum_{y} \Pr[y] \sum_{x} \Pr[x \mid y] \log_2 \Pr[x \mid y] \\ &= -\sum_{x,y} \Pr[x,y] \big(\log_2 \Pr[x,y] - \log_2 \Pr[y] \big) \\ &= H(X,Y) - H(Y), \end{aligned}$$

where the last step uses $\sum_x \Pr[x,y] = \Pr[y]$. Part (b) follows by substituting (a) into the definition of I, and (c) is immediate from (b), which is symmetric in X and Y.

For (d), if X and Y are independent then $I(x,y) = I(x) + I(y)$ for every pair of values by the additivity of self-information, so taking expectations gives $H(X,Y) = H(X) + H(Y)$

by linearity, and (b) makes $I(X : Y) = 0$. The converse also holds, though we will not prove it. □

Part (b) says that mutual information is the amount by which the entropies of X and Y *overstate* the entropy of the pair. If the two variables share information, counting them separately counts the shared part twice, and $I(X : Y)$ measures the double count. This is very similar to the inclusion/exclusion correction of Section 1.3.

Applying (a) with X and Y exchanged and substituting into (b) gives $I(X : Y) = H(Y) - H(Y \mid X)$, the precise form of the decomposition in Figure 3.2: the randomness in the answer, minus the part $H(Y \mid X)$ that has nothing to do with the word. For the lying answerer, $H(Y \mid X)$ is the entropy of the lie, about 0.650 bits.

Figure 3.2 encodes the whole proposition. Reading it, $H(X \mid Y)$ is what remains of $H(X)$ after removing the overlap, which is part (a); the union is the two circles minus the double-counted overlap, which is part (b); the overlap is symmetric, which is part (c); and independence is the case of no overlap at all, which is part (d).

> *Remark* 3.1.13. As with Venn diagrams in Lecture 1, the picture is an aid rather than a proof. It is reliable for two variables. For three or more it can suggest false statements, because the region corresponding to "information shared by all three" can be negative, and no diagram of overlapping areas can have a negative piece.

3.1.6 Back to Wordle

We now have the vocabulary to explain the opening puzzle. A guess is a question, and a good question is one with high mutual information with the answer: it should split the remaining candidate words into groups of roughly equal probability.

ADIEU fails this test. Nearly every five-letter word contains vowels, so learning that a word contains some vowels eliminates very little. The guess has low mutual information with the answer because its likely outcomes are lopsided.

CRANE does a lot better. The letters C, R, N are common enough that roughly half of candidate words contain each of them and roughly half do not, so each letter behaves like a balanced question. The additional trick is that a negative result is also informative: if you play CRANE and both A and E are absent, you have learned that the answer contains one of the rarer vowels I, O, or U.

> *Remark* 3.1.14. Shannon's 1948 paper founded this subject and set up both of the next two lectures: the limits of compression and the limits of error correction. He also co-organized the 1956 Dartmouth workshop that is generally credited as the birthplace of artificial intelligence, which is where the last lecture of this course ends up.

3.2 Compression

The last lecture changed our perspective on what a file is. Every bit answers a yes/no question (perhaps a strange one) and a file is a sequence of answers that narrows the possibilities down to the one message it actually contains. Entropy measured how many answers a message needs on average. This lecture shows how we can break a file into questions that actually achieve the entropy. This answers the first of the questions from Chapter 0: what is a compression program actually doing when it makes a file smaller?

Standard encodings such as ASCII are *fixed-length*: every letter gets the same number of bits, so the reader always knows which question the next bit answers. Each letter is spelled out as a sequence of questions that narrow it down. If every letter is equally likely,

splitting the remaining letters into two groups of equal size at each step makes every answer a fair coin, worth a full bit, and the code wastes nothing. If the letters are not equally common, equal-size groups will not equally split the probability, which means they will not give us a full bit of information per bit of memory. Compression is a matter of asking better questions.

3.2.1 Decision Trees

A sequence of questions can be drawn as a *decision tree*. Each point where the tree branches asks one question, its two branches are the two answers, and following the answers down from the top ends at a letter.

The Hawaiian alphabet has thirteen letters: the five vowels a, e, i, o, u, the seven consonants h, k, l, m, n, p, w, and the *ʻokina*, a consonant written ‘ that marks a glottal stop (like "uh'oh"). Thirteen letters need four bits, since three bits give only $2^3 = 8$ codewords, so a fixed-length code gives each letter its own four-bit string and leaves three of the sixteen unused. Figure 3.3 draws the code as a tree, with the letters split as evenly as possible at every question. Its first bit asks whether the letter is one of a, e, i, o, u, h, k, and reading 0110 means answering four questions as 0, 1, 1, and 0, which gives us a path that ends at k.

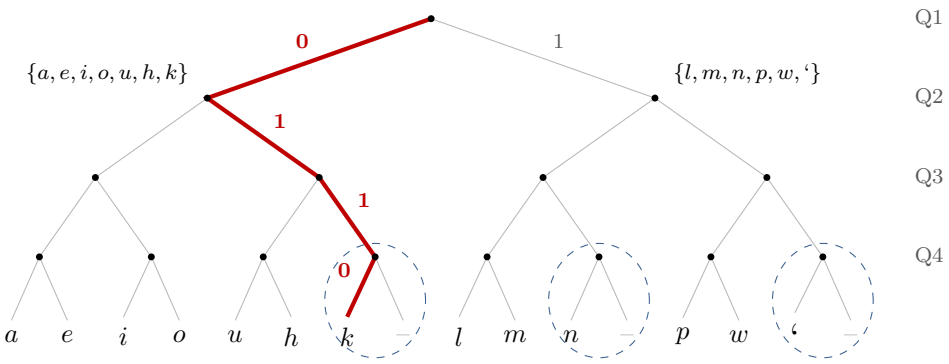


Figure 3.3: The fixed-length code for the Hawaiian alphabet as a decision tree. Each branch point asks one question, with an answer of 0 moving to the left and 1 to the right, and the letters are split as evenly as possible at every question: the first asks whether the letter is one of a, e, i, o, u, h, k, versus the other six. The red path follows the answers $0, 1, 1, 0$ to the letter k. Three of the sixteen four-bit codewords, marked with dashes, are never used, so the circled branch points ask useless questions: one of their two answers never happens.

This code spends four bits on every letter, which is wasteful in multiple ways. First, because three codewords are never used, some questions are pointless. Once the first three answers are $0, 1, 1$, the letter can only be k, since 0111 is never used, so the fourth question has a known answer and carries no information. Dropping it gives k the three-bit codeword 011, and the same works for n and ‘, which become 101 and 111. The decoder still knows where each letter ends, since after reading 011 it has already arrived at k. With three letters at three bits and ten at four, thirteen equally likely letters would cost $49/13 \approx 3.77$ bits on average, closer to their entropy of $\log_2 13 \approx 3.70$ bits.

Second, and more importantly, the letters are far from equally common, so it matters which letters get the short codewords. Every Hawaiian syllable is a vowel, possibly preceded by a single consonant, so at least half the letters of any Hawaiian text are vowels: five letters carry half the text, yet the code spends as many bits on a as on w. All five vowels fall on the left of the first question, so its answer is 0 much more often than 1, and such a predictable answer is not worth much in terms of bits of information.

Both problems point to the same idea: encode letters into strings of *variable length*.

3.2.2 Variable-Length Codes

To see what variable length can buy, take a language with just four letters.

Example 3.2.1. Suppose a language uses four letters with frequencies

$$\Pr[A] = \tfrac{1}{2}, \quad \Pr[B] = \tfrac{1}{4}, \quad \Pr[C] = \tfrac{1}{8}, \quad \Pr[D] = \tfrac{1}{8}.$$

Then

$$H(X) = \tfrac{1}{2}(1) + \tfrac{1}{4}(2) + \tfrac{1}{8}(3) + \tfrac{1}{8}(3) = 1.75 \text{ bits},$$

so a letter takes 1.75 questions to identify on average, not the 2 bits of a fixed-length code.

What if the more common letters got shorter strings of bits? Try

$$A \to 0, \quad B \to 1, \quad C \to 00, \quad D \to 01.$$

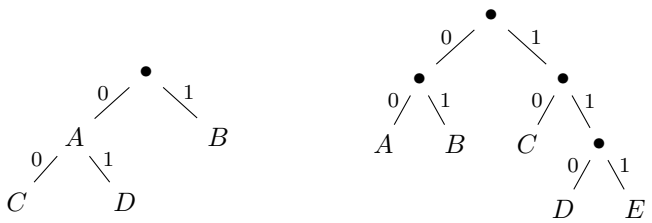
Now three quarters of the letters cost a single bit, and the average length is $\frac{1}{2}(1) + \frac{1}{4}(1) + \frac{1}{8}(2) + \frac{1}{8}(2) = 1.25$ bits per letter, which is suspicious: it beats the entropy.

It is also broken. The string 00 could be AA or C, and 01001 could be read as DAD or as $ABCB$. The code is ambiguous, so those 1.25 bits per letter do not actually carry the message.

3.2.3 Prefix-Free Codes

The trouble we are running into is that the codeword for A, a single 0, is the beginning of the codewords for C and D. After reading a 0, the decoder cannot tell whether to stop and call the letter A or to keep reading. So a code should never let one codeword be a prefix of another.

In the decision tree this has a simple picture. The endpoints of a tree, where no further question is asked, are called its *leaves*, and the points where it still branches are its *internal nodes*. In the broken code, A sits at a branching point on the way to C and D. In a code that decodes cleanly, every letter is a leaf.



not prefix-free prefix-free

Figure 3.4: Two decision trees. On the left is the broken code for four letters: A sits at a branch point, so its codeword 0 is a prefix of those for C and D and the decoder stalls. On the right is a prefix-free code for five letters, $A \to 00$, $B \to 01$, $C \to 10$, $D \to 110$, $E \to 111$, in which every letter is a leaf, an endpoint of the tree.

Definition 3.2.2 (Prefix-free code)**.** A code is *prefix-free* if no codeword is a prefix of another, equivalently if the letters are assigned only to the leaves of the decision tree.

This makes the problem simple. Any tree with the letters on its leaves gives a code that can be decoded without ambiguity: read bits, walk down the tree, and the moment a leaf is reached, decide on that letter and return to the top. Now we can look for a cheap one!

3.2.4 Shannon–Fano Coding

Inspired by information theory, Claude Shannon and Robert Fano tried to make every bit carry as much information as possible: split the letters into two groups whose probabilities are as close to 50/50 as possible, and repeat inside each group. Shannon's 1948 paper and a 1949 report by Fano each described a method of this kind. For the four letters of Example 3.2.1 the first split is $\{A\}$ against $\{B, C, D\}$, each with probability $\frac{1}{2}$; the second is $\{B\}$ against $\{C, D\}$, at $\frac{1}{4}$ each; and the last distinguishes C from D.

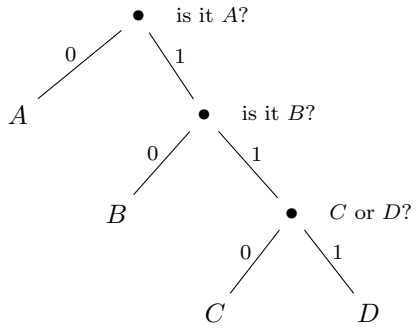


Figure 3.5: The tree that splits the probability evenly at every question. "Is it A?" is answered yes half the time; "is it B?" splits the remaining half evenly; and the last question tells C from D. A yes takes the left branch, labeled 0, so the codewords are $A \to 0$, $B \to 10$, $C \to 110$, $D \to 111$.

Every question eliminates exactly half of the remaining probability, which is the most a yes/no question can do, and the code $A \to 0$, $B \to 10$, $C \to 110$, $D \to 111$ averages

$$\tfrac{1}{2}(1) + \tfrac{1}{4}(2) + \tfrac{1}{8}(3) + \tfrac{1}{8}(3) = 1.75 \text{ bits per letter},$$

exactly the entropy.

The method is greedy, which is good here...but not always best. It commits to its first question without seeing what that choice will cost further down, and the best questions now can spoil the questions later.

Example 3.2.3. Take frequencies $\Pr[A] = 0.4$, $\Pr[B] = 0.4$, $\Pr[C] = 0.1$, $\Pr[D] = 0.1$. The most even first split is $\{A, C\}$ against $\{B, D\}$, exactly 50/50. Each group then splits again, giving every letter a two-bit codeword and an average of 2 bits per letter, no better than a fixed-length code.

Splitting less evenly does better. Take $\{A\}$ against $\{B, C, D\}$, then $\{B\}$ against $\{C, D\}$, for lengths $1, 2, 3, 3$ and an average of

$$0.4(1) + 0.4(2) + 0.1(3) + 0.1(3) = 1.8 \text{ bits per letter}.$$

This does not contradict Section 3.1, where a balanced question gained the most information. The perfectly even first question gains a full bit, while "is it A?", answered yes four times in ten, gains only about 0.97. But letters cannot be split, so the best first question on its own can leave groups that split badly later, and making each question as good as possible is not the same as making the whole code as good as possible.

> *Remark* 3.2.4 (Fano's own version). Fano's procedure sorted the letters by frequency and cut the sorted list in one place, which here gives $\{A\}$ against $\{B, C, D\}$ and finds the 1.8-bit code. It can still be fooled. For frequencies A:15, B:7, C:6, D:6, E:5 out of 39, its

most even cut is $\{A, B\}$ against $\{C, D, E\}$, 22 against 17, which leaves the common letter A with two bits and costs 89 bits in all, against the 87 of the code in Figure 3.6.

3.2.5 Huffman Coding

Remark 3.2.5 (History). In 1951 Fano taught a graduate course on information theory at MIT and gave his students a choice: take the final exam, or write a term paper on finding the most efficient binary code. He did not mention that he and Shannon had been working on that problem themselves. David Huffman, then 25, chose the paper and struggled with it for months. With the exam days away he gave up, threw his notes in the bin and started to study. At that moment he had an idea, wrote it up, and it was good enough that he was excused from the final. It wasn't just good, it was optimal. He published the method in 1952, never patented it, and said later that he might never have tried had he known his professor had struggled with the problem.

How do we find the best prefix-free code? Two ideas point the way: common letters should get the shortest codewords, and which part of the tree to extend should be decided by frequency. Suppose a tree already gives $A \to 0$, $B \to 10$, $C \to 11$, and a fourth letter D must be added. Splitting C's leaf gives $C \to 110$ and $D \to 111$; splitting A's gives $A \to 00$ and $D \to 01$, leaving B and C at two bits. For the frequencies of Example 3.2.1 the first is better, 1.75 bits against 2, while if all four letters are equally likely the second is, 2 bits against 2.25. The cost is determined by the frequency of *both* the added letter and the letter being split.

Huffman's algorithm makes decisions from the bottom up, focusing on adding a bit that separates the two least common groups. Every merge adds one bit to the codewords of everything being merged, so the cheapest place to spend it is on the two least frequent groups, and the proof below shows the choice never leads to any negative downstream consequences. Where the top-down method tries to keep the code as even as it can, the bottom-up one accepts from the start that the lengths will differ, and puts the long codewords where they cost least.

> **Huffman's algorithm.** Treat each letter as a node weighted by its frequency. Repeatedly merge the two nodes of smallest weight into a new node whose weight is their sum, until one node remains. The resulting binary tree is the code.

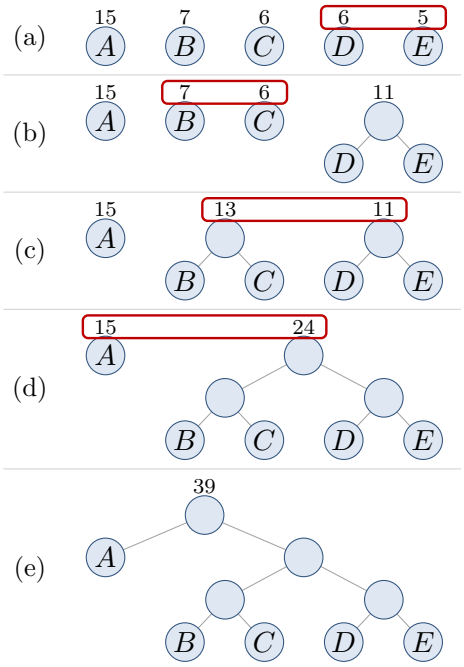


Figure 3.6: Huffman's algorithm on the frequencies A:15, B:7, C:6, D:6, E:5. Each group's weight is written above it, and at every step the two lightest groups, boxed in red, are merged into one whose weight is their sum: $(D, E) \to 11$, then $(B, C) \to 13$, then $(13, 11) \to 24$, then $(A, 24) \to 39$. The rarest letters are merged first, so they end up deepest and receive the longest codewords, while A gets a single bit.

Figure 3.6 runs the algorithm step by step. It gives A a codeword of one bit and the other four letters three bits each, for $15 + 3(24) = 87$ bits, against the 89 of Fano's procedure.

3.2.6 Optimality of Huffman Codes

This proof was not covered in lecture; it is here for anyone who wants to see why the greedy merge is safe.

Theorem 3.2.6. *Among all prefix-free codes for a given set of frequencies, a Huffman code has minimum average codeword length.*

The proof is an induction on the size of the alphabet, using the induction template of Section 1.2. We sketch it in two steps.

First, we argue that there is always an optimal tree where the two least frequent symbols are siblings at maximum depth. If the deepest level contained a letter x that is more frequent than one of the two least frequent letters y, swapping x and y would not increase the average length, because the more frequent letter would move up and the less frequent one down. And any leaf at maximum depth has a sibling, since a node with one child could be replaced by that child, shortening the code.

Now we can do the induction. Merge the two least frequent letters y_1, y_2 into a single symbol z of frequency $\Pr[y_1] + \Pr[y_2]$, which gives an alphabet one smaller. Any tree for the merged alphabet extends to a tree for the original by splitting z, and the two average lengths differ by exactly $\Pr[y_1] + \Pr[y_2]$, the cost of the extra bit given to y_1 and y_2. Since that difference does not depend on the tree, a tree is optimal for the original alphabet exactly when the merged tree is optimal for the smaller one. By the inductive hypothesis

Huffman's algorithm is optimal on the smaller alphabet, and the base case of two letters is immediate.

Looking back

The quantity $\Pr[y_1]+\Pr[y_2]$ in the second step is the observation that drove the algorithm in the first place. Pushing a group one level deeper costs one bit for every letter in it, so it costs the group's total frequency.

Huffman codes are optimal among prefix-free codes for coding letters one at a time. They can still fall short of the entropy, since codeword lengths are whole numbers while $-\log_2 \Pr[x]$ usually is not. Coding blocks of several letters at once closes most of that gap, and Shannon's source coding theorem says the entropy is the limit that no scheme can beat.

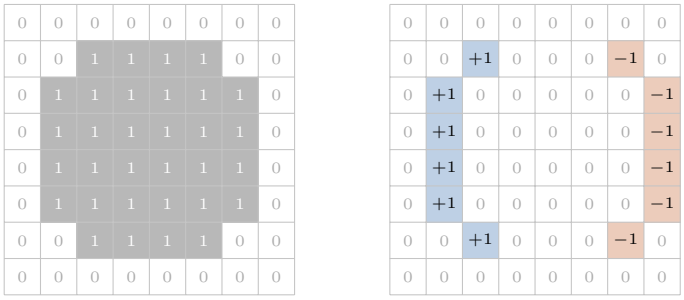
Remark 3.2.7 (What real compressors do). The qualifier "one at a time" is what leaves room to compress even better than Huffman. Huffman needs the letter frequencies in advance, and it treats each letter as independent of the ones around it, so the best it can do is the per-letter entropy. Real text is not like that: `th` is far more likely than `tq`, and the word `the` recurs as a unit. A source with that kind of memory has an *entropy rate* per symbol lower than its per-symbol entropy, and no symbol-by-symbol code can reach it.

The Lempel–Ziv family of compression, which is what actually runs inside `gzip`, `zip` and the PNG format, exploits this. Instead of a codeword per letter it builds a dictionary of substrings as it reads, and replaces any repeat with a short reference back to where the substring appeared before. Nothing needs to be known about the source ahead of time, so one program compresses English, source code and genomes without being told which it is given. In practice the two ideas are combined: `gzip` finds repeats with Lempel–Ziv and then Huffman codes the output of that stage.

3.2.7 Lossless and Lossy Compression

Everything above is *lossless*: the original message is recovered exactly. Before any code is chosen, though, we can perform a transformation on the data that changes what we need to encode.

Figure 3.7 shows a small black-and-white image, a dark disk on a light background, stored as an 8×8 grid of 1s and 0s. Instead of storing the pixels, slide the pattern $[-1, +1]$ along each row, weighting each pixel's left neighbor by -1 and the pixel itself by $+1$, so that every pixel is replaced by its difference from the one before it, much like a derivative. Sliding a pattern along a signal like this is called a *convolution*. What remains is only the outline of the disk, a 1 where each row enters it and a -1 where it leaves, with 0 everywhere else. Nothing is lost, since adding the differences back along each row recovers the image, but 52 of the 64 entries are now 0.



(a) the image (b) the differences

Figure 3.7: (a) An 8×8 image of a dark disk, with 1 for dark and 0 for light. (b) Each pixel replaced by its difference from its left neighbor, with the first pixel of each row compared to 0. Only the outline survives: $+1$ where a row enters the disk, -1 just after it leaves, and 0 everywhere else.

On this tiny grid the entropy per pixel falls only from 1 bit to about 0.88, because the outline is a large share of an 8×8 picture. In larger images, the outline grows with the width of the image and the disk with its area, so a 64×64 version falls from 0.99 bits per pixel to 0.18. As with the letters earlier, coding the differences one at a time would still cost at least a whole bit each, and it is coding them in blocks that turns the lower entropy into a smaller file.

Example 3.2.8. Video is often preprocessed as between-frame differences. This is why a shaky video call is slow while a static shot is not: motion destroys the similarity between consecutive frames, so the differences stop being near zero and the entropy rises. It is also why photographs of forests and foliage take up more space than photographs of a clear sky.

Real images are not this clean. In a photograph of the disk, its pixels would vary slightly in darkness, and every small variation would make a small nonzero difference, filling the inside of the outline with numbers to store. Round every pixel of the disk to exactly black, and those variations are lost, but the eye barely notices them, and the differences collapse back to a thin outline. Giving up exactness in exchange for a much smaller file is *lossy* compression, which is why a photograph can be shrunk far more aggressively than a legal document.

3.3 Error Correction

The last lecture removed redundancy to make messages short. This one puts carefully chosen redundancy back, to make codes more robust to errors. The two ideas are not conflicting opposites: it's perfectly fine to compress first to remove the accidental redundancy, then add structured redundancy that is designed to be useful.

3.3.1 Noisy Channels

Figure 3.8 is the setting for the rest of the lecture. The channel might be a radio link with interference, a scratched disc, or a wire with mechanical faults. We model its effect as flipping some of the bits, and the question is how to design the encoder and decoder so that the flips can be undone. We can think of the engineering problem as building a spell-checker, with the advantage that we get to design the language.

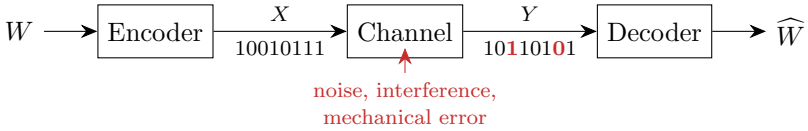


Figure 3.8: The channel model. Alice has a message W, encodes it as X, and the channel delivers a corrupted version Y, here with two bits flipped, shown in red. Bob decodes to $\widehat{W}$ and we want $\widehat{W} = W$.

3.3.2 Hamming Distance

Definition 3.3.1 (Hamming distance)**.** The *Hamming distance* between two bit strings of equal length is the number of positions in which they differ, equivalently the number of bit flips needed to turn one into the other.

Example 3.3.2. The strings 011 0101 00 and 011 1110 00 differ in three positions, so their Hamming distance is 3.

Hamming distance counts errors, which suggests a geometric picture. Take all strings of a given length as points, and join two points when their distance is 1. For length 3 the result is a cube; in general it is called the *hypercube*.

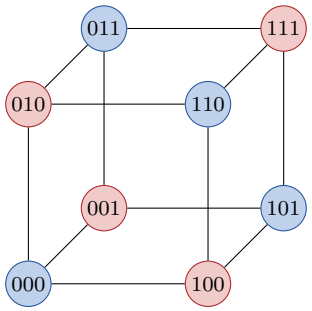


Figure 3.9: The hypercube on strings of length three. Two strings are joined when they differ in one bit, so a single error moves the message to a neighbor. The coloring is by *parity*: blue strings have an even number of 1s and red strings an odd number, and every edge joins one of each.

Figure 3.9 turns error correction into a question about spacing. Suppose we use only some of the strings as legal codewords, and reserve the rest. A single error moves a codeword to one of its neighbors.

- If no two codewords are adjacent, then a single error always lands on a string that is not a codeword, and we can *detect* that something went wrong.
- If the codewords are spaced further apart, then a corrupted string is still closer to the codeword we sent than to any other, and we can *correct* the error by decoding to the nearest codeword. This is autocorrect, in a language we designed.

In general, a code with minimum Hamming distance d between codewords can detect $d - 1$ errors and correct $\lfloor (d-1)/2 \rfloor$ of them.

Looking ahead

The hypercube is our first real use of a graph, and we have taken only the vocabulary we need: vertices, edges, neighbors, distance. Chapter 4 develops graphs properly, and the notion of a set of vertices with no edges between them, which is what the detecting codes above are, turns out to have a name there.

3.3.3 Parity Checks

How can we enforce that our codewords have enough Hamming distance to correct? The naive approach is repetition: Send the message twice (repeating the whole message, 0101 goes to 01010101 or pairwise doubling takes 0101 to 00110011), and if the two copies disagree we know something went wrong, but we cannot tell which copy is right, so we can *detect* one error without *correcting* it. Send the message three times, and we can take a majority vote position by position, which corrects a single error. The price is a threefold increase in length, which is very expensive.

A better idea is parity. The coloring in Figure 3.9 follows a simple rule, parity, and that rule makes good codes cheap to build.

Definition 3.3.3 (Parity)**.** The *parity* of a bit string is whether it contains an even or odd number of 1s. It is computed by addition modulo 2, also called XOR and written $b_1 \oplus b_2$.

Add one extra bit to every message, chosen so that the whole codeword has even parity: the *parity check bit* is 1 if the message had an odd number of 1s and 0 otherwise. Now every legal codeword has even parity, and every single bit flip changes the parity, so any single error is detected. This includes an error in the parity bit itself.

The hypercube illustrates what we are setting up: the codewords are exactly the blue vertices, and no two of them are adjacent. This detects one error but locates nothing. The idea that fixes that is to add more parity bits, each covering only a *subset* of the positions. For five-bit words $b_1b_2b_3b_4b_5$, require the sub-word $b_1b_3b_5$ to have even parity as well. If that check fails, the error happened at an odd position; if it passes, at an even one. The check is a yes/no question: is the error in $\{b_1, b_3, b_5\}$? With well-chosen subsets, the pattern of answers identifies the position exactly.

3.3.4 Hamming Codes

Hamming's idea is to choose the subsets using binary arithmetic. Number the positions $b_0, b_1, b_2, \ldots$ and reserve the positions at powers of two, p_1, p_2, p_4, p_8, as parity bits. Parity bit p_{2^j} covers every position whose index has a 1 in bit j.

Parity bit	Covers positions with	Positions among $1, \ldots, 15$
p_1	a 1 in the last bit of the index	$1, 3, 5, 7, 9, 11, 13, 15$
p_2	a 1 in the second-to-last bit	$2, 3, 6, 7, 10, 11, 14, 15$
p_4	a 1 in the third-to-last bit	$4, 5, 6, 7, 12, 13, 14, 15$
p_8	a 1 in the fourth-to-last bit	$8, 9, 10, 11, 12, 13, 14, 15$

Each parity bit is set so that its own group has even parity, which is possible because p_{2^j} belongs to its own group and to no other parity bit's group. Figure 3.10 lays the whole scheme out, and the next example runs it on a message.

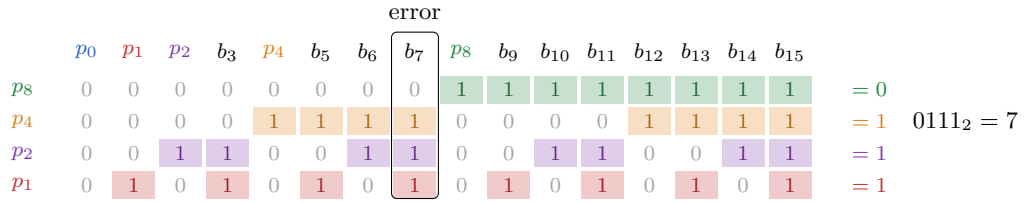


Figure 3.10: The parity groups of a Hamming code. Each of the four rows is one bit of the column index written in binary, and parity bit p_{2^j} checks exactly the colored cells in its row. Position 0 has no 1s in its index, so it belongs to no group, which is what frees it up to serve as an overall parity bit in the extended code. Each p_{2^j} has a single 1 in its own column, so it sits in its own group and in no other. The boxed column is the worked example: a flip at position 7 makes the p_1, p_2 and p_4 checks come back odd and leaves p_8 even, and reading those results as bits gives 0111, the binary index of the error.

Example 3.3.4. Take the eleven message bits 10011010110 and place them in order in the positions that are not powers of two, $3, 5, 6, 7, 9, \ldots, 15$. Each parity bit makes its group even. For instance,

$$p_1 = b_3 \oplus b_5 \oplus b_7 \oplus b_9 \oplus b_{11} \oplus b_{13} \oplus b_{15} = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 1,$$

and in the same way $p_2 = 0$, $p_4 = 1$ and $p_8 = 0$. The overall parity bit p_0, which covers all fifteen other positions and is used by the extended code below, comes out 0. The sixteen bits sent, positions 0 through 15, are

$$0101\ 1001\ 0101\ 0110.$$

Now suppose bit 7 is flipped in transit, so that 0101 1000 0101 0110 arrives, the flipped bit highlighted. Recomputing each group's parity, its own parity bit included,

$$\begin{aligned} p_8 \oplus b_9 \oplus b_{10} \oplus b_{11} \oplus b_{12} \oplus b_{13} \oplus b_{14} \oplus b_{15} &= 0, \\ p_4 \oplus b_5 \oplus b_6 \oplus b_7 \oplus b_{12} \oplus b_{13} \oplus b_{14} \oplus b_{15} &= 1, \\ p_2 \oplus b_3 \oplus b_6 \oplus b_7 \oplus b_{10} \oplus b_{11} \oplus b_{14} \oplus b_{15} &= 1, \\ p_1 \oplus b_3 \oplus b_5 \oplus b_7 \oplus b_9 \oplus b_{11} \oplus b_{13} \oplus b_{15} &= 1. \end{aligned}$$

Reading the results from top to bottom as a binary number, $0111 = 4 + 2 + 1 = 7$, so the error is at position 7: flip it back and the message is recovered. The overall parity of what arrived is odd, as it should be after a single error. If all four checks come back even, no single error occurred.

This looks like magic, but it is by design. Each group's parity started at 0 and changes only if that group contains the error, and the groups are exactly the positions with a 1 in a given binary digit of their index: p_1 watches the last digit, p_2 the second to last, p_4 the third and p_8 the fourth. So the four checks spell out the binary index of the error, one digit each.

The construction is an information-theoretic idea in disguise. Each parity check is a yes/no question about where the error is, and each one is designed to split the possible positions in half, which is the condition from Section 3.1 for a question to deliver a full bit. Four such questions locate one error among fifteen positions, and no scheme with fewer questions could.

Remark 3.3.5 (Rates and extensions). Using positions 1 through 15 with four parity bits leaves eleven positions for the message, which is called a $(15, 11)$ Hamming code: fifteen total bits carrying eleven bits of information.

That is the best possible. Theorem 1.3.17 bounds a single-error-correcting code of length 15 by $2^{15}/(1+15) = 2^{11}$ codewords, and this code has exactly 2^{11} of them, so the counting argument from Lecture 3 is tight here. The same holds for the $(7, 4)$ code, which meets the bound of 16.

If we also keep position 0 as an overall parity bit p_0 covering everything, we get the *extended* Hamming code. With two errors, the four group checks will point somewhere, but the overall parity comes back even, which tells us that two errors occurred and that we should not attempt a correction. So the extended code corrects one error and detects two.

3.3.5 Comments and Further Reading

The number of parity bits grows like $\log_2 n$, which is the real payoff over repetition. A 31-bit code needs only five parity checks, giving $(31, 26)$ for single-error correction, or $(32, 26)$ with an overall parity bit, which also detects two errors. In practice one chooses a block length short enough that two errors in one block are unlikely, since these codes handle only one.

The limit on all of this is Shannon's channel capacity. For a channel with input X and output Y,

$$C = \sup_{p_X} I(X : Y),$$

the largest mutual information achievable over all input distributions. The reasoning behind the supremum (sup) is that the engineer is free to choose an encoding of the words that selects a good probability distribution on X. If some values of X always incur errors, e.g. 000 always fails, then we can purposefully avoid encoding words to that string. The downside, of course, is that we are decreasing $H(X)$ through that choice, which limits the amount of information we can send through the channel because $I(X : Y) \leq H(X)$ (think back to information diagrams, since mutual information is the shared entropy).

The capacity of the channel given here is the maximum rate at which information can be pushed through the channel with vanishing error probability (as the sequence gets longer). Shannon proved that codes reaching this rate exist, but his argument chose codes at random and gave no practical way to build or decode one, so reaching the limit in practice took another forty-five years.

Reed–Solomon codes, from 1960, correct many errors at once and are the codes that actually make scratched CDs and DVDs readable. In 1993 Claude Berrou and his colleagues introduced turbo codes, the first practical codes to come close to the Shannon limit, which is what made 3G and 4G wireless communication possible.

Low-density parity-check codes do as well, and they are what Wi-Fi, 5G and satellite television use today. Their idea is close to this lecture's: many parity checks, each covering only a few bits chosen at random, so that the bits and the checks form a random "bipartite graph", as in Figure 3.11. Robert Gallager invented them in his 1960 doctoral thesis at MIT, but decoding them needed more computing than existed, and they were forgotten until David MacKay and Radford Neal rediscovered them in the mid-1990s.

Everything here protects against bits *changing*. Other kinds of damage exist, notably *deletions*, where a bit vanishes and the rest shift over. Deletions are much harder, and are the dominant error mode in DNA storage. Levenshtein showed in 1966 that correcting k deletions needs on the order of $k \log n$ redundant bits, and built codes that do this for a

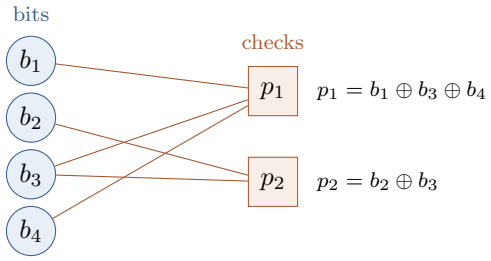


Figure 3.11: A small parity-check graph. Each check covers only a few of the bits. A low-density parity-check code uses thousands of bits and checks joined at random in this way.

single deletion. For more than one, nobody could construct codes within a constant factor of that bound until 2019, when Jin Sima and Jehoshua (Shuki) Bruck at Caltech did. I was in Shuki's lab when they discovered this, though I contributed nothing to the breakthrough...

Chapter 4

Graphs and Random Graphs

Graphs are central to this course, and you have actually already seen them: the hypercube of Section 3.3, whose vertices were bit strings, and the decision trees of Section 3.2, which are graphs of a particular shape. Graphs will appear again as the state diagram of a machine in Chapter 5, as a random process in Chapter 6, and as a record of causal dependence in Chapter 8. This chapter sets up the vocabulary and studies what happens when we make the edges random.

4.1 Introduction to Graphs

A graph abstracts out the concept of things being related, though the interpretation of what "relation" means can be very flexible. Sometimes an edge means "you can get from here to there," and the questions are about connections: is there a route, how short is the shortest one, is there a walk that covers everything once? Sometimes an edge means "these two cannot go together," and the questions are about restrictions: how few colors, time slots or registers are needed? This lecture sets up the vocabulary with Euler's original puzzle, and then goes over both interpretations.

4.1.1 The Bridges of Königsberg

The Prussian city of Königsberg sat on the river Pregel, which split around an island and then forked, dividing the city into four land masses joined by seven bridges. During the early 1700s, it was a popular puzzle to ask whether one could take a walk that crossed every bridge exactly once.

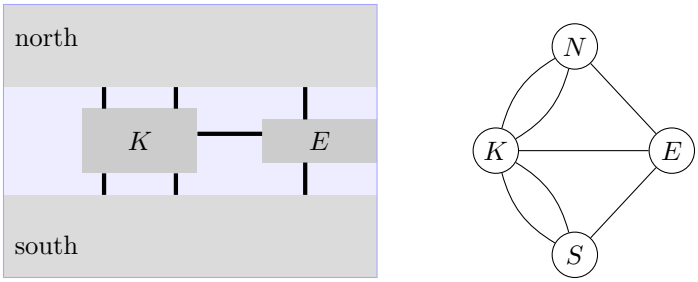


Figure 4.1: Königsberg, and the graph Euler replaced it with. The land masses are the north bank, the south bank, the island K, and the eastern land E. Only the pattern of connections matters for the puzzle, so distances, shapes and positions can be discarded. Two pairs of land masses are joined by two bridges each, so this graph has repeated edges.

In 1736 Leonhard Euler came up with a simple way to think about the puzzle. He observed that nothing about the geography is relevant except which land masses are joined to which. Shrink each land mass to a point, draw each bridge as a line between the appropriate points, and the puzzle becomes Figure 4.1: four points and seven lines.

This structure helped him reason about the problem. Except at the start and the end of the walk, every visit to a land mass arrives on one bridge and departs on another, so the bridges at that land mass get used up in pairs. Any land mass with an odd number of bridges must therefore be the start or the end of the walk. Königsberg has four land masses with an odd number of bridges, and a walk has only two ends, so the puzzle was impossible!

4.1.2 Graphs and Walks

Definition 4.1.1 (Graph)**.** A *graph* $G = (V, E)$ consists of a set V of *vertices* and a set E of *edges*, each edge being a pair of vertices. Vertices joined by an edge are *adjacent*, or *neighbors*.

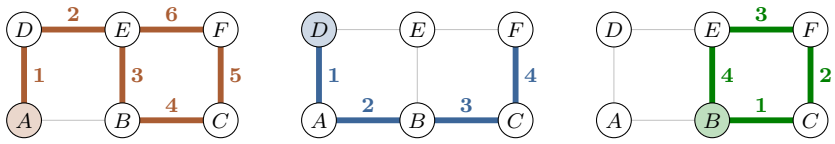
Definition 4.1.2 (Neighborhood and degree)**.** The *neighborhood* $N(v)$ of a vertex v is the set of its neighbors, and its *degree* $\deg(v) = |N(v)|$ is how many there are.

A graph is *simple* if it has no repeated edges and no edge from a vertex to itself. Figure 4.1 is not simple, and a graph with repeated edges is called a *multigraph*. Everything below is about simple graphs unless stated otherwise.

Definition 4.1.3 (Walk, trail, path, cycle)**.** A *walk* is a sequence of vertices $v_0, v_1, \ldots, v_k$ in which consecutive vertices are adjacent. Its *length* is k, the number of edges used, and it is *closed* if $v_k = v_0$. A walk is

- a *trail* if it repeats no edge,
- a *path* if it repeats no vertex,
- a *cycle* if it is closed, has length at least 3, and repeats no vertex other than its start.

Figure 4.2 shows all three on the same graph. The three conditions are nested: a path repeats no vertex and therefore no edge, so every path is a trail, and every trail is a walk.



(a) a trail (b) a path (c) a cycle

Figure 4.2: Three walks on the same graph, with each edge numbered in the order it is used, the vertices lettered, and the starting vertex shaded. (a) A trail: no edge is used twice, but vertex E is visited twice. (b) A path: no vertex is visited twice. (c) A cycle: a closed walk that returns to its start and repeats no other vertex.

Definition 4.1.4 (Connectivity and distance)**.** Vertices u and v are *connected* if some path joins them, and G is connected if every pair is. The maximal connected pieces of a graph are its *connected components*. The *distance* $d(u, v)$ is the length of a shortest path from u to v, or ∞ if there is none.

Figure 4.5 shows a connected graph and one with two components.

Proposition 4.1.5. *If there is a walk from u to v, there is a path from u to v.*

Proof. If a walk from u to v repeats a vertex, cut out the cycle between its first and last visits to that vertex. What remains is still a walk from u to v, and it is shorter. Repeating this must stop, since the walk keeps getting shorter, and it stops only when no vertex repeats, which is a path. □

Because of Proposition 4.1.5, "connected" could just as well have been defined with walks. The loop cut out in the proof will become an important concept when we talk about the pumping lemma for regular languages.

Distances in real networks are surprisingly short. On the same Facebook graph as in the handshake example below, Backstrom, Boldi, Rosa, Ugander and Vigna found in a 2012 paper that the average distance between two active users was 4.74: fewer than four people in between, rather than the famous six degrees of separation.

4.1.3 The Handshake Lemma

How can we estimate the number of handshakes at a party when nobody witnessed every handshake, but every guest knows how many hands they shook? Adding up the number of handshakes reported by each person counts each handshake twice, once by each of the two people involved, so we can get the answer by dividing by two.

The same arithmetic is how a company like Facebook can count its friendships. Each user's friend count is stored with that user, so the counts can be added up server by server, without anyone ever examining pairs of users. In May 2011 Facebook's researchers (Ugander, Karrer, Backstrom and Marlow) measured 721 million active users and 68.7 billion friendships, and the numbers fit the lemma: 2×68.7 billion divided by 721 million is about 190, the average number of friends they reported.

Theorem 4.1.6 (Handshake lemma)**.** *For any graph $G = (V, E)$,*

$$\sum_{v \in V} \deg(v) = 2\,|E|\,.$$

Corollary 4.1.7. *Every graph has an even number of vertices of odd degree.*

Proof. The total in Theorem 4.1.6 is even, so we need an even number of odd contributions to that sum. □

The corollary rules certain settings out. Seven people cannot each shake hands with exactly three others, since the degrees would add up to 21, which is odd; for the same reason nine machines cannot each have exactly three links. The same parity, applied to each land mass, is what settles Königsberg.

Theorem 4.1.8 (Euler, 1736)**.** *A connected multigraph has a trail using every edge exactly once if and only if the number of vertices of odd degree is* 0 *or* 2*. Such a trail is called an* Eulerian trail*.*

We proved the forward direction above, which is the direction that is needed for Königsberg. The converse is a construction: given the degree condition, one can build such a walk by repeatedly following unused edges and splicing in detours.

4.1.4 Graphs as Connections

A graph edge can mean you can get from one vertex to another, yeilding questions about routes: is there a path, how short is the shortest one, is there a walk that covers everything exactly once? Königsberg falls under this category. Let's look at a few more...

Flights. Take airports as vertices and join two when there is a direct flight between them, as in Figure 4.3. Whether you can fly from one city to another is a question about connectivity, and the fewest connections is a distance.

This setting also gives rise to a famously hard problem: a *traveling salesman* wants to visit every city once and return home as cheaply as possible, which asks for a cycle through every vertex of least total cost. For a few cities, you can brute force every order, but the number of orders grows like a factorial, and no efficient method for the general problem is known.

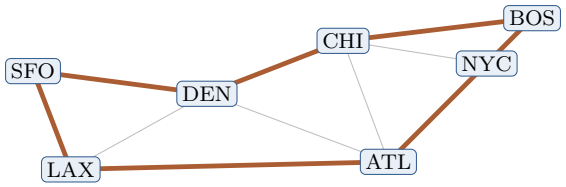


Figure 4.3: A flight network: airports are vertices and an edge is a direct flight. The highlighted edges form a round trip visiting every city once, the kind of cycle the traveling salesman problem asks for once each flight has a cost.

Errors. The hypercube of Section 3.3 also represents reachability from bit flips. Its vertices are bit strings and its edges are single bit flips, so a path of length d is a run of d errors and the Hamming distance is the distance in the graph.

Puzzles and games. Any puzzle can be drawn as a graph, with a vertex for each state and an edge for each legal move. Solving it means finding a path from the starting state to the goal, and the puzzle is unsolvable when the two lie in different connected components. The 2×2 sliding puzzle, with three tiles and one gap, is small enough to draw completely, in Figure 4.4. Each state allows exactly two moves, sliding one of the two tiles next to the gap into it, so every connected component is a cycle. There are $4! = 24$ arrangements in all, and they split into two cycles of 12: half of the arrangements can never be reached from the solved one, however you slide. Swapping two tiles gives one such (normally) unreachable states.

The same picture describes the Rubik's cube. Its graph has about 4.3×10^{19} vertices, one for each reachable arrangement, and in 2010 a computer search showed that every one of them is within 20 moves of solved. Games with two players work the same way, with positions as vertices, moves as edges, and the winning positions for each player as the goals.

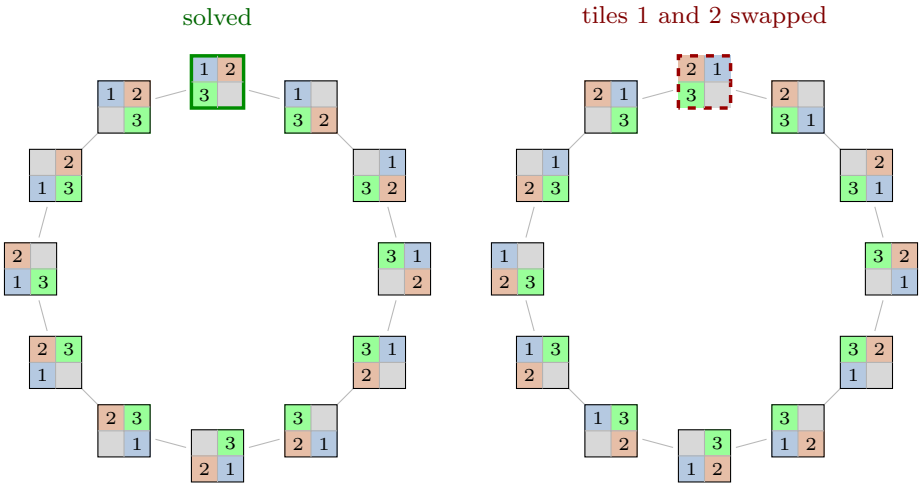
Trees. The leanest connected graphs have no cycles, so that any two vertices are joined by exactly one path.

Definition 4.1.9 (Tree)**.** A *tree* is a connected graph with no cycles, and a vertex of degree 1 in a tree is a *leaf*.

The decision trees of Section 3.2 were trees in this sense, rooted, with the letters at the leaves.

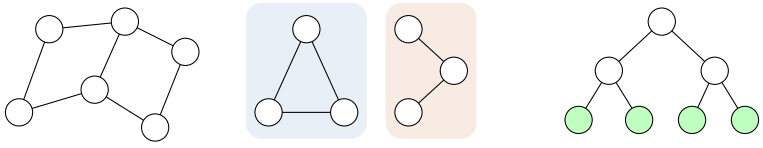
4.1.5 Graphs as Restrictions

In the second reading an edge means that two things cannot go together, and the questions are about assignment.



(a) reachable from solved (b) the other twelve

Figure 4.4: The 2×2 sliding puzzle as a graph, with all 24 arrangements. Each vertex is an arrangement, with the gap in gray, and each edge slides one tile into the gap; the tiles are colored so that their movement can be followed. The arrangements split into two cycles of twelve that never meet: (a) contains the solved arrangement, and (b) contains the one with tiles 1 and 2 swapped, so no sequence of slides turns one into the other.



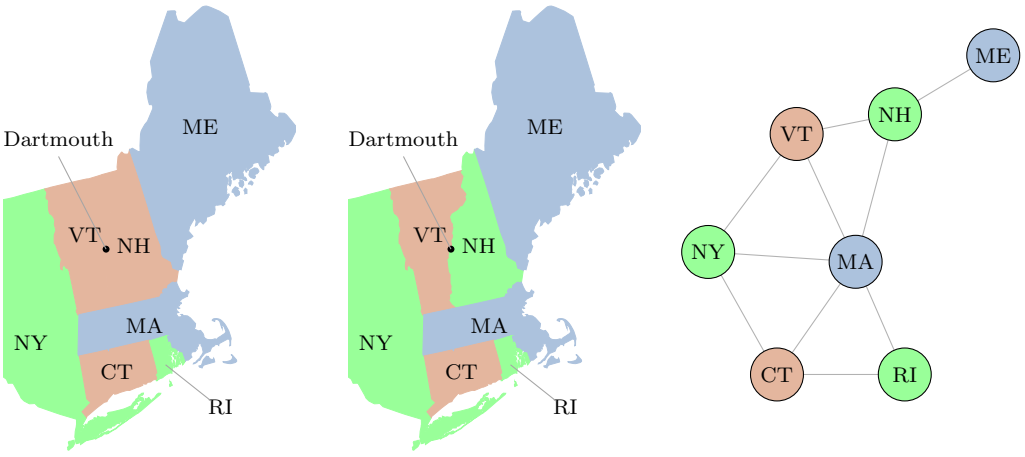
(a) connected (b) two components (c) a tree

Figure 4.5: (a) A connected graph: every pair of vertices is joined by a path. (b) A graph with two connected components, shaded. (c) A tree, connected and with no cycles; its leaves, the vertices of degree 1, are shaded.

Definition 4.1.10 (Proper coloring)**.** A *proper coloring* of G assigns a color to each vertex so that adjacent vertices get different colors. The *chromatic number* $\chi(G)$ is the fewest colors for which this is possible.

Example 4.1.11 (Map coloring)**.** Color a map so that states sharing a border get different colors. If two neighbors get the same color, the border between them disappears: in Figure 4.6(a), Vermont and New Hampshire read as one state, and the river that Dartmouth sits beside no longer marks a border. To turn this into a graph, make each state a vertex and join two states when they share a border, as in Figure 4.6(c). A good map coloring is exactly a proper coloring of that graph. Three colors suffice here, and you canot do any better, since Massachusetts, Connecticut and Rhode Island all border one another.

A famous result is the four color theorem: any map drawn in the plane without overlapping edges (all the maps I can think of follow this) can be colored with four colors. Conjectured in 1852, it resisted proof until 1976, and the proof was the first major theorem to depend on computer verification of cases (which means it took some time to verify).



(a) VT and NH alike (b) a proper coloring (c) its graph

Figure 4.6: (a) Coloring Vermont and New Hampshire alike erases the border between them. (b) A proper coloring, with every border visible. (c) The graph of the map, with one vertex per state and an edge when two states share a border, colored as in (b). The outlines come from a public-domain map of the United States on Wikimedia Commons.

The reason three colors were needed generalizes into a fact about fully connected subcomponents called cliques. If k vertices are all adjacent to one another, they need k different colors, so $\chi(G)$ is at least the largest number of mutually adjacent vertices.

Each color in a proper coloring picks out a set of vertices with no edges among them, so a coloring with k colors splits the vertices into k such "independent sets."

Definition 4.1.12 (Independent set)**.** An *independent set* is a set of vertices with no edges among them.

When two colors suffice, each color class is an independent set and every edge runs between the two classes.

Definition 4.1.13 (Bipartite graph)**.** A graph is *bipartite* if its vertices split into two sets with every edge going between the sets, which is the same as having a proper coloring with two colors.

A graph is bipartite if and only iff it has no cycle of odd length. One direction is easy: going around a cycle in a bipartite graph alternates between the two sides, so getting back to the start takes an even number of steps. The other direction holds too, though we will not prove it.

The hypercube of Section 3.3 is also an example of a restriction graph as well. Its parity coloring, blue for even and red for odd, is a proper coloring with two colors, so the hypercube is bipartite. Its cycles are all even, as they must be: a closed walk has to flip every bit an even number of times to get back to where it started. A code that detects single errors is exactly an independent set: no two codewords are adjacent, so no single bit flip can turn one codeword into another. Spreading the codewords further apart, so that not even the neighbors of different codewords overlap, is what lets a code correct errors as well.

Example 4.1.14 (Scheduling)**.** A lab must schedule experiments that share equipment. Make each experiment a vertex and join two when they need the same instrument, so they cannot run at the same time. A proper coloring assigns time slots, and $\chi(G)$ is the fewest slots that work; Figure 4.7 is a small instance. The same setup schedules exams so that no student has two at once, with courses as vertices and an edge indicating that a student takes both courses.

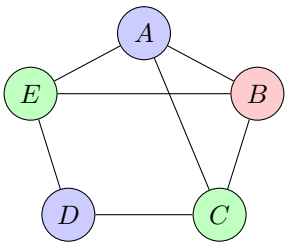
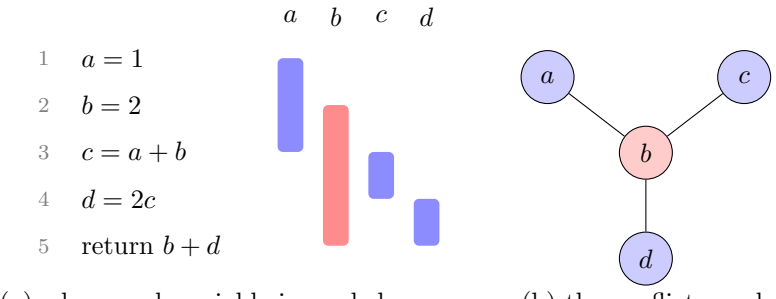


Figure 4.7: Five experiments, with an edge whenever two of them need the same instrument. The shading is a proper coloring with three colors, so three time slots suffice. Two colors would not: A, B, E are mutually adjacent, and three mutually adjacent vertices always need three colors.



(a) where each variable is needed (b) the conflict graph

Figure 4.8: (a) Each bar runs from the line where a variable is defined to the line where it is last used; line 3 reads a before writing c, so their bars only touch. (b) Variables whose bars overlap are joined. Two colors suffice: b in one register, and a, c and d, whose bars never overlap, sharing the other.

Example 4.1.15 (Register allocation)**.** A processor computes with a small number of registers, and a compiler must decide which variable lives in which. Two variables that are

needed at the same time cannot share a register. In the program

$$a = 1; \quad b = 2; \quad c = a + b; \quad d = 2c; \quad \text{return } b + d,$$

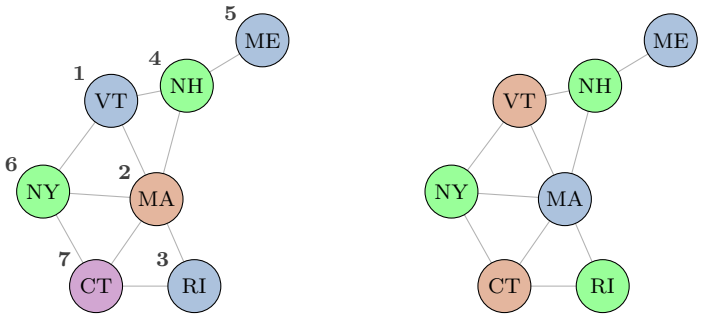
the variable b is needed from its definition to the last line, overlapping each of a, c and d, while no two of a, c and d are ever needed at the same time. Joining variables that overlap gives a graph with b adjacent to the other three (Figure 4.8), which two colors color properly, so two registers suffice: one for b and one shared by a, c and d. When the graph needs more colors than there are registers, some variables must be kept in memory instead, which is very slow. It turns out that most compilers are running some kind of graph coloring in the background to resolve this!

Coloring greedily. Both examples call for an algorithm, and the obvious one is greedy: take the vertices in some order, and give each the smallest color not already used by its colored neighbors.

Theorem 4.1.16 (Greedy coloring)**.** *Greedy coloring, in any order, uses at most* $\Delta + 1$ *colors, where* Δ *is the largest degree in* G*. In particular* $\chi(G) \leq \Delta + 1$*.*

Proof. When a vertex v is colored, at most $\deg(v) \leq \Delta$ of its neighbors already have colors, so at least one of the colors $1, \ldots, \Delta + 1$ is free for it. □

The theorem bounds greedy coloring, but it turns out this is not necessarily optimal. New England needs only three colors, but greedy coloring in the order VT, MA, RI, NH, ME, NY, CT uses four, as Figure 4.9 shows: by the time Connecticut is reached, its neighbors already hold three different colors. In alphabetical order greedy uses three. Of all 5040 orders of the seven states, 2298 waste a color, so this is not bad luck with one unusual order.



(a) greedy in a bad order: four colors (b) three colors suffice

Figure 4.9: Greedy coloring of the New England map. (a) In the order VT, MA, RI, NH, ME, NY, CT, numbered, each state takes the smallest color its colored neighbors leave free. By the time Connecticut is reached, New York, Massachusetts and Rhode Island hold three different colors, so it needs a fourth. (b) A proper coloring with three colors, as in Figure 4.6.

Two colorings do not suffer from the limitations of greedy coloring. In a connected graph, color any vertex, then keep giving each uncolored neighbor of a colored vertex the opposite color. Every vertex's color is forced, so this succeeds exactly when the graph is bipartite and therefore two-colorable. It turns out that the difficulty emerges once you need three colors.

Remark 4.1.17 (Hard problems). Four colors always suffice for maps on a plane (planar graphs), but for general graphs even deciding whether three colors suffice is hard. It is *NP-complete*, just like the decision version of the traveling salesman problem, which asks whether there is a tour costing at most a given budget. Being NP-complete means that each can be translated into the other efficiently, so a fast algorithm for either would give fast algorithms for both. There are tons of NP-complete problems, and an algorithm to solve any of them efficiently would essentially solve all of them. Whether such an algorithm exists is the P versus NP question, one of the major open problems in mathematics.

4.1.6 Vocabulary

The terms of this lecture, and where each was introduced:

graph, adjacent, neighbor	Definition 4.1.1
neighborhood, degree	Definition 4.1.2
walk, trail, path, cycle	Definition 4.1.3
connected, component, distance	Definition 4.1.4
tree, leaf	Definition 4.1.9
proper coloring, chromatic number	Definition 4.1.10
independent set	Definition 4.1.12
bipartite graph	Definition 4.1.13

4.2 Random Graphs and Thresholds

Networks often form or break at random: contacts and friendships form without any plan, and storms, earthquakes and wear break the lines of power grids and communication networks. Studying random graphs helps us plan for epidemics, respond to breakdowns of the electrical grid, and understand physical properties such as why materials shatter. This will help us address one of the questions from Chapter 0: why do diseases and social media posts go viral?

Somewhat surprisingly, connectivity in a random graph does not improve gradually as edges are added. Figure 4.10 shows a random graph growing. For a long time there are only small pieces; then, over a narrow range of edge density, one giant component appears; and a little later the last stragglers join it. This lecture finds those thresholds by counting expected structures, and the next explains the giant component.

4.2.1 From Expectations to Probabilities

Everything we are about to do follows the same high-level steps: compute the expected number of some structure, and conclude that the structure is either certainly absent or surely present. The tool that enables this is Markov's inequality from Section 2.5, which turns a statement about an average into a statement about a probability without knowing anything else about the distribution.

Corollary 4.2.1 (The first moment method)**.** *If X is a nonnegative integer, then* $\Pr[X \geq 1] \leq \mathbb{E}[X]$. *In particular, if* $\mathbb{E}[X] \to 0$ *then the probability that any structure counted by X exists tends to zero.*

Corollary 4.2.1 gives a method: to show that some structure almost surely does *not* appear, count how many of them we expect and show the expectation is small. Equivalently, one can bound the probability that any one of them appears by the union bound, which gives the same estimate.

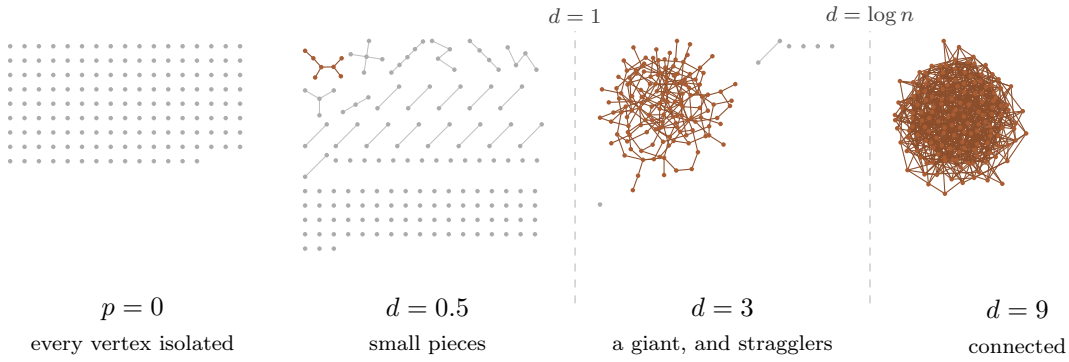


Figure 4.10: A random graph (simulated using Python) on $n = 150$ vertices as its average degree d grows, with the largest component in orange. With no edges every vertex is isolated. Below $d = 1$ there are only small pieces, each of expected size at most $1/(1-d)$. Once d passes 1 a giant component appears, here with 143 of the 150 vertices, while a few stragglers stay isolated. Once d passes $\log n \approx 5$ the stragglers are gone and the graph is connected.

This is powerful: expectation is linear[1], so finding an expected count doesn't require understanding how multiple occuring events interact. Of course, the approach gives a bound, so its possible that the probability goes to zero earlier or faster than the expected value goes to zero. Still, its often enough to get a feel for what is going on.

4.2.2 Birthdays and Collisions

The simplest threshold can be found in a problem we have already studied. Place n items into s slots at random: n people into $s = 365$ birthdays, or n records into a hash table with s locations. By Example 2.3.8, the expected number of colliding pairs is

$$\mathbb{E}[\text{collisions}] = \binom{n}{2}\frac{1}{s} \approx \frac{n^2}{2s},$$

because the number of pairs grows quadratically while each pair's chance of colliding stays fixed. Now let the space grow with the number of items, so that $s \approx n^a$. The expected number of collisions is then about $n^{2-a}/2$, which goes to infinity if $a < 2$, stays constant if $a = 2$, and goes to zero if $a > 2$. To avoid collisions the space must grow like the square of the number of items: doubling the items takes four times as many slots.

4.2.3 The Erdős–Rényi Random Graph

Definition 4.2.2 (Erdős–Rényi graphs)**.** The random graph $G(n,p)$ has n vertices, and each of the $\binom{n}{2}$ possible edges is present independently with probability p.

Everything we need to know about $G(n,p)$ can be computed with the tools of Chapter 2. The number of edges is a sum of $\binom{n}{2}$ independent indicators, so by linearity of expectation

$$\mathbb{E}[|E|] = \binom{n}{2}p,$$

and the expected degree of a vertex is $(n-1)p$. Write $d = (n-1)p$ for that expected degree, since it is the quantity that controls everything below.

[1]Are you getting tired of hearing me say this yet?

4.2.4 Small Structures

A structure with k vertices and m edges has about n^k possible copies, by Remark 1.3.11, each present with probability p^m, so it first appears around

$$p \approx n^{-k/m},$$

A single edge, with $k = 2$ and $m = 1$, appears at $p \approx n^{-2}$, which is a reconstruction of the birthday problem result, with $s \approx n^2$. A triangle, with $k = m = 3$, appears at $p \approx 1/n$.

4.2.5 Isolated Vertices

Now consider isolated vertices, i.e., "hermits" with no edges at all. A given vertex is isolated when all $n - 1$ of its potential edges are absent, which has probability $(1 - p)^{n-1}$. Let X count the isolated vertices. Indicators and linearity give

$$\mathbb{E}[X] = n(1 - p)^{n-1}.$$

Now set $p = c \log n/n$ for a constant c and use $(1-p)^{n-1} \approx e^{-pn}$, from Proposition 2.4.8, which applies because $np^2 = c^2 \log^2 n/n$ is small and because $n - 1 \approx n$ when n is very large:

$$\mathbb{E}[X] \approx ne^{-c \log n} = n \cdot n^{-c} = n^{1-c}.$$

Notice that the behavior switches when $c = 1$. If $c > 1$ then $\mathbb{E}[X] \to 0$, so by Markov's inequality the probability of even one isolated vertex tends to zero. If $c < 1$ then $\mathbb{E}[X] \to \infty$, but a large expectation alone does not show that isolated vertices are really present rather than merely expected; that takes a second-moment argument, using Chebyshev's inequality, which we do not carry out. It confirms that

$$p = \frac{\log n}{n}$$

is the threshold for isolated vertices: below it they are present, and above it they are gone.

The $\log n$ should remind you of Example 2.4.11. Covering every one of n things by random sampling costs $\log n$ times more than covering a typical one due to the increasing time to collect each sequential coupon. In this case, the coupons are vertices, each needing at least one edge.

4.2.6 The Connectivity Threshold

Having no isolated vertices is necessary for connectivity but not sufficient, since the graph could still split into two larger pieces. To show that it is connected, we must rule out *every* way it could fall apart.

A graph is disconnected exactly when its vertices split into two nonempty parts with no edge crossing between them. So let

$$X = \#\big\{\, S \subseteq V : \emptyset \neq S,\ S \neq V, \text{ no edge joins } S \text{ to } V \setminus S \,\big\},$$

which counts the ways the graph could be cut in two. Figure 4.11 shows an example split. The graph is disconnected if and only if $X \geq 1$, so Corollary 4.2.1 applies once we can compute $\mathbb{E}[X]$.

Theorem 4.2.3. *If $p = c \log n/n$ with $c > 2$, then $\Pr[G(n, p)$ is disconnected$] \to 0$ as $n \to \infty$.*

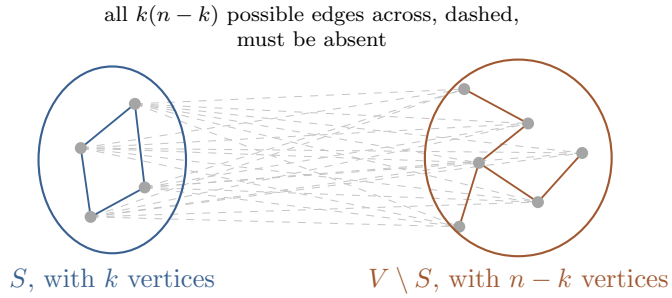


Figure 4.11: A graph is disconnected exactly when its vertices split into a set S and its complement $V \setminus S$ with no edge between them. If $|S| = k$, there are $k(n-k)$ possible edges across, drawn dashed, and all of them must be absent, which happens with probability $(1-p)^{k(n-k)}$.

Proof. Write X as a sum of indicators, one for each candidate set S, and apply linearity of expectation. A set S with $|S| = k$ has $k(n-k)$ potential edges leaving it, each absent with probability $1-p$ independently, so

$$\mathbb{E}[X] = \sum_{k=1}^{n-1} \binom{n}{k} (1-p)^{k(n-k)} \;\le\; 2 \sum_{k=1}^{n/2} \binom{n}{k} (1-p)^{k(n-k)},$$

using the symmetry $S \leftrightarrow V \setminus S$ to restrict to $k \le n/2$.

Now estimate the two factors. For the first, $\binom{n}{k} \le n^k$, by Remark 1.3.11. For the second, Proposition 2.4.8 gives $1-p \le e^{-p}$, and $k \le n/2$ gives $n-k \ge n/2$, so

$$(1-p)^{k(n-k)} \le e^{-pk(n-k)} \le e^{-pkn/2}.$$

With $p = c \log n / n$ we have $e^{-pn/2} = e^{-(c/2)\log n} = n^{-c/2}$, and therefore

$$\mathbb{E}[X] \;\le\; 2 \sum_{k \ge 1} \left(n \cdot n^{-c/2} \right)^k = 2 \sum_{k \ge 1} \left(n^{1-c/2} \right)^k.$$

When $c > 2$ the ratio $n^{1-c/2}$ tends to zero, so the geometric series tends to zero and $\mathbb{E}[X] \to 0$. By Corollary 4.2.1, $\Pr[X \ge 1] \to 0$. □

The arguments for hermits and for divisions parallel eachother in structure: the expected number of isolated vertices dissappears once $c > 1$ and the expected number of cuts once $c > 2$. Neither shows that the graph is disconnected below a threshold, which would need the second moment. Erdős and Rényi showed that the true threshold for connectivity is $c = 1$, which is the same as the isolated vertices threshold. This means that isolated vertices are the last obstacle to disappear when connecting a graph randomly.

> *Remark* 4.2.4. The two calculations in this section are even more closely related than their structure. The $k = 1$ term of the sum for $\mathbb{E}[X]$ is
>
> $$\binom{n}{1} (1-p)^{1 \cdot (n-1)} = n(1-p)^{n-1},$$
>
> which is exactly the expected number of isolated vertices from Section 4.2: a single vertex cut off from the rest is the smallest way for the graph to be disconnected. That term alone switches over at $c = 1$, which is also the true threshold. All the slack in Theorem 4.2.3

therefore comes from the other terms, where we applied fairly crude bounds. Replacing $\binom{n}{k}$ by n^k is generous for k near $n/2$, and bounding $n-k$ below by $n/2$ throws away most of the decay for small k.

4.2.7 The Giant Component

The fact that isolated vertices disappear at the same time the graph connects suggests that the graph almost always becomes connected at the very moment its last isolated vertex receives an edge. A random network's connectivity is decided by its single loneliest vertex, which suggests connecting the world through electricity and internet will require individuals joining in, rather than two large sub-networks merging.

Right before we connect, we have a large sub-graph with many individuals waiting to join in. Can we show when we transition to having this sub-graph? It turns out we can, and it ends up being degree $d = 1$, which is well before the $d \approx \log(n)$ transition for full connectivity. Below this threshold, every component is small: Section 4.3 proves that the component of a typical vertex has expected size at most $1/(1-d)$, no matter how large the graph. Above the threshold, an exploration from a typical vertex keeps finding new vertices rather than running out, and a single *giant* component appears, containing a positive fraction of all n vertices.

4.2.8 Summary

Figure 4.12 summarizes the lecture, and Figure 4.10 showed the same stages in real random graphs. Below $d = 1$ a random graph is only small pieces, of expected size at most $1/(1-d)$. Between $d = 1$ and $d = \log n$, that is, between $p \approx 1/n$ and $p \approx \log n/n$, one giant component holds most of the vertices while a few hermits remain. Above $d = \log n$ the hermits are gone and the graph is connected.

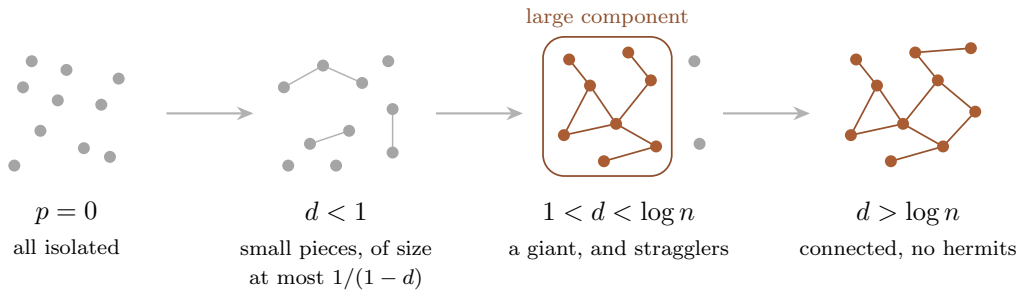
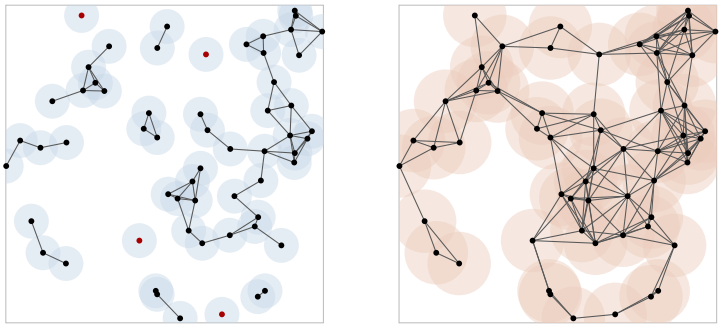


Figure 4.12: The life of a random graph, as its average degree $d = (n-1)p$ grows. With no edges every vertex is isolated. Below $d = 1$, that is $p < 1/(n-1)$, there are only small pieces, of expected size at most $1/(1-d)$ (Theorem 4.3.1). Above $d = 1$ one large component appears, while a few hermits stay isolated, and above $d = \log n$, that is $p > \log n/n$, they are gone and the graph is connected (Theorem 4.2.3 proves this above $2\log n/n$).

4.2.9 Implications

Example 4.2.5 (Random sensors)**.** Suppose 10,000 devices each link to random others. For most of the network to hang together, an average of just over one link per device is enough, as the giant component showed. For *no* device to be cut off, the expected degree has to reach about $\log 10{,}000 \approx 9.2$. Wireless sensor networks face exactly this choice. Scatter n sensors at random in a region of unit area, and let each reach every sensor within distance

r, which is set by its transmission power. Gupta and Kumar showed in 1998 that the network is connected, with probability tending to one, exactly when $\pi r^2 = (\log n + c_n)/n$ with $c_n \to \infty$. That says each sensor's expected number of neighbors must be $\log n + c_n$: the same $\log n$ as before. Less power leaves sensors stranded, and more wastes battery and creates interference. Figure 4.13 illustrates the effect of choosing a large enough radius, which exhibits a phase change from disconnected networks to fully connected ones.



(a) a small radius: 12 pieces (b) a larger radius: connected

Figure 4.13: Sixty sensors scattered at random in a square, each linked to every sensor within distance r of it, the range set by its transmission power. Each disk has radius $r/2$, so two disks overlap exactly when their sensors are linked. (a) With a small radius the network is in 12 pieces, and the four sensors in red are cut off entirely. (b) With a larger radius every sensor is connected. The threshold of Gupta and Kumar sits between the two, where $\pi r^2 \approx \log n/n$, here at $r \approx 0.15$ for a square of side 1.

Example 4.2.6 (The emergence of the internet)**.** The same reasoning explains how networks built from local, uncoordinated links can connect almost everyone without a central plan, as the internet did. No network needs to know the whole map: once each one has a few links to the rest, isolated pockets become rare. Real internet links are neither independent nor equally likely, and its hubs make a big difference, as Remark 4.2.8 explains, but the threshold intuition carries over.

Example 4.2.7 (Ranking runners)**.** I built a ranking system for runners, `lacctic.com`, that compares runners through their finishes in shared races. Think about a graph that draws an edge between two runners whenever they have finished in the same race. A natural worry is that islands (disconnected components) could form, meaning groups of runners cannot be compared. Real race networks are not $G(n, p)$, since runners cluster by region, age and level, which makes islands more likely. Just like the internet example, runner networks have "hubs" with may connections.

The last two examples end up connected because of large degree "hubs", which are not modeled by $G(n, p)$ but can be modeled using more sophisticated random graphs like Barabsi-Albert graphs.

> *Remark* 4.2.8 (Model assumptions). Every result in this lecture rests on the assumptions of $G(n, p)$: each possible edge is present with the same probability p, independently of every other. In real networks, edges sometimes cluster, as runners do by region and age and sensors do by distance, so islands can be more likely than the model predicts. Degrees also vary far more than in $G(n, p)$, where every degree stays close to their expected degree d: many real networks have *hubs*, a few very well-connected vertices, like the big races that

hold a race network together. In 1999 Albert-László Barabási and Réka Albert proposed a random graph that grows by *preferential attachment*: each new vertex links to existing ones with probability proportional to their degrees, so the well connected become better connected, and the fraction of vertices of degree k falls off only like k^{-3}. Hubs change how a network fails. Albert, Jeong and Barabási showed in 2000 that such a graph shrugs off random failures but falls apart quickly when its hubs are attacked. Neither model is simply right, and how often real networks have true hubs is still debated. Choosing a good model and justifying its assumptions is an important part of stochastic modeling, and something we are aiming to build intuition on during this course.

4.2.10 Growing Versus Breaking

Instead of randomly *adding* edges, we can also think of $G(n,p)$ as starting with a complete graph (all $\binom{n}{p}$ edges present) and removing edges with probability $1-p$. *Percolation* studies this process on more general and less connected structures.

The removed can represent failed bonds in a material, which is how things shatter; or failed lines in a network, which is how engineers first model the reliability of power grids and communication networks. Alternatively, we can think of the kept edges as the contacts along which a disease passes, which is how epidemics spread, the subject of the last section of this lecture and of the next one.

4.3 Branching Processes and Epidemics

Section 4.2 showed that a random graph connects by picking up its last few isolated vertices, not by joining two large groups. This lecture studies when the large component that everyone ends up joining actually emerges, which is mathematically equivalent to asking the question of when an epidemic takes off.

4.3.1 How Big Is a Component?

Fix a vertex v and let $C(v)$ be its connected component. We bound $\mathbb{E}[|C(v)|]$ by counting the vertices reachable from v, one distance at a time, as in Figure 4.14.

Theorem 4.3.1. *If $d=(n-1)p<1$, then for every vertex v,*

$$\mathbb{E}\big[|C(v)|\big] \;\le\; \frac{1}{1-d}.$$

Proof. Every vertex of $C(v)$ other than v itself is the endpoint of at least one path from v that is present in the graph, so

$$|C(v)| \;\le\; 1+\sum_{k\ge 1} X_k, \qquad X_k = \#\{\text{present paths of length } k \text{ starting at } v\}.$$

Now count. A path of length k from v is a choice of k distinct other vertices in order, so by the product rule of Section 1.3 there are at most $(n-1)(n-2)\cdots(n-k)\le (n-1)^k$ of them. Each is present when all k of its edges are, which by independence has probability p^k. Writing X_k as a sum of indicators, one per candidate path, linearity of expectation from Section 2.3 gives

$$\mathbb{E}[X_k] \;\le\; (n-1)^k p^k = d^k.$$

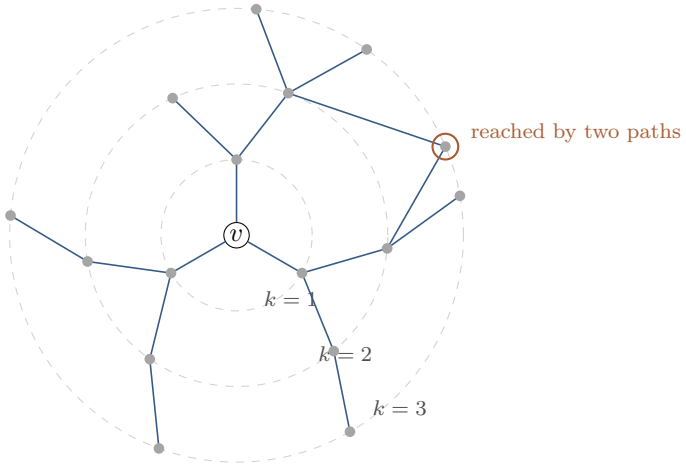


Figure 4.14: Counting the component of v by paths. Every vertex of $C(v)$ other than v ends at least one path from v that is present in the graph, so $|C(v)| \le 1 + \sum_k X_k$, where X_k counts the paths of length k from v. A vertex reached by two different paths, circled, is counted twice, which only makes the bound larger. There are at most $(n-1)^k$ such paths and each is present with probability p^k, so $\mathbb{E}[X_k] \le d^k$.

Summing over k and using linearity again,

$$\mathbb{E}\big[|C(v)|\big] \;\le\; 1 + \sum_{k \ge 1} d^k = \sum_{k \ge 0} d^k = \frac{1}{1-d},$$

where the geometric series converges precisely because $d < 1$. □

The bound does not depend on n, meaning that the expected size of a connected component does not depend on the size of the graph. By Markov's inequality, for any $\epsilon > 0$,

$$\Pr\big[|C(v)| \ge \epsilon n\big] \le \frac{1}{(1-d)\,\epsilon n} \xrightarrow[n \to \infty]{} 0,$$

so for $d < 1$ there is no giant component. This tells us when a large component *disappears*, not when it *emerges*: above $d = 1$ the series diverges and the bound says nothing. To talk about when the component appears, we will need some new mathematical techniques.

4.3.2 Convolutions

A large component can gain vertices in many ways, so we need to understand sums of random variables. Roll two dice:

$$\Pr[X + Y = 7] = \sum_{j=1}^{6} \Pr[X = j]\,\Pr[Y = 7 - j] = \frac{6}{36}.$$

In general, for independent counts X and Y,

$$\Pr[X + Y = k] = \sum_{j} \Pr[X = j]\,\Pr[Y = k - j],$$

the *convolution* of the two distributions, which collects all the ways the two counts can add up to k.

4.3.3 Generating Functions

The same idea of collecting terms shows up when polynomials multiply. In

$$\big(a_0 + a_1 z + a_2 z^2 + \cdots\big)\big(b_0 + b_1 z + b_2 z^2 + \cdots\big),$$

the coefficient of z^k is $\sum_j a_j b_{k-j}$: the convolution of the coefficients. So we store a distribution as the coefficients of a polynomial.

Definition 4.3.2 (Probability generating function)**.** For a random variable X taking values $0, 1, 2, \ldots$, the *probability generating function* is

$$G_X(z) = \mathbb{E}\big[z^X\big] = \sum_{k \geq 0} \Pr[X = k]\, z^k.$$

Since $z^k = e^{k \log z}$, this should remind you of the Laplace transform; in signal processing the same object is the z-transform.

Proposition 4.3.3 (Sums)**.** *If X and Y are independent, then $G_{X+Y} = G_X\, G_Y$.*

Proof. $\mathbb{E}\big[z^{X+Y}\big] = \sum_{a,b} \Pr[X = a] \Pr[Y = b]\, z^a z^b = \Big(\sum_a \Pr[X = a]\, z^a\Big)\Big(\sum_b \Pr[Y = b]\, z^b\Big) = G_X(z)\, G_Y(z)$. □

Proposition 4.3.4 (Mixtures)**.** *If, in disjoint cases $C_1, C_2, \ldots$ with probabilities $w_1, w_2, \ldots$, the count X has generating function G_i in case C_i, then $G_X = \sum_i w_i\, G_i$.*

Proof. By the law of total probability, $\mathbb{E}\big[z^X\big] = \sum_i \Pr[C_i]\, \mathbb{E}\big[z^X \mid C_i\big] = \sum_i w_i\, G_i(z)$. □

These two rules let us find the generating functions of complicated counts from simple ones. For two dice, each has $G(z) = (z + z^2 + \cdots + z^6)/6$, so their total has $G(z)^2$, whose coefficient of z^7 is $6/36$. For a binomial, $B = X_1 + \cdots + X_n$ with each X_i a Bernoulli(p) trial, so $G_{X_i}(z) = (1 - p) + pz = q + pz$ and

$$G_B(z) = (q + pz)^n = \sum_{k=0}^{n} \binom{n}{k} p^k q^{n-k} z^k.$$

Notice that we can sometimes simplify the asymptotics of a generating function (because its just a function), which may help us better simplify an asymptotic expectation.

Three more facts:

- $G(1) = 1$, since the probabilities add up to 1;
- $G(0) = \Pr[X = 0]$, since every term but the first vanishes;
- $G'(1) = \mathbb{E}[X]$, since differentiating z^k gives kz^{k-1}, so $G'(1) = \sum_k k \Pr[X = k]$.

4.3.4 Sums of Random Length

If person i infects D_i others, the number infected by the people i infects is

$$D_i^{(2)} = \sum_{j \in N(i)} D_j,$$

a sum whose number of terms, D_i [COMMENT: should this be $N(i)$???], is itself random. The two rules handle sums like this. Flip a coin: if it lands heads, roll two dice and add them;

if tails, roll one. Let $C \in \{1, 2\}$ be the number of dice, with $G_C(z) = \frac{1}{2}z + \frac{1}{2}z^2$. If $C = 1$ the total has generating function $G_Y(z)$, and if $C = 2$ it has $G_Y(z)^2$, so by Proposition 4.3.4

$$G_X(z) = \Pr[C = 1]\, G_Y(z) + \Pr[C = 2]\, G_Y(z)^2 = G_C\big(G_Y(z)\big).$$

Notice that this expression looks a lot like a generating function for $G_C(z)$, but that $G_Y(z)$ has taken the place of z. Notice that $G_Y(z)$ has been squared, because the $C = 2$ case corresponds to us actually summing two independent copies of Y, requiring their convolution. Now, lets generalize this idea!

Proposition 4.3.5 (Random sums)**.** *Let $N, Y_1, Y_2, \ldots$ be independent counts, with every Y_i distributed like Y, and let $X = Y_1 + \cdots + Y_N$. Then*

$$G_X(z) = G_N\big(G_Y(z)\big).$$

Proof. When $N = n$, the sum has n terms and generating function $G_Y(z)^n$, by Proposition 4.3.3. The cases $N = n$ are disjoint, so by Proposition 4.3.4, $\mathbb{E}\big[z^X\big] = \sum_n \Pr[N = n]\, G_Y(z)^n = G_N\big(G_Y(z)\big)$. □

Composition does two things at once: the power $G_Y(z)^n$ adds up n copies, and plugging into G_N weights each length n by its probability.

4.3.5 Branching Processes

Definition 4.3.6 (Branching process)**.** Start with one individual. Its number of children is distributed like X, and so is the number of children of every descendant, independently. The family *dies out* if some generation is empty.

Figure 4.15 shows one instantiation of a branching process. These processes model anything that copies itself: populations in ecology, DNA in the polymerase chain reaction, where every molecule is copied with some probability in each cycle, and an epidemic, which is the exploration of a (potentially giant) component of a random graph.

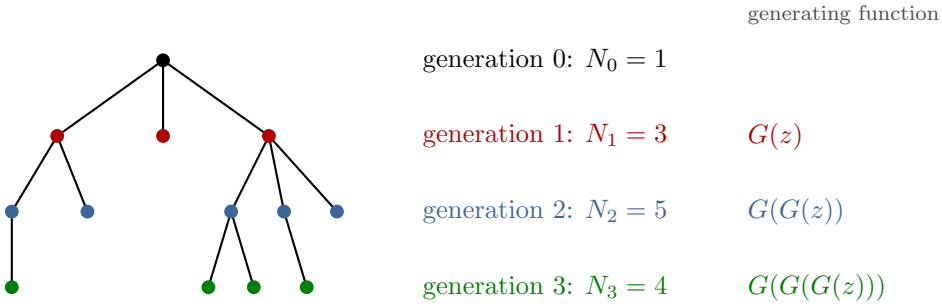


Figure 4.15: A branching process. Each generation is a random sum, $N_t = X_1 + \cdots + X_{N_{t-1}}$, one count of children for each of the N_{t-1} members of the generation before, so its generating function is one more composition of G.

Theorem 4.3.7 (Extinction)**.** *Let G be the generating function of X, and q the probability that the family dies out. Then q is the smallest solution of $q = G(q)$ in $[0, 1]$. If $\mathbb{E}[X] \le 1$, and X is not always exactly 1, then $q = 1$; if $\mathbb{E}[X] > 1$, then $q < 1$.*

Proof. Let q_t be the probability that the family has died out by generation t, so $q_0 = 0$. The family is gone by generation t when every child's line is gone within $t-1$ more generations. Each line is a copy of the whole process, so with k children this has probability q_{t-1}^k, and averaging over the probability of children k gives a polynomial exactly equal to the generating function. This means $q_t = G(q_{t-1})$, with the idea illustrated in Figure 4.16(a).

We can therefore conclude that q_t climbs from 0 and settles at a fixed point $q = G(q)$. Since G is increasing, it cannot climb past one: it stops at the smallest fixed point, as in Figure 4.16(b). Finally, G passes through $(1, 1)$ with slope $G'(1) = \mathbb{E}[X]$, and it is convex, since its coefficients are nonnegative, so there is a fixed point below 1 exactly when the slope exceeds 1, as in Figure 4.16(c) and (d). □

The idea of fixed points, such as $q = G(q)$, will return in Chapter 6, where a stationary distribution is a fixed point of a Markov chain.

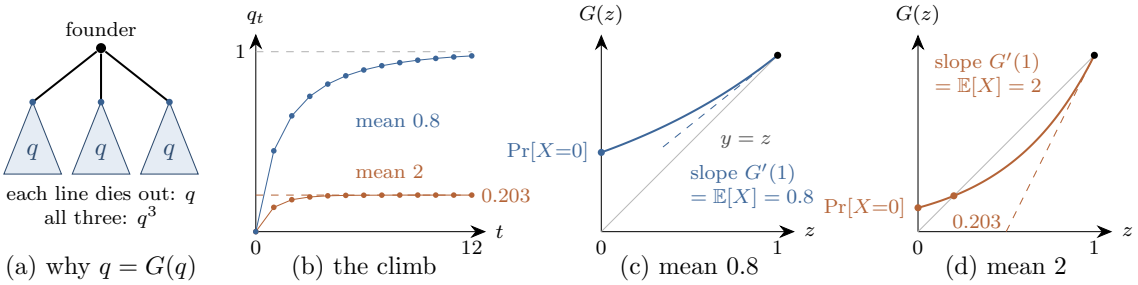


Figure 4.16: The extinction theorem in pictures. (a) Why $q = G(q)$: the family dies out when every child's line does. Each line is a copy of the whole process and dies out with probability q, so with k children all of them die out with probability q^k, and averaging over k gives $G(q)$. (b) Why q is the smallest solution: the probability q_t of dying out by generation t starts at 0 and climbs accordin to $q_t = G(q_{t-1})$. With mean 2 it levels off at 0.203, the smaller solution of $q = G(q)$, and never reaches the other solution, 1; with mean 0.8 it climbs toward 1. (c), (d) These plots show why the mean $\mathbb{E}[X]$ drives everything: each curve is $G(z)$ and the gray line is $y = z$. Each curve starts at $G(0) = \Pr[X = 0]$, on or above the diagonal. Every generating function passes through $(1, 1)$, with slope equal to the mean, shown by the dashed tangent. With slope below 1 the curve stays above the diagonal, so 1 is the only solution; with slope above 1 it dips below the diagonal and must cross it again, in this case crossing at 0.203.

4.3.6 When the Giant Component Emerges

Corollary 4.3.8 (The giant component)**.** *In $G(n, p)$ with average degree $d = (n-1)p$ held fixed as n grows, a giant component holding a positive fraction of the vertices appears exactly when $d > 1$, and it holds a fraction S of them, where*

$$S = 1 - e^{-dS}.$$

Sketch. Explore the component of a vertex outward. While the explored part is small, the graph near the vertex looks like a tree, and each newly reached vertex has Binomial$(n-1, p)$ new neighbors, so the exploration is a branching process. Each possible neighbor is a Bernoulli(p) count, so by Proposition 4.3.3 the number of children has generating function $(1-p+pz)^{n-1}$, which tends to $e^{d(z-1)}$ as n grows, with mean $G'(1) = d$. By Theorem 4.3.7, the exploration survives with positive probability exactly when $d > 1$. A vertex is in the

giant component exactly when its exploration survives, so $S = 1 - q$ with $q = e^{d(q-1)}$, which is $S = 1 - e^{-dS}$. □

At $d = 2$, $S \approx 0.797$: the survival probability of Figure 4.16. Figure 4.17 plots S against d. Together with Section 4.2, this completes the picture of Figure 4.12: below $d = 1$ only small pieces, above it one giant component, and above $d = \log n$ everything connected. The same argument gives the epidemic threshold: in an outbreak, d is the reproduction number R_0, the average number of people each case infects, so an epidemic can take off exactly when $R_0 > 1$. Problem Set 4 works out what changes when some people have far more contacts than others.

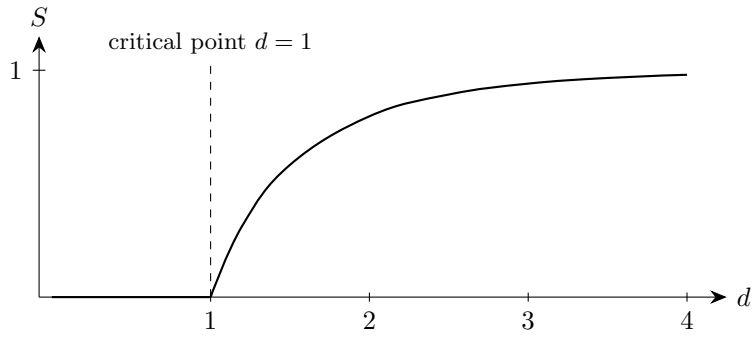


Figure 4.17: The fraction S of vertices in the largest component of $G(n, p)$, plotted against the expected degree d. The curve is the solution of $S = 1 - e^{-dS}$, from Corollary 4.3.8, and the corner at $d = 1$ shows a sudden phase transition.

Chapter 5

Regular Languages and Automata

This chapter is about machines with finite memory and what what they can recognize (and what they cannot). It is the one unit of the course with no randomness, but it will set us up nicely with the concept of a system with an evolving state, which will become a Markov chain once we add randomness back in. By the end of the chapter, we will answer the question from Chapter 0 about why a virus can always defeat a filter with limited memory.

5.1 Finite-State Machines

You actually saw a machine like an automata in Section 3.2. The decoder for a prefix-free code reads one bit at a time, keeps track of where it is in the code tree, and needs to remember nothing else. We will now extend this to include loops and ask what a machine can do if all it ever remembers is which state it is in and if we only have finitely many of those states?

5.1.1 A Machine You Have Already Built

Recall the decision tree for the code $A \to 0$, $B \to 10$, $C \to 110$, $D \to 111$ from Section 3.2, shown again in Figure 5.1(a). Decoding a long message walks down the tree one bit at a time and jumps back to the top after every letter.

This is very redundant: each leaf sends the decoder back to the root, acting exactly the same way. Merging each leaf into the root leaves only the internal nodes, which record how much of the current codeword has been read: nothing yet, a 1, or 11. We call each of these nodes and the merged root "states," since they interact with the input to determine the subsequent behavior of the machine. The new state machine is depicted in Figure 5.1(b).

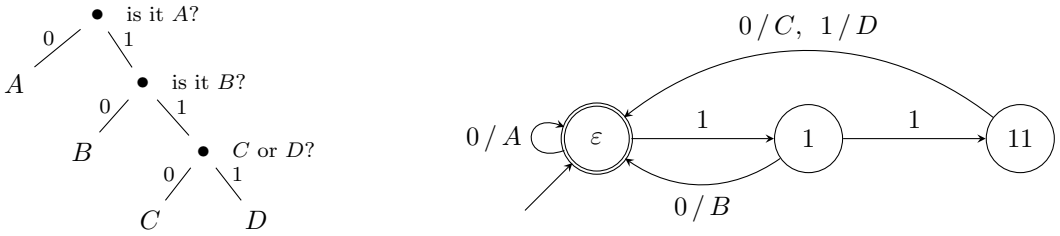
5.1.2 Directed Graphs

The state diagram of Figure 5.1 is a directed graph: its arrows run one way, from tail to head.

Definition 5.1.1 (Directed graph)**.** In a *directed graph* each edge is an ordered pair (u, v), drawn as an arrow from u to v. The *out-degree* of a vertex counts arrows leaving it and the *in-degree* counts arrows arriving.

5.1.3 Strings and Languages

Unlike the decoder's tree, an automaton can have cycles, so it can read strings of any length and detect patterns that are arbitrarily long.



(a) the decision tree

(b) compressed into states

Figure 5.1: (a) The decision tree for the code $A = 0$, $B = 10$, $C = 110$, $D = 111$ from Section 3.2. (b) The same decoder as a machine. Its states are the internal nodes of the tree, labeled by the bits read so far of the current codeword. Each bit moves it along one arrow, and an arrow marked $0\,/\,A$ also emits the letter A and returns to the start, where the leaf A used to send the decoder back to the root.

Definition 5.1.2 (Alphabet, string, language)**.** An *alphabet* Σ is a finite set of symbols. A *string* over Σ is a finite sequence of symbols from it, its *length* $|w|$ is the number of symbols, and the *empty string* of length zero is written ε. The set of all strings over Σ is Σ^*. A *language* is any subset $L \subseteq \Sigma^*$.

5.1.4 Deterministic Finite Automata

Definition 5.1.3 (DFA)**.** A *deterministic finite automaton* is a tuple $M = (Q, \Sigma, \delta, q_0, F)$ where Q is a finite set of *states*, Σ an alphabet, $\delta : Q \times \Sigma \to Q$ the *transition function*, $q_0 \in Q$ the start state, and $F \subseteq Q$ the *accepting* states.

The machine starts in q_0, reads the string one symbol at a time, and follows the arrow δ gives for each symbol. It *accepts* the string if it finishes in a state of F, and the *language of* M is the set of strings it accepts. The key word here is *finite*: between symbols, all the machine knows about what it has read so far is which of its finite $|Q|$ states it occupies.

A DFA has an arrow for every state and symbol. To keep things uncluttered, we sometimes leave some arrows out, and the convention is that every missing arrow goes to a *dead state*. Dead states do not accept and they have self loops for all symbos so they never find themselves back on a path to acceptance.

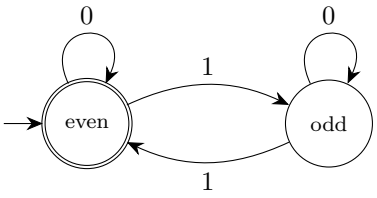


Figure 5.2: A DFA accepting the binary strings with an even number of 1s. The double circle marks the accepting state and the incoming arrow from nowhere marks the start. Reading a 0 changes nothing, and reading a 1 flips the parity. Two states suffice because parity is all the machine needs to remember, however long the string.

If 0110 is processed in the machine of Figure 5.2, we end in the accepting state and accept; if 111 is inputted, we end outside F and reject.

5.1.5 Simple Computations

DFAs are surprisingly powerful. To decide whether a binary number is divisible by 3, we can keep track of its remainder. Appending a bit b doubles the number and adds b: from 101, which is 5, appending 0 gives 1010, which is 10, and appending 1 gives 10101, which is 21. So a bit b takes the remainder from r to $2r + b \bmod 3$, and a machine with three states, one for each remainder, decides divisibility by 3 for numbers of any length (Figure 5.3). Reading 10101, it moves $0 \to 1 \to 2 \to 2 \to 1 \to 0$ and accepts.

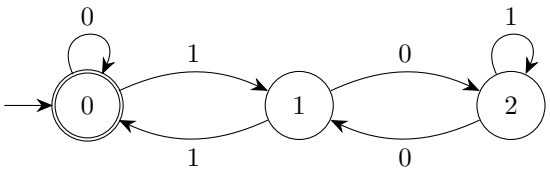


Figure 5.3: A DFA for the binary numbers divisible by 3, read from the most significant bit. The state is the remainder of the number read so far, and reading a bit b moves from r to $2r + b \bmod 3$.

5.1.6 Waiting for a Pattern

A machine that watches for a fixed pattern records how much of it has matched so far, as in Figure 5.4. Detectors like these find genes and other sequences in a genome: the restriction enzyme EcoRI cuts DNA wherever it reads $GAATTC$, so finding its cut sites means running a six-letter pattern detector over the alphabet $\{A, C, G, T\}$.

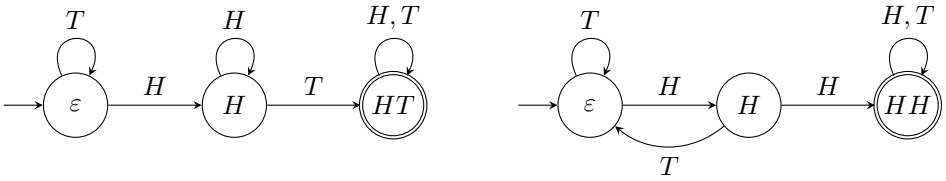


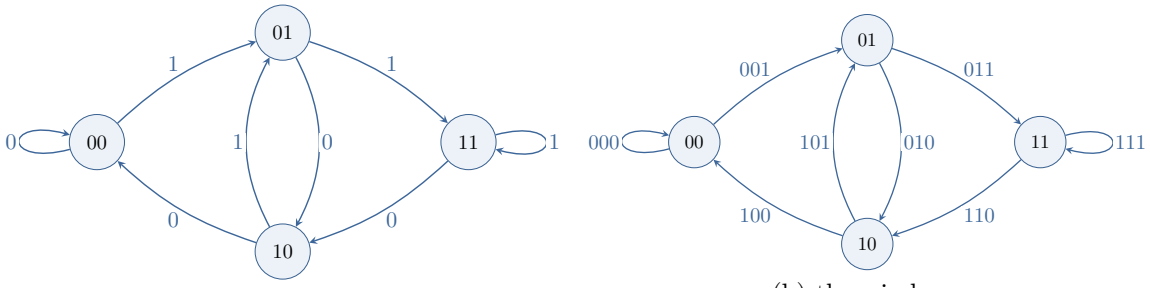
Figure 5.4: Two pattern detectors over the alphabet $\{H, T\}$. Each state records how much of the pattern has matched so far, and the accepting state keeps the machine there, since once the pattern has appeared the string always accepts. The machines differ in a single arrow. Having seen H, the left-hand machine stays put on another H, because that H starts a fresh attempt; the right-hand machine, on a T, has nothing left to build on and reverts back to the start.

5.1.7 De Bruijn Graphs

DFAs are an abstract description of *sequential circuits*, circuits with memory: the state is held in flip-flops, and ordinary logic computes the next state from the current state and the input, once per tick of the clock.

Garage door key systems wait for a k-bit code, they are often implemented using hardware version sof DFAs to listen for the correct code. After every bit the receiver checks the last k bits against the code, so it needs to remember the last $k - 1$.

Definition 5.1.4 (De Bruijn graph)**.** For $k \geq 2$, the binary *de Bruijn graph* of order k has a vertex for each binary string of length $k-1$, and for each binary string of length k an arrow from its first $k-1$ bits to its last $k-1$ bits.



(a) the automaton (b) the windows

Figure 5.5: The binary de Bruijn graph of order 3. (a) As the receiver's automaton: each arrow is labeled with the bit it reads. (b) With each arrow labeled by the three-bit window it completes, its starting state followed by the new bit. Each vertex is a two-bit string and each arrow a three-bit string, running from its first two bits to its last two, and every vertex has two arrows in and two out. An Eularian trail is given by following $000, 001, 011, 111, 110, 101, 010, 100$, which uses every arrow once and returns to 00. Writing down the starting vertex 00 and then the last bit of each arrow gives the de Bruijn sequence, 0001110100, in which every three-bit string appears exactly once as a window.

To get into a garage, we need to reach the accepting arrow efficiently. Because the windows overlap, the string 10100 tests three three-bit codes, 101, 010 and 100, in five bits rather than nine. Each arrow represents one substring of length k, so a string containing every k-bit code is a trail using every arrow exactly once.

Theorem 5.1.5 (Euler, for directed graphs)**.** *A directed multigraph that is connected, ignoring the directions of its arrows, has a trail using every arrow exactly once if and only if every vertex has equal in-degree and out-degree, except possibly for one vertex with one more arrow out than in, where the trail must start, and one with one more arrow in than out, where it must end. If all vertices have the same in- and out-degree then the treck finishes where it starts, i.e., it is closed).*

Every vertex of the de Bruijn graph has in-degree and out-degree 2, so it has a closed (same start and end) trail through every arrow. Its 2^{k-1} vertices are each visited twice, so writing down the first $k-1$ bits and then one bit per arrow gives a *de Bruijn sequence* of length $2^k + k - 1$ containing every k-bit code. This is approximately k times as efficient as the $k\,2^k$ bits needed to send every code separately. For $k = 3$ the de Bruijn sequence is 0001110100.

In 2015 the security researcher Samy Kamkar used this speedup to show that many garage doors with codes of 8 to 12 bits could be opened in about eight seconds, whereas trying every code separately takes about three minutes. One fix for this is to use a rolling code for digital remotes, which changes after every use. Another fix it o have a start or end key on keypads, which stop the windows from overlapping.

In 1951 Tatyana van Aardenne-Ehrenfest and Nicolaas de Bruijn generalized these graphs to larger alphabets, and in 2001 Pavel Pevzner, Haixu Tang and Michael Waterman realized that the generalization was incredibly usefull for genome sequencing. If you chop many copies of a genome into short reads, you can stitch them together by looking at a de Bruijn graph that processes their overlap as edges. Re-assembling the sequence boils

down to finding an Eulerian trail, which can be done quickly even with millions of vertices. Figure 5.6 demonstrates this on a tiny example.

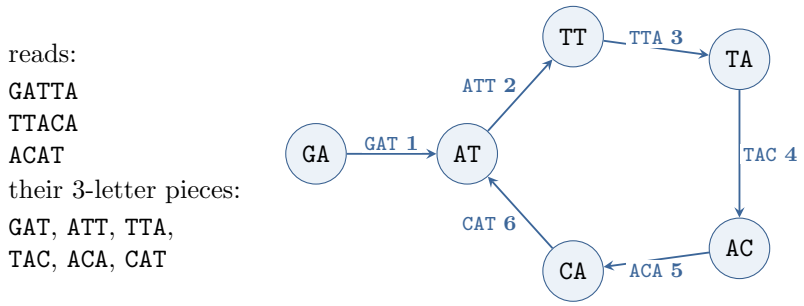


Figure 5.6: Assembling a genome from reads. Each read is cut into its overlapping 3-letter pieces, and each piece becomes an arrow from its first two letters to its last two. The genome is a trail through every arrow. Here there is only one, numbered in order, and reading it off gives GATTACAT. The trail passes through AT twice because AT occurs twice in the genome.

5.2 Regular Expressions

DFAs capture patterns, but they are hard to type into a computer. Regular expressions are a notation for patterns that can be typed, and this lecture shows that they are really just a different way of expressing DFAs.

5.2.1 Languages from Languages

Two processes are helpful to define, since regular expressions build languages from smaller language components.

Definition 5.2.1 (Concatenation and star)**.** The *concatenation* of languages is $L_1L_2 = \{\, uv : u \in L_1,\ v \in L_2 \,\}$, formed by gluing a string from the first onto a string from the second. The *Kleene star* is

$$L^* = \{\varepsilon\} \cup L \cup LL \cup LLL \cup \cdots,$$

all strings obtained by concatenating zero or more pieces from L.

For example,

$$\{\texttt{cat}, \texttt{dog}\}\{\varepsilon, \texttt{s}\} = \{\texttt{cat}, \texttt{cats}, \texttt{dog}, \texttt{dogs}\}, \qquad \{\texttt{ha}\}^* = \{\varepsilon, \texttt{ha}, \texttt{haha}, \texttt{hahaha}, \ldots\},$$

and $\{A, C, G, T\}^*$ is every possible DNA sequence.

5.2.2 Regular Expressions

Definition 5.2.2 (Regular expression)**.** A *regular expression* over Σ is built from the following, and nothing else:

- $\varnothing$, denoting the empty language, and ε, denoting $\{\varepsilon\}$;
- a single symbol $a \in \Sigma$, denoting $\{a\}$;

- $R_1 \mid R_2$, denoting the union of the two languages;
- $R_1 R_2$, denoting their concatenation;
- R^*, denoting the Kleene star.

There is an "order of operations": star binds tightest, then concatenation, then union, so $ab^* \mid c$ means $(a(b^*)) \mid c$, not $(ab)^* \mid c$.

Example 5.2.3. Over $\Sigma = \{0, 1\}$:

Expression	Language
$(0 \mid 1)^*$	every binary string
$(0 \mid 1)^*0$	strings ending in 0
$0^*10^*10^*$	strings containing exactly two 1s
$(0 \mid 1)^*101(0 \mid 1)^*$	strings containing 101 somewhere
$(0 \mid 11)^*$	strings in which every block of 1s has even length

Example 5.2.4 (Email addresses)**.** A Dartmouth undergraduate address looks like

`theodor.s.geisel.25@dartmouth.edu`

Writing ℓ for $(a \mid b \mid \cdots \mid z)$ and d for $(0 \mid 1 \mid \cdots \mid 9)$, these addresses are

$$\ell\ell^* \texttt{ . } \ell \texttt{ . } \ell\ell^* \texttt{ . } dd \texttt{@dartmouth.edu},$$

where the dots and the domain are literal symbols. Practical tools abbreviate this as

`[a-z]+\.[a-z]\.[a-z]+\.[0-9]{2}@dartmouth\.edu`

a range like `[a-z]` is a union, `+` means one or more, and `{2}` means two copies. A rough check for any address, $\ell\ell^*$ `@` $\ell\ell^*$ `.` `(com | edu)`, is just as short.

Two expressions can describe the same language: an even number of 1s is $0^*(10^*10^*)^*$, and also $(0 \mid 10^*1)^*$. In general, regular expressions are useful for find-and-replace, where `colo(u | ε)r` matches both spellings, and for finding genes: the enzyme HinfI cuts DNA at $GANTC$, where N is any base, which is the regular expression $GA(A \mid C \mid G \mid T)TC$.

5.2.3 From Regular Expressions to NFAs

In 1956 Kleene showed that regular expressions can all be converted to DFAs, and in 1968 Ken Thompson gave an algorithm for doing so that led to the Unix tool `grep`. This made searching for regular expressions *very* efficient, which is why they are so frequently used for pattern detection.

Theorem 5.2.5 (Kleene)**.** *A language is denoted by some regular expression if and only if it is the language of some finite automaton.*

We prove that every regular expression has a machine by induction, combining the machines for smaller parts. Every machine we build has one start state and one accepting state, which is what we use to join them starting from the base cases: machines for a single symbol, for ε, and for $\varnothing$ (Figure 5.8, top). First, we will introduce nondeterminism which will help make the combination process cleaner.

Definition 5.2.6 (NFA)**.** A *nondeterministic finite automaton* replaces the transition function by $\delta : Q \times \Sigma \to \mathcal{P}(Q)$, so a state and symbol may lead to several states or to none, and may also allow ε-transitions taken without consuming a symbol. An NFA *accepts* a string if *some* path through it ends in an accepting state.

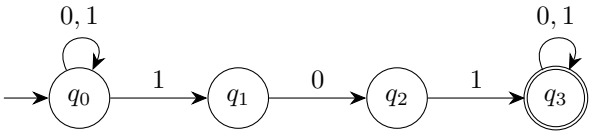


Figure 5.7: An NFA for strings containing 101. It reads arbitrary symbols, guesses when the pattern begins, checks 1 then 0 then 1, and then reads whatever follows. Writing a DFA for the same language means building a pattern detector like those of Figure 5.4, which takes more thought and more states.

An NFA may move to many states or to none, and an ε-transition joins two states, so being in the first means also being in the second. A path with no arrow to follow simply dies, so an NFA needs no dead state. Every DFA is an NFA, since NFAs only *add* capabilities.

Now suppose we have machines for R_1 and R_2, the inductive hypothesis (Figure 5.8):

- for $R_1 \mid R_2$, add a new start state with ε-arrows into both machines, and ε-arrows from their accepting states to a new accepting state;
- for R_1R_2, join each accepting state of the first machine to the start of the second by an ε-arrow;
- for R^*, add new start and accepting states, ε-arrows into and out of the machine, one from its accepting states back to its start, so that it can repeat, and one from the new start straight to the new accepting state, so that it accepts ε.

By induction on the structure of the expression, every regular expression has an NFA.

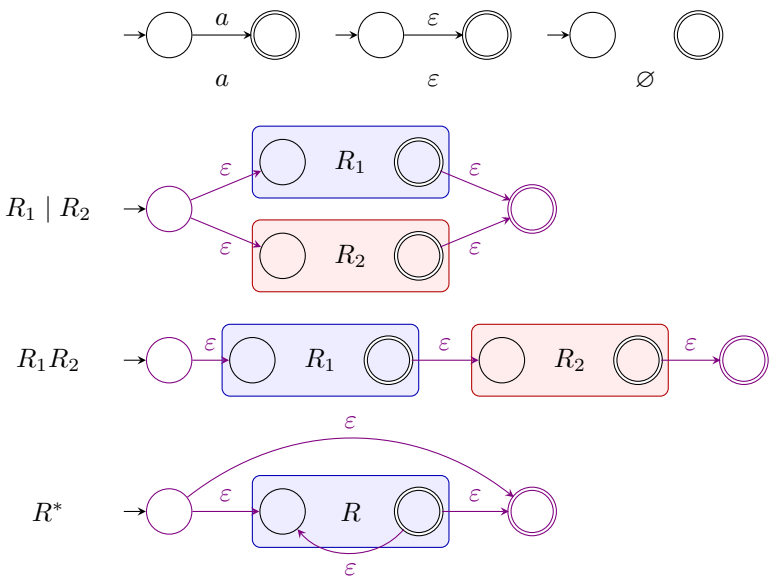


Figure 5.8: Building an NFA from a regular expression. Top: the base cases, a single symbol a, the empty string ε, and the empty language $\varnothing$, whose machine has no path to its accepting state. Below: the boxes are machines built for smaller expressions, blue for R_1 and red for R_2, shown by their start and accepting states, and the new states and ε-arrows, in purple, join them into machines for $R_1 \mid R_2$, for R_1R_2, and for R^*, where the arrow from the new start straight to the new accepting state accepts zero copies.

Going the other way, from a DFA to a regular expression, is also possible, by deleting the machine's states one at a time and recording the paths through each as expressions on

the arrows. Due to time constraints, we will not prove this. Still, the conclusion is that the three descriptions, regular expression, NFA and DFA, are interchangeable.

5.3 The Limits of Finite Memory

This lecture finishes the equivalence of the last lecture, then studies what finite is incapable of destinguishing.

5.3.1 From NFAs to DFAs

In the last lecture, we showed how to construct an NFA from a regular expression. To finish the equivalence, we need to turn NFAs into DFAs. The key idea is this: While an NFA can be in several states at once, the set of states it is in is itself a single state of knowledge. Since there are only finitely many such sets, we can make each set of states into a new state.

Theorem 5.3.1 (Subset construction)**.** *For every NFA there is a DFA accepting the same language.*

Sketch. The key difference between NFAs and DFAs is that, after reading a prefix into an NFA, we do not know which state it is in. Instead, we will track the *set* of states it could be in. We build a DFA whose states are the elements of $\mathcal{P}(Q)$, with start state $\{q_0\}$ closed under ε-transitions[1], transition function

$$\delta'(S, a) = \bigcup_{q \in S} \delta(q, a),$$

again closed under ε-transitions. The accepting states are subsets containing at least one accepting state of the NFA. Reading a string in the new machine simulates all paths of the old one at once, and it finishes in an accepting subset exactly when some path accepted. □

If every path has died, the set of states the NFA could be in is $\varnothing$: the empty set is the dead state of the new machine.

Running the construction on the NFA of Figure 5.7, and keeping only the subsets reachable from $\{q_0\}$, gives Figure 5.9: only 6 of the 16 subsets are reachable. An NFA with n states becomes a DFA with up to 2^n, the size of the power set. The blowup is usually far smaller in practice, because most subsets are unreachable, but there are special cases where it can occur.

DFAs, NFAs and regular expressions therefore describe exactly the same languages. These languages have a name!

Definition 5.3.2 (Regular language)**.** A language is *regular* if a DFA, an NFA or a regular expression describes it.

5.3.2 Simplifying DFAs

The power set construction built six states for the strings containing 101, but the three that contain q_3 behave identically: from each of them, every continuation is accepted. States like these can be merged.

Definition 5.3.3 (Equivalent states)**.** Two states of a DFA are *equivalent* if, for every string w, reading w from one ends in an accepting state exactly when reading w from the other does.

[1] "Closing under ε" means following all ε edges and adding those states to the set of "current" states.

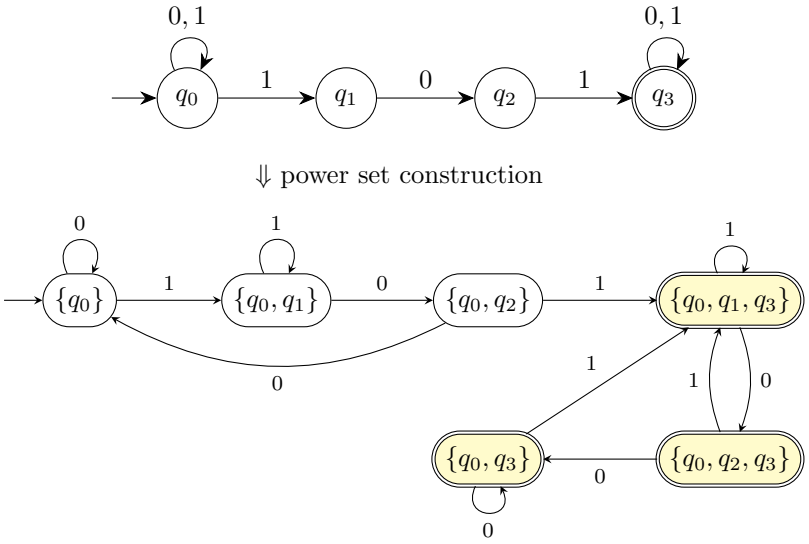


Figure 5.9: The power set construction. Top: the NFA of Figure 5.7 for strings containing 101. Bottom: the DFA it builds, keeping only the six subsets reachable from $\{q_0\}$. Each state is a set of NFA states, and the three accepting sets, shaded, are those containing q_3.

Merging equivalent states never changes the language. To find them, start with two groups, the accepting and the non-accepting states. If two states in the same group move to different groups on some symbol, they are not equivalent, so split the group. If you repeat until no group splits, then the groups that remain are the states of the simplified machine.[2]

Example 5.3.4 (Remembering the last two bits)**.** To recognize the strings whose second-to-last bit is 1, a natural first design remembers the last two bits read: a start state s, the one-bit histories 0 and 1, and the two-bit histories 00, 01, 10 and 11, with 10 and 11 accepting (Figure 5.10a). Reading a bit appends it and forgets the oldest. Start with the groups $\{10, 11\}$ and $\{s, 0, 1, 00, 01\}$.

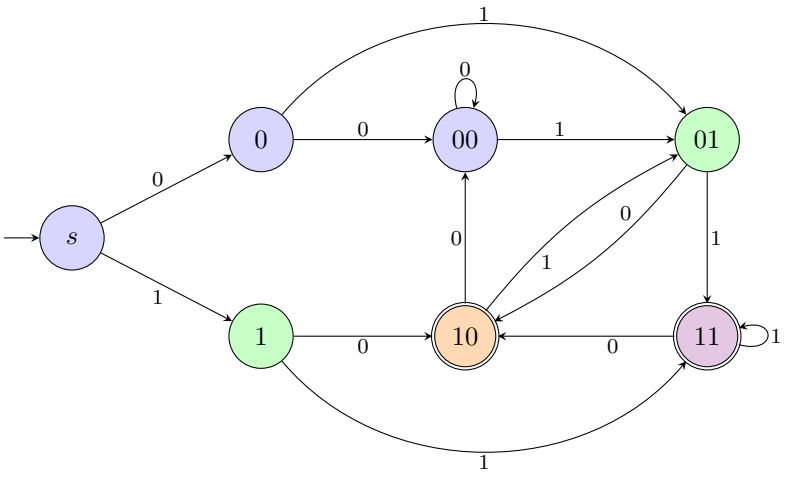
On a 0, 10 moves to 00, outside its group, while 11 moves to 10, inside it, so 10 and 11 split: both accept, but they are not equivalent. Also on a 0, the states 1 and 01 move to the accepting state 10, while s, 0 and 00 move to 0 or 00, so the other group splits into $\{1, 01\}$ and $\{s, 0, 00\}$. Checking the four groups again, every state in a group moves to the same group on each bit, so nothing else splits.

The result has four states (Figure 5.10b) and is interpretable: the machine remembers the last two bits, with missing bits counted as 0s. The start state behaves like 00, and the history 1 like 01, which is why they merge. An NFA for this language needs only three states, guessing that the current bit is second-to-last and checking that exactly one more follows, but no DFA has fewer than four. For the language that requires a 1 in the kth bit from the end, the gap is $k + 1$ NFA states against 2^k for a DFA. These "kth bit" langugages therefore make up a family where the blowup in the number of states from the power set construction occurs.

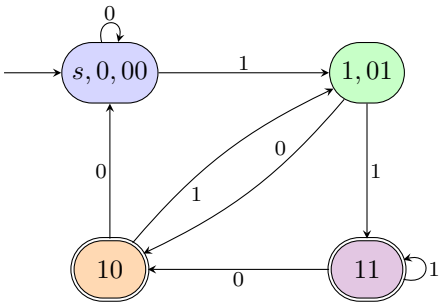
Definition 5.3.5 (Distinguishable)**.** Two states that are not equivalent are *distinguishable*: some string w, read from one, ends in an accepting state, and read from the other does not.

Strings can be distinguishable too: u and v are distinguishable if some continuation w makes exactly one of uw and vw accepted, and then a DFA must be in distinguishable

[2] A theorem of Myhill and Nerode says that merging equivalent states is the only simplifaction we can do and gives the smallest possible DFA.



(a) remembering the last two bits: seven states, colored by group



(b) merged: four states

Figure 5.10: Simplifying a DFA for the strings whose second-to-last bit is 1. (a) A first design remembers the last two bits read, with s the start and 10 and 11 accepting. States of the same color turn out to be equivalent. (b) Merging them leaves four states, which remember the last two bits with missing bits counted as 0s.

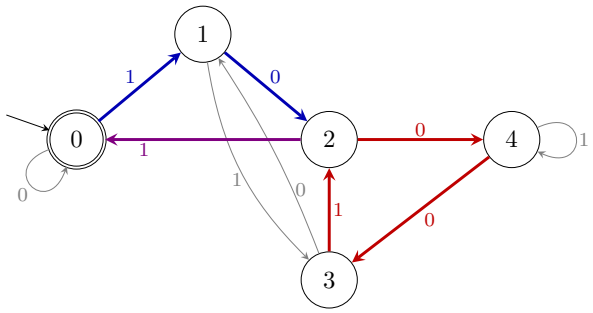
states after reading them. So if a language has n strings that are pairwise distinguishable, every DFA for it needs at least n states, and if it has infinitely many, no DFA recognizes it at all. For the correctly matched brackets $\{ \langle^n \rangle^n : n \geq 0 \}$, the strings $\langle^0, \langle^1, \langle^2, \ldots$ are pairwise distinguishable, since only $\rangle^i$ completes $\langle^i$. This set is infinite, so recognizing it would take infinite memory.

5.3.3 The Pumping Lemma

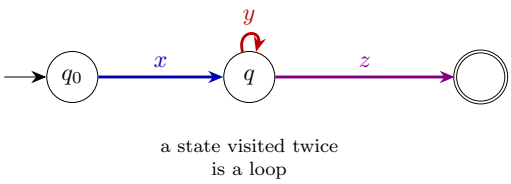
Antivirus software and network intrusion detectors look for known attacks by scanning files and traffic for their *signatures*, i.e., patterns written as regular expressions. A scanner with a fixed, finite memory is a DFA: it turns the signatures into an NFA and then into a DFA, as above, which can scan as fast as data arrives. Attackers have avoided these scanners by hiding the pattern. A scanner must look inside containers, such as archives inside archives or comments inside comments, and every scanner has a DFA-based limit on how deep it will look, since each layer takes up memory. The rest of this lecture proves that, for nesting like comments inside comments, the limit cannot be avoided: there is no perfect virus filter that uses a finite amount of memory.

To understand the limits of finite memory, run a DFA with p states on a string of length at least p. The machine passes through $p + 1$ states, so some state repeats, and between the two visits the machine has gone around a loop (more precisely, a closed walk). That loop can be skipped or repeated, and the machine cannot tell the difference once its back to where it started: it accepts all of the resulting strings, or rejects all of them.

Example 5.3.6 (Pumping a string)**.** The DFA of Figure 5.11(a) reads a binary number and tracks its remainder mod 5: reading the bit b takes remainder r to $2r + b$ mod 5. It has $p = 5$ states. Reading $w = 100011$, which is 35, takes it through $0, 1, 2, 4, 3, 2, 0$, so state 2 repeats after two symbols and after five. Splitting w at the repeat gives a prefix $x = 10$, a loop $y = 001$ and a suffix $z = 1$. Skipping the loop gives 101, which is 5, and repeating it gives 100010011, which is 275: both multiples of 5, and both accepted. The pumping lemma says this always happens.



(a) the walk on $w = 100011$



(b) the same split, in general

Figure 5.11: Why the pumping lemma is true. (a) A DFA that reads a binary number and accepts the multiples of 5; its states are the remainders 0 to 4. Reading $w = 100011$, which is 35, the prefix $x = 10$ reaches state 2, the loop $y = 001$ visits 4 and 3 and returns to 2, and the suffix $z = 1$ finishes in the accepting state 0. The machine's other transitions are gray. (b) In general, a string long enough to force a repeated state splits into the part before the repeat, the loop between the two visits, and the part after. The machine returns to the same state however many times the loop is taken, so it cannot distinguish xz from xyz from $xyyz$.

Theorem 5.3.7 (Pumping lemma)**.** *Let L be regular. Then there is a number $p \geq 1$, the pumping length, such that every $w \in L$ with $|w| \geq p$ can be written $w = xyz$ with*

$$|xy| \leq p, \qquad |y| \geq 1, \qquad xy^i z \in L \ \text{ for every } i \geq 0.$$

Proof. Let M be a DFA for L and let $p = |Q|$ be its number of states. Take any $w \in L$ with $|w| \geq p$ and run M on it. Reading the first p symbols, the machine occupies $p + 1$ states, counting the one it starts in. There are only p states available, so by the pigeonhole principle of Section 1.3, two of those must be the same state.

Say the machine is in state q after reading the first j symbols and in q again after the first ℓ symbols, with $j < \ell \leq p$. Split w accordingly: x is the first j symbols, y the next $\ell - j$, and z the rest. Then $|xy| = \ell \leq p$ and $|y| \geq 1$.

Reading y from state q returns the machine to state q, so it is a loop. In the language of Chapter 4, the run is a walk through the state diagram, and a walk with more steps than the diagram has vertices must repeat a vertex. The machine cannot tell how many times the loop was traversed: after x, after xy, after xyy, and so on, it is in state q every time, which is all the machine remembers. Since $xyz \in L$, also $xy^i z \in L$ for every $i \geq 0$. □

> *Remark* 5.3.8 (What would be needed). Matched nesting needs unbounded memory of a particular shape: a *stack*. A finite automaton with a stack, which pushes a symbol on each opening bracket and pops one on each closing bracket, is a *pushdown automaton*. The languages such machines recognize are the *context-free* languages, the ones described by *context-free grammars*. Compilers for programming languages use these languages to check and parse syntax.

5.3.4 A Language That Is Not Regular

Theorem 5.3.9. *The language $L = \{ \langle^n \rangle^n : n \geq 0 \}$ of correctly nested strings is not regular.*

Proof. Suppose it were, and let p be a pumping length. Choose

$$w = \langle^p \rangle^p,$$

which is in L and has length $2p \geq p$. Any split with $|xy| \leq p$ puts x and y inside the opening brackets, so $y = \langle^k$ for some $k \geq 1$. Then

$$xy^2 z = \langle^{p+k} \rangle^p,$$

which has more opening than closing brackets and so is not in L. This contradicts the lemma, so L is not regular. □

A DFA *can* recognize correctly nested strings if the depth of nesting is bounded by k: states $0, 1, \ldots, k$ record the current depth, $\langle$ moves up and $\rangle$ moves down, and anything that would leave the range goes to a dead state. This uses a total of $k + 2$ states. Hence, a finite machine can count, but only up to a bound fixed when the machine is built.

5.3.5 Why the Filter Fails

We can now answer the question from Chapter 0 precisely. If an attack were a fixed string, a filter would not need to understand nesting at all: it could reject every message containing the string, which is the regular language $\Sigma^* \, \texttt{attack} \, \Sigma^*$. The problem is that the same string can be harmless or dangerous depending on where it sits.

In Rust, as in Haskell and OCaml, block comments nest: `/*` opens a comment, `*/` closes the most recent one, and anything inside a comment is ignored. The following two messages differ only in the position of one `*/`:

`/* /* note */ send_password(); */`	inside a comment: harmless
`/* /* note */ */ send_password();`	both closed: runs

Code is often commented out in a debugging process, and the location of `send_password()` can determine whether it is a normal login or a password leak. A filter that rejects every message containing `send_password();` blocks or flags both programs, even though only one of them is sending the password. To block exactly the dangerous programs, a filter must decide whether `send_password();` appears at depth 0, outside every comment.

No filter with finite memory can do this. A filter with $p = 3$ states cannot tell these two messages apart:

`/* /* /* */ */ */ send_password();`	three open, three closed: runs
`/* /* /* /* */ */ */ send_password();`	four open, three closed: harmless

So every filter with finite memory either blocks code with a harmless comment or lets an attack through, and the attacker can choose a depth, $p + 1$ openings, after the filter has been built.

Chapter 6

Markov Chains

A Markov chain takes the state graphs of Chapter 4 and follows a walk at random. Stochasticly changing of states can be used to model many things: the weather, a queue, a diffusing gas, a web surfer clicking links, a molecule folding. A central question in Markov chains is their long term behavior, which is what this chapter is primarily about.

6.1 Adjacency Matrices

Questions about matrix connectivity are easy to answer by eye on a small drawn graph, but are very difficult to answer using computers. Computer scientists have developed a number of serach algorithms for connectivity, such as breadth-first and depth-first search. We take a different route that is more closely related to our eventual topic of Markov chains: arithmetic on a matrix.

6.1.1 The Adjacency Matrix

To turn the questions into arithmetic, we use the observation from Section 1.1: an indicator function represents set membership wit 0 and 1, which lets us turn logical structure into algebraic structure.

Definition 6.1.1 (Adjacency matrix)**.** Label the vertices $v_1, \ldots, v_n$. The *adjacency matrix* A is the $n \times n$ matrix whose entries are the indicators of the possible edges,

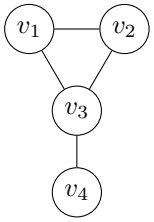
$$A_{ij} = \mathbb{1}_{\{(v_i,v_j)\in E\}} = \begin{cases} 1 & \text{if } v_i \text{ and } v_j \text{ are adjacent,} \\ 0 & \text{otherwise.} \end{cases}$$

For an undirected graph A is symmetric, since $(v_i, v_j) \in E$ exactly when $(v_j, v_i) \in E$. For a directed graph it need not be.

Summing a row gives $\sum_j A_{ij} = \deg(v_i)$, because the row is the list of indicators for all possible edges at v_i. Summing every entry gives $2\,|E|$, which is the handshake lemma of Theorem 4.1.6 read off the matrix: adding row-by-rows gives $\sum_i \deg(v_i)$, while adding by entries counts each edge twice.

6.1.2 Multiplying by the Adjacency Matrix

Now represent a position in the graph the same way. Let x be the row vector that is 1 at v_i and 0 elsewhere, which is the indicator of the set $\{v_i\}$, and multiply it by A. Take the graph of Figure 6.1 and start at v_1:



$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \qquad A^2 = \begin{pmatrix} 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 1 \\ 1 & 1 & 3 & 0 \\ 1 & 1 & 0 & 1 \end{pmatrix}$$

Figure 6.1: A graph, its adjacency matrix, and the square of that matrix. The entry $(A^2)_{ij}$ counts walks of length two from v_i to v_j: for instance $(A^2)_{14} = 1$ records v_1, v_3, v_4, and $(A^2)_{34} = 0$ because a two-step route from v_3 to v_4 would need a common neighbor, and the only neighbor of v_4 is v_3 itself. The diagonal are the degrees, since a two-step route from v_i back to v_i is a step out to a neighbor and back.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 1 & 0 \end{pmatrix}.$$

The single 1 in x picks out row 1 of A, and the result is the indicator of the neighborhood, $N(v_1) = \{v_2, v_3\}$. In general $(xA)_j = \sum_k x_k A_{kj} = A_{ij}$: multiplying by A moves you from where you are to everywhere you could be after one step.

Now multiply the result by A again. Its two 1s pick out two rows. Write $(0,1,1,0) = (0,1,0,0) + (0,0,1,0)$; matrix multiplication is linear, $(x+y)A = xA + yA$, so the product is the sum of the two rows:

$$\begin{pmatrix} 0 & 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 & 0 \end{pmatrix} + \begin{pmatrix} 1 & 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 & 1 \end{pmatrix}.$$

More generally, if x is the indicator of a set of vertices, then xA is the sum of their rows, so $(xA)_j$ counts how many vertices of the set are adjacent to v_j. The 2 in the first entry is v_1 being reached from both v_2 and v_3, by the two walks v_1, v_2, v_1 and v_1, v_3, v_1.

Multiply one more time. Now the first entry is 2, so row 1 is counted twice, and the other three rows once each:

$$\begin{pmatrix} 2 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = 2 \begin{pmatrix} 0 & 1 & 1 & 0 \end{pmatrix} + \begin{pmatrix} 1 & 0 & 1 & 0 \end{pmatrix}$$
$$+ \begin{pmatrix} 1 & 1 & 0 & 1 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 4 & 1 \end{pmatrix}.$$

The 4 at v_3 counts the four walks of length three from v_1 to v_3: v_1, v_2, v_1, v_3, and v_1, v_3, v_1, v_3, and v_1, v_3, v_2, v_3, and v_1, v_3, v_4, v_3.

When interpreted this way, a 1 marks a vertex the walk has reached, and multiplying by A sends a copy of each vertex's number to every neighbor, adding up the copies when there are multiple walks that end up on the same place. Figure 6.2 runs this process for three steps.

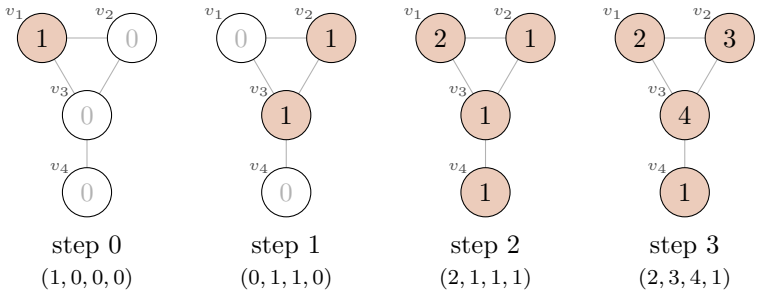


Figure 6.2: Counting walks from v_1 by passing numbers along edges, on the graph of Figure 6.1; small labels are the vertex names. At each step every vertex sends its number to each of its neighbors, and each vertex adds up what it receives. Step 0 is the indicator of $\{v_1\}$, and step 1 is the indicator of $N(v_1) = \{v_2, v_3\}$. At step 2, v_1 receives a 1 from both v_2 and v_3, giving the walks v_1, v_2, v_1 and v_1, v_3, v_1. Each step is the previous one multiplied by A, so step k is row 1 of A^k.

6.1.3 Powers of the Adjacency Matrix

The numbers of Figure 6.2 are rows of the powers of A: step 2, $(2, 1, 1, 1)$, is the first row of the A^2 in Figure 6.1. Writing out the product,

$$(A^2)_{ij} = \sum_m A_{im} A_{mj},$$

and the term for m is 1 exactly when v_m is adjacent to both v_i and v_j, which is to say exactly when v_i, v_m, v_j is a two-step walk. The sum counts the intermediate vertices that are valid middle steps of two-step walks. The same argument repeats, which gives us a more general statement.

Theorem 6.1.2. *For every $k \geq 1$, the entry $(A^k)_{ij}$ equals the number of walks (Definition 4.1.3) of length exactly k from v_i to v_j.*

Proof. Let x be the indicator of $\{v_i\}$. Since xA^k is row i of A^k, the claim is that $(xA^k)_j$ counts the walks of length k from v_i to v_j. Induct on k. For $k = 1$, xA is row i of A, and a walk of length one is an edge. Now, suppose that the claim holds for k, and write $w = xA^k$. Then $xA^{k+1} = wA$, and

$$(wA)_j = \sum_m w_m A_{mj}.$$

A walk of length $k+1$ from v_i to v_j is a walk of length k from v_i to some neighbor v_m of v_j, followed by the edge from v_m to v_j, and distinct choices give distinct walks. By the induction hypothesis there are w_m walks of the first kind ending at each v_m, and A_{mj} is 1 exactly when v_m is a neighbor of v_j, so the sum counts exactly the walks of length $k+1$. □

Example 6.1.3 (The blackboard problem from *Good Will Hunting*)**.** In the 1997 film, a professor leaves a four-part problem on a hallway blackboard as a challenge. Given the multigraph of Figure 6.3, it asks for

(1) the adjacency matrix A,

(2) the matrix giving the number of three-step walks,

(3) the generating function for walks from v_i to v_j,

(4) the generating function for walks from v_1 to v_3.

The first two parts are the content of this lecture. The graph has a double edge between v_2 and v_3, so we extend the definition slightly and let A_{ij} count the edges between v_i and v_j rather than merely indicate whether there is one. Reading the picture gives part (1), and Theorem 6.1.2 tells us that the answer to part (2) is A^3:

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 2 & 1 \\ 0 & 2 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}, \qquad A^3 = \begin{pmatrix} 2 & 7 & 2 & 3 \\ 7 & 2 & 12 & 7 \\ 2 & 12 & 0 & 2 \\ 3 & 7 & 2 & 2 \end{pmatrix}.$$

So there are twelve three-step walks from v_2 to v_3, and none at all from v_3 back to itself, since every walk leaving v_3 must return through v_2 and so takes an even number of steps.

Parts (3) and (4) ask for a power series $\sum_k \omega_k z^k$ whose coefficients are the walk counts of every length at once. Generating functions of that kind are a useful tool, but we do not develop them in this course.

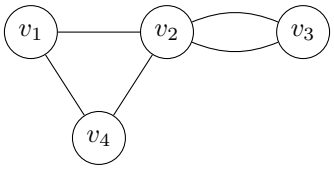


Figure 6.3: The multigraph from the *Good Will Hunting* blackboard problem. Vertices v_2 and v_3 are joined by two edges, so its adjacency matrix has an entry of 2 and the graph is not simple.

Looking ahead

Powers of A also detect a structural property. By the alternation argument of Section 4.1, a bipartite graph has no closed walk of odd length either, so every diagonal entry of A^k is zero for odd k: leaving a vertex and returning takes an even number of steps. In Section 6.4 the same parity will reappear as *periodicity* of a Markov chain, and the example there is a random walk on a bipartite graph.

Theorem 6.1.2 answers both of the questions this lecture opened with. The number of ways to get from v_i to v_j in k steps is an entry of A^k, and v_j is reachable from v_i exactly when $(A^k)_{ij} > 0$ for some $k \leq n-1$, since a path visits at most n vertices. To check reachability for a single pair, it is much faster in practice to explore the graph outward from the start (one vertex at a time) using a search algorithm. Matrix powers are useful because they answer the connectivity question for every pair at once, and because they help build intuition for Markov chains.

6.1.4 Counting Writable Strings

Problem Set 1 counted the *writable* binary strings with no two adjacent 1s. The main technique there was splitting on the last bit, which gave us the Fibonacci numbers. The adjacency matrix does that bookkeeping for us without needing recursion. Make a directed graph whose two vertices record the last bit written, as in Figure 6.4. From 0 we may write either bit, and from 1 only a 0. Its adjacency matrix is

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

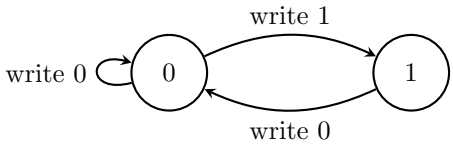


Figure 6.4: Writable strings as walks. Each vertex records the last bit written. From 0 either bit may come next; from 1 only a subsequent 0 may be written.

A walk in this graph gives a writable string: each step writes the bit of the vertex it moves to, and the graph never allows a 1 after a 1. A walk of k steps visits $k+1$ vertices, so it corresponds to a string of $k+1$ bits. By Theorem 6.1.2 each entry of A^k counts the writable strings of length $k+1$ that start and end with the given bits:

$$A^k = \begin{bmatrix} \text{start 0, end 0} & \text{start 0, end 1} \\ \text{start 1, end 0} & \text{start 1, end 1} \end{bmatrix}.$$

Adding all four entries counts every writable string of length $k+1$. For instance

$$A^9 = \begin{bmatrix} 55 & 34 \\ 34 & 21 \end{bmatrix},$$

so there are $55 + 34 + 34 + 21 = 144$ writable strings of length 10, which matches the Fibonacci number Problem Set 1 found by splitting on the last bit.

Real CDs use a more complicated version of the rule: between consecutive 1s there must be at least 2 zeros and at most 10. Finding a recursion for that by hand might be difficult, but the rule can be described using a graph with 11 vertices, recording how many zeros have been written since the last 1. Counting its walks with powers of its adjacency matrix shows that $\log_2(\text{number of strings})/k$ settles at about 0.542, so each slot carries 0.542 bits, compared with 0.694 for the rule of Problem Set 1. The code a CD actually uses stores 8 bits of music in 17 slots, about 0.47 bits per slot, close to that limit.

6.2 Intro to Markov Chains

The chain rule of Section 2.2 factors the probability of any sequence of random variables:

$$\Pr[X_0, X_1, \ldots, X_n] = \Pr[X_0]\ \Pr[X_1 \mid X_0]\ \Pr[X_2 \mid X_0, X_1] \cdots \Pr[X_n \mid X_0, \ldots, X_{n-1}].$$

Each factor may depend on the whole history of preceeding random variables, which means a whole lot of conditional probabilities to estimate and store.

At the other extreme is the geometric distribution of Section 2.4, which is *memoryless* and requires no memory at all. A natural question is what happens in between these cases, i.e., when a process has memory like a DFA (Chapter 5). We call what it remembers its *state* and (like in DFAs) require that the future depend on nothing other than the current state.

6.2.1 The Markov Property

In 1913 Andrey Markov went through the first 20,000 letters of Pushkin's *Eugene Onegin* by hand, counting how vowels and consonants follow one another. He found that a vowel follows a vowel only about 13% of the time, but follows a consonant about 66% of the time: the next letter dependen on what immediately preceeded it.

Definition 6.2.1 (Markov chain)**.** A sequence of random variables $X_0, X_1, X_2, \ldots$ taking values in a set S of *states* is a *Markov chain* if

$$\Pr[X_{t+1} = j \mid X_t = i,\ X_{t-1}, \ldots, X_0] = \Pr[X_{t+1} = j \mid X_t = i]$$

for all t and all states. This condition is the *Markov property*.

In English: the future may be tied to the past, but the only such link can be through dependence on the present. As we've emphasized a few times now, this property is a memory restriction that is very similar to the one we saw in DFAs. In fact, a Markov chain can be thought of as an automaton whose input is chosen by chance.

The Markov property collapses the chain rule into a simpler product. Every factor $\Pr[X_{t+1} \mid X_0, \ldots, X_t]$ becomes $\Pr[X_{t+1} \mid X_t]$, so

$$\Pr[X_0 = i_0, X_1 = i_1, \ldots, X_n = i_n] = \Pr[X_0 = i_0] \prod_{t=0}^{n-1} \Pr[X_{t+1} = i_{t+1} \mid X_t = i_t].$$

The probability of a whole history is the probability of where it starts, times one factor for each step. Everything about the chain is captured by its starting distribution and the "transition" probabilities of moving from one state to the next.

6.2.2 Examples

Example 6.2.2 (Predictive text)**.** A phone keyboard suggests the next word from the word you just typed. Model the text as a Markov chain whose state is the current word, with $\Pr[X_{t+1} = j \mid X_t = i]$ the fraction of times word j follows word i in a large body of text. Sampling the chain generates text. Claude Shannon actually did this in 1948, by hand, with letters and then with words. To choose each next letter, he opened a book at random, read until he reached the current letter, and copied the one after it, so the book itself simulated the transition probabilities. The results looked like English. Modern chatbots condition on far more than the previous word, using a mechanism called *attention*, but the basic idea is the same in principle.

Example 6.2.3 (A random walk at the roulette wheel)**.** A gambler bets one dollar on black at a roulette wheel, again and again. Their fortune m_t after t bets goes up or down by one dollar: up with probability $\frac{18}{38}$ on an American wheel, which has 18 red, 18 black and 2 green pockets, and down otherwise. The fortune is a *random walk*, and a Markov chain (Figure 6.5): your money at the next time step is only dependent on your money at the previous step and not on the run of luck (or lack of luck) that got it there. The same model describes prices: in 1900 Louis Bachelier modeled stock prices as a random walk, which makes sense if, as the *efficient market hypothesis* says, the current price already reflects everything known. This chain has infinitely many states, which is why the definition above did not require S to be finite. Still, this class will focus on finite Markov chains.

The property is an assumption about the model, and whether it holds depends on how the state is defined.

Example 6.2.4 (The motion of a molecule)**.** Record the position X_t of a molecule in a gas at times $t = 0, 1, 2, \ldots$. The positions are not a Markov chain: where the molecule goes next depends on how fast it is moving, and in which direction, not just on where it is. Add its momentum to the state, though, and the pair (position, momentum) is a Markov chain. That pair is exactly what physics calls the state of a particle, and random collisions with other molecules make its motion random.

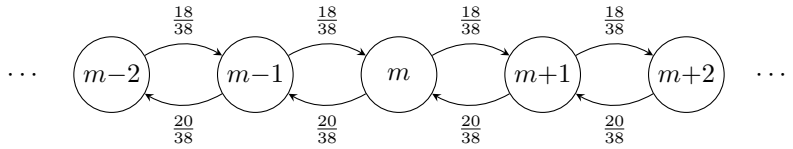


Figure 6.5: The gambler's fortune at the roulette wheel as a Markov chain. From any fortune m, a one-dollar bet on black moves up a dollar with probability $\frac{18}{38}$ and down a dollar with probability $\frac{20}{38}$, no matter how the gambler got to m. The fortune can climb without limit, so the chain has infinitely many states. Section 6.4 cuts it off with walls at 0 and a target N.

In discrete time the momentum can be computed using the last two positions, since their difference is the velocity, so the pair (X_{t-1}, X_t) carries the same information. Enlarging the state this way is the standard way to increase memory until the Markov property holds. In general, suppose X_{t+1} depends on both X_t and X_{t-1}. The pairs $Y_t = (X_{t-1}, X_t)$ then form a Markov chain, since Y_t holds everything the next step needs. The transitions are restricted, because transitioning pairs pairs share their middle value, i.e., (a, b) can be followed only by a pair of the form (b, c), which happens with probability $\Pr[X_{t+1} = c \mid X_{t-1} = a,\ X_t = b]$. The arrows run from each pair to the pairs that begin with its second entry, which is exactly the rule of the de Bruijn graph in Section 5.1.7. For a chain on two values that remembers two steps, the transition graph of the pairs is the de Bruijn graph of Figure 5.5 with a probability on each arrow (Figure 6.6), and remembering m steps gives the de Bruijn graph of order $m + 1$. Overall, any process whose next step depends on a bounded number of previous states is a Markov chain once you expand the state space enough to include the required history. The only thing prohibited by the Markov property is when the future is dependent on an unbounded past.

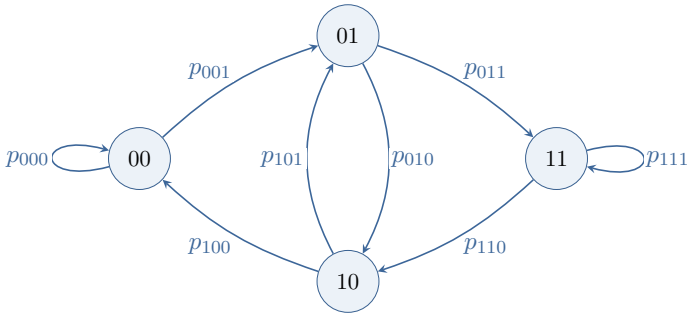


Figure 6.6: The pair chain $Y_t = (X_{t-1}, X_t)$ for a process on $\{0, 1\}$ that remembers two steps. Notice that this is exactly the de Bruijn graph of Figure 5.5, with each arrow $ab \to bc$ labeled by its probability $p_{abc} = \Pr[X_{t+1} = c \mid X_{t-1} = a,\ X_t = b]$. The two arrows out of each pair carry the two possible next values, so $p_{ab0} + p_{ab1} = 1$.

6.2.3 The Transition Matrix

All that matters in a Markov chain are the probabilities of moving from each state to each other state. Draw the states as vertices, and an arrow from i to j labeled with the probability of moving from i to j. Each step, every state *sends its probability* along its arrows, split according to their labels. This (intentionally) echoes Section 6.1, where every vertex sent its count to each of its neighbors. There is an adjacency matrix analog for these

transition probabilities, called the transition matrix.

Definition 6.2.5 (Transition matrix)**.** The *transition matrix* of a Markov chain on n states is the $n \times n$ matrix

$$P_{ij} = \Pr[X_{t+1} = j \mid X_t = i].$$

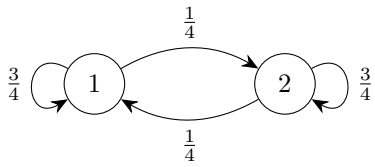
Every row of P is a probability distribution, namely the distribution of where the chain goes next given where it is, so every row sums to 1. A nonnegative matrix with this property is called *stochastic*.

Write $\pi^{(t)}$ for the row vector recording the distribution of X_t, so $\pi_i^{(t)} = \Pr[X_t = i]$. One step of the chain is one matrix multiplication:

$$\pi_j^{(t+1)} = \sum_i \Pr[X_t = i] \Pr[X_{t+1} = j \mid X_t = i] = \sum_i \pi_i^{(t)} P_{ij}, \qquad \text{that is} \qquad \pi^{(t+1)} = \pi^{(t)} P.$$

The middle step is the law of total probability from Section 2.2, with the states as the partition. Iterating, $\pi^{(t)} = \pi^{(0)} P^t$.

6.2.4 Moving Probability Around



$$P = \begin{pmatrix} \frac{3}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{3}{4} \end{pmatrix}$$

Figure 6.7: A sticky two-state chain and its transition matrix, such as an ion channel that is open (state 1) or closed (state 2). Each state stays put with probability 3/4 and switches with probability 1/4, so each row sums to 1. We return to it in the next two lectures, where it turns out to converge to the distribution $(\frac{1}{2}, \frac{1}{2})$ at a rate we can compute.

Example 6.2.6 (A sticky chain)**.** An ion channel in a cell membrane flickers between open and closed, and tends to stay as it is for a while before switching. Model it with the chain of Figure 6.7, checking it every millisecond: state 1 is open and state 2 is closed. Electronics has the same chain, as *random telegraph noise*, when a defect in a transistor traps and releases a single electron and the current jumps between two levels. Starting with the channel open, so $\pi^{(0)} = (1, 0)$, one step gives $\pi^{(1)} = (\frac{3}{4}, \frac{1}{4})$, and one more gives

$$\pi^{(2)} = \begin{pmatrix} \frac{3}{4} & \frac{1}{4} \end{pmatrix} \begin{pmatrix} \frac{3}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{3}{4} \end{pmatrix} = \begin{pmatrix} \frac{10}{16} & \frac{6}{16} \end{pmatrix}.$$

Each step, every state splits its probability and sends it along its arrows (Figure 6.8): state 1 keeps $\frac{3}{4}$ of its $\frac{3}{4}$ and sends $\frac{1}{4}$ of it across, and state 2 does the same with its $\frac{1}{4}$. Continuing,

$$\left(\tfrac{3}{4}, \tfrac{1}{4}\right), \quad \left(\tfrac{10}{16}, \tfrac{6}{16}\right), \quad \left(\tfrac{9}{16}, \tfrac{7}{16}\right), \quad \left(\tfrac{17}{32}, \tfrac{15}{32}\right), \ \ldots$$

so the distribution is getting closer to $(\frac{1}{2}, \frac{1}{2})$, and the distance to it halves at every step.

Try that distribution:

$$\begin{pmatrix} \frac{1}{2} & \frac{1}{2} \end{pmatrix} \begin{pmatrix} \frac{3}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{3}{4} \end{pmatrix} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \end{pmatrix}.$$

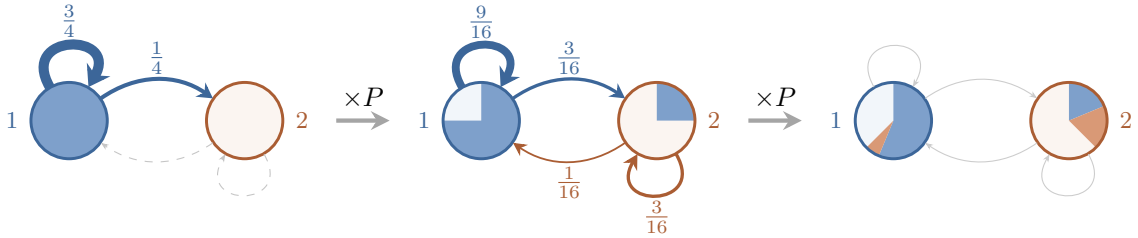


Figure 6.8: Two steps of the sticky chain, starting open, $\pi^{(0)} = (1, 0)$. Each snapshot is the chain of Figure 6.7, with each state drawn as a pie chart whose filled fraction is its probability. Each state has its own color, blue for state 1 and orange for state 2. Every step, each state splits what it holds along its arrows, keeping $\frac{3}{4}$ and sending $\frac{1}{4}$ across, and the arrows are labeled with the amounts they carry, drawn in the color of the state sending them. The pies in the next snapshot are colored by where their probability came from. After one step everything came from state 1, so both pies are blue. After two the colors combine: state 1's $\frac{10}{16}$ is the $\frac{9}{16}$ it kept plus $\frac{1}{16}$ from state 2, and state 2's $\frac{6}{16}$ is $\frac{3}{16}$ from each. Each snapshot to the next is one multiplication by P.

It does not move. A distribution with $\pi P = \pi$ is a *fixed point* of the chain, often called a *stationary distribution*: once the chain is in it, it stays in it. In the language of the next lecture, π is an *eigenvector* of P, from the German *eigen*, "own": a vector the matrix sends back to itself. Section 6.4 explains why the chain approaches it, and why the distance halves.

6.2.5 Time until Absorption

A second kind of question asks not where a chain settles, but when it stops, at a state it can never leave, called *absorbing*. The tool is the law of total probability (in this case, it is technically the tower rule) applied to the first step.

Example 6.2.7 (Waiting for HH or for HT)**.** Does a fair coin take longer, on average, to show HH or HT? Two consecutive flips are HH with probability $\frac{1}{4}$, and HT with probability $\frac{1}{4}$ too, so it seems they should take equally long. To better understand the expected time, feed the coin into the pattern detectors of Figure 5.4. The input is now random, so each detector becomes a Markov chain, and its accepting state is absorbing (Figure 6.9). Write d_ε and d_H for the expected number of further flips from each state.

For HT, a second H keeps the detector at H:

$$d_H = 1 + \tfrac{1}{2}d_H, \qquad d_\varepsilon = 1 + \tfrac{1}{2}d_H + \tfrac{1}{2}d_\varepsilon,$$

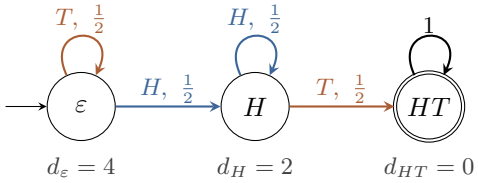
so $d_H = 2$ and $d_\varepsilon = 4$. For HH, a T after an H falls all the way back:

$$d_H = 1 + \tfrac{1}{2}d_\varepsilon, \qquad d_\varepsilon = 1 + \tfrac{1}{2}d_H + \tfrac{1}{2}d_\varepsilon,$$

so $d_H = 4$ and $d_\varepsilon = 6$.

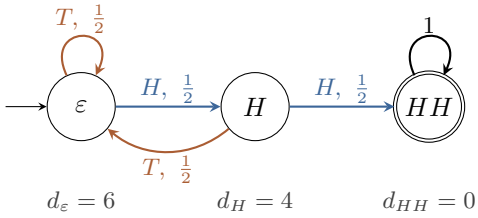
Both patterns have probability $\frac{1}{4}$ at any given position, yet HH takes six flips on average and HT only four. The difference is the single arrow in which the machines differ: a failed HH loses its progress, while a failed HT is already the start of the next attempt.

Conditioning on the first step and solving the resulting equations is called *first-step analysis*. It is the standard tool for anything absorbing: the expected time to failure, or the probability of reaching one state before another.



watching for HT:

$$d_\varepsilon = 1 + \tfrac{1}{2}\, d_H + \tfrac{1}{2}\, d_\varepsilon$$
$$d_H = 1 + \tfrac{1}{2}\, d_H + \tfrac{1}{2} \cdot 0$$



watching for HH:

$$d_\varepsilon = 1 + \tfrac{1}{2}\, d_H + \tfrac{1}{2}\, d_\varepsilon$$
$$d_H = 1 + \tfrac{1}{2} \cdot 0 + \tfrac{1}{2}\, d_\varepsilon$$

Figure 6.9: The pattern detectors of Figure 5.4, fed a fair coin. Every arrow out of a transient state now has probability $\frac{1}{2}$, with heads in blue and tails in orange, and the accepting state is absorbing. First-step analysis reads one equation off each state: one flip, plus $\frac{1}{2}$ times the expected remaining time from wherever each arrow leads, with each term colored to match its arrow. The accepting state needs no more flips, which is where the 0s come from. Solving gives the expected number of further flips, shown under each state. The machines differ in a single arrow, the tail out of H, which finishes the wait for HT but sends the wait for HH back to the start.

6.3 Linear Algebra and Eigenvalues

Last time we saw that the sticky chain of Figure 6.7 has the fixed point

$$\begin{pmatrix} \frac{1}{2} & \frac{1}{2} \end{pmatrix} \begin{pmatrix} \frac{3}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{3}{4} \end{pmatrix} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \end{pmatrix}, \qquad \text{that is,} \qquad \pi M = \pi.$$

This is an example of an *eigenvector* of M, and it gives a stationary distribution for the Markov chain. This lecture develops the linear algebra behind it.

6.3.1 What Is Linearity?

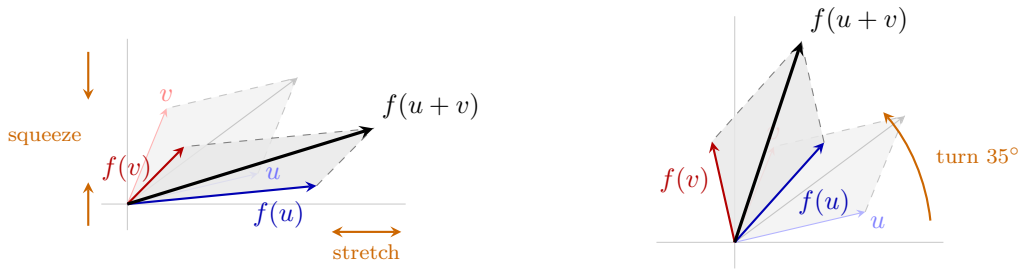
Definition 6.3.1 (Linear)**.** A function f is *linear* if

$$f(a+b) = f(a) + f(b) \qquad \text{and} \qquad f(\alpha a) = \alpha f(a).$$

If this holds, then $f(a) = f(a+0) = f(a) + f(0)$, so $f(0)$ must be 0: a linear function passes through the origin. A function such as $f(x) = mx + b$ with $b \neq 0$ is *affine* but not linear.

6.3.2 Linear Transformations and Vectors

In one dimension, a linear transformation moves points on the number line to other points. In two dimensions, it moves pairs of numbers to pairs of numbers. Stretching and rotating are both linear, as Figure 6.10 shows.



(a) stretched (b) rotated

Figure 6.10: Stretching and rotating are linear. The faded arrows are u and v, whose sum is the diagonal of the parallelogram they make. (a) Stretching the plane horizontally and squeezing it vertically, or (b) rotating it, carries the parallelogram to another parallelogram, so the "image" (function output) of the sum is the sum of the images: $f(u+v) = f(u)+f(v)$.

6.3.3 Stretching

Definition 6.3.2 (Eigenvector, eigenvalue)**.** The direction of stretching or compressing is called an *eigenvector*: a nonzero vector v with $f(v) = \lambda v$. The number λ is its *eigenvalue*.

Writing $f(x) = xM$ for a matrix M, take a diagonal matrix:

$$M = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}, \qquad f(e_1) = e_1 M = \lambda_1 e_1, \qquad f(e_2) = e_2 M = \lambda_2 e_2.$$

So $e_1 = (1\ 0)$ and $e_2 = (0\ 1)$ are eigenvectors, with eigenvalues λ_1 and λ_2 (Figure 6.11): diagonal matrices have the standard basis as eigenvectors.

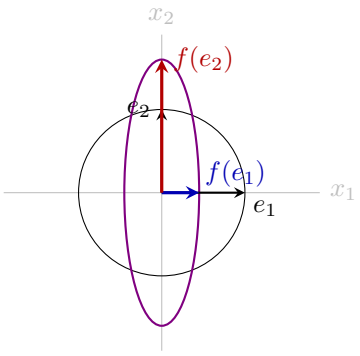


Figure 6.11: A diagonal matrix with $\lambda_1 < 1 < \lambda_2$ takes the unit circle to the ellipse. It sends e_1 to $\lambda_1 e_1$ and e_2 to $\lambda_2 e_2$: the axes are only stretched or compressed, never turned.

Eigenvectors need not be the axes, though. For $M = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$, $f(e_1) = 2e_1 + e_2$ and $f(e_2) = e_1 + 2e_2$, and the eigenvectors are the diagonal directions:

$$v_1 = (1, 1),\ \lambda_1 = 3, \qquad v_2 = (1, -1),\ \lambda_2 = 1$$

(Figure 6.12). Finding eigenvectors can be hard, but checking a guess takes one multiplication.

6.3.4 Basis Changes

A vector can be written in terms of the standard basis e_1, e_2, or in terms of eigenvectors v_1, v_2 (Figure 6.13):

$$(1, 0) = 1 \cdot e_1 = \tfrac{1}{2}(1, 1) + \tfrac{1}{2}(1, -1).$$

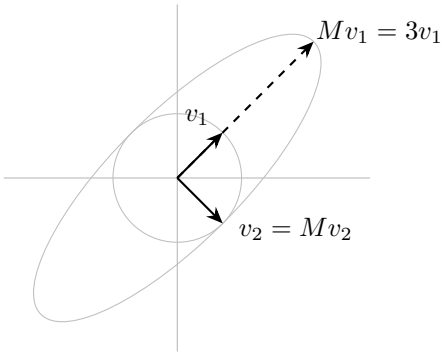
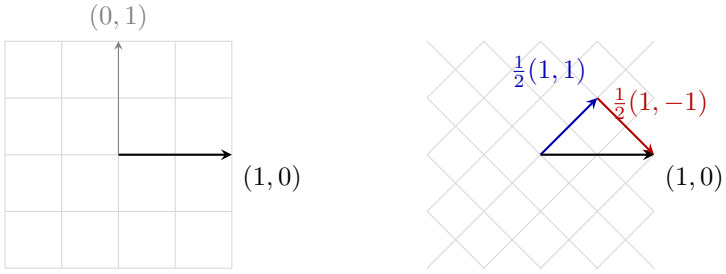


Figure 6.12: The map $M = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$ applied to the unit circle, which it takes to the ellipse. The two eigenvectors are $v_1 = (1, 1)$ with $\lambda_1 = 3$ and $v_2 = (1, -1)$ with $\lambda_2 = 1$. Along those two directions the map only stretches.

This is what shows what a matrix does to a vector that is not an eigenvector: it scales each eigenvector component by its own eigenvalue. For $M = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$,

$$(1, 0)\, M = \tfrac{1}{2} \cdot 3\,(1, 1) + \tfrac{1}{2} \cdot 1\,(1, -1) = (2, 1).$$

The two components are scaled by different amounts, so the vector turns, toward $(1, 1)$, the direction stretched more. If the eigenvalues are equal, nothing turns: $\left(\begin{smallmatrix} 2 & 0 \\ 0 & 2 \end{smallmatrix}\right)$ scales both components by 2, so every vector is an eigenvector, with eigenvalue 2.



(a) the usual grid (b) the eigenvector grid

Figure 6.13: The same vector in two bases. (a) On the usual grid, $(1, 0)$ is one step along the first axis. (b) On a grid tilted along the eigenvectors $(1, 1)$ and $(1, -1)$ of the sticky chain, it is half a step along each: $(1, 0) = \frac{1}{2}(1, 1) + \frac{1}{2}(1, -1)$.

6.3.5 The Power Method

Suppose M has two eigenvectors v_1, v_2 with eigenvalues $|\lambda_1| > |\lambda_2|$. For any $x = \alpha v_1 + \beta v_2$,

$$xM = \alpha\lambda_1 v_1 + \beta\lambda_2 v_2 \quad \Longrightarrow \quad xM^k = \alpha\lambda_1^k v_1 + \beta\lambda_2^k v_2.$$

Both λ_1^k and λ_2^k are exponential, but λ_1^k grows much faster, so as k increases,

$$\frac{xM^k}{\|xM^k\|} \longrightarrow \frac{v_1}{\|v_1\|}$$

(up to sign, and provided $\alpha \neq 0$). This is a numerical way to find an eigenvector, and it shows that repeated linear transformations converge on the eigenvector with the largest eigenvalue, if the eigenvalues are distinct in size.

6.3.6 Rotations

The power method assumed that the largest eigenvalue is real and positive. What else can eigenvalues do? Start with a negative eigenvalue. The reflection $F = \left(\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right)$ flips the plane (Figure 6.14), and $F^2 = I$: it is periodic with period 2. In general, if $xA = \lambda x$ then $xA^2 = \lambda^2 x$.

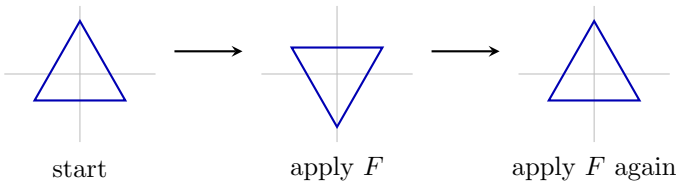


Figure 6.14: The reflection $F = \left(\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right)$ flips the plane top to bottom. Its eigenvectors are the axes: $(1, 0)$ with eigenvalue 1, and $(0, 1)$ with eigenvalue -1. Applying it twice gives back the start, so a negative eigenvalue makes an oscillation of period 2.

An eigenvalue of $\sqrt{-1} = i$ has period 4, since $i^4 = 1$. If we have two complex eigenvalues, they "trade off": together they make a rotation. Take the quarter turn (Figure 6.15)

$$R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

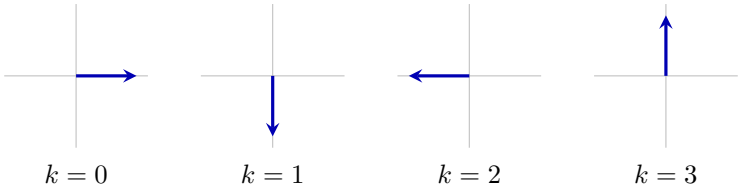


Figure 6.15: Applying the quarter turn R again and again: $(1, 0)R^k$ for $k = 0, 1, 2, 3$ is $(1, 0)$, $(0, -1)$, $(-1, 0)$, $(0, 1)$.

Since $R^2 = -I$, its eigenvalues satisfy $\lambda^2 = -1$, so they are $\pm i$. An eigenvector can always be scaled to start with 1, so try $(1, c)$: $(1, c)\, R = (c, -1)$, and setting this equal to $i(1, c)$ gives $c = i$. So $(1, i)$ is an eigenvector with eigenvalue i, and $(1, -i)$ is one with eigenvalue $-i$. Writing a real vector as half of each,

$$(1, 0) = \tfrac{1}{2}\big[(1, i) + (1, -i)\big] \quad \Longrightarrow \quad (1, 0)\, R = \tfrac{1}{2}\big[(i, -1) + (-i, -1)\big] = (0, -1),$$

the imaginary parts cancel, and each step is a quarter turn.

6.3.7 Useful Facts

1. Symmetric matrices, with $A_{ij} = A_{ji}$, always have real eigenvalues. Think of $\left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$ and how a vector with its entries flipped must map to that vector's output with its entries also flipped. This always lacks rotation.

2. Matrices with all positive entries have at least one real, positive eigenvalue.

3. Stochastic matrices, whose rows sum to 1, have 1 as their largest eigenvalue (Section 6.4). Transition matrices are stochastic.

6.4 Convergence, Ergodicity, and Stationary Distributions

Running a chain forward is repeated multiplication by P, the power method says repeated multiplication turns toward the eigenvector with the largest eigenvalue, and 1 is always an eigenvalue of a stochastic matrix. In Section 6.2 the sticky chain settled on $(\frac{1}{2}, \frac{1}{2})$, and the distance to it halved at every step; the chain's eigenvalues are 1 and $\frac{1}{2}$. This lecture first pins down when the power method actually works, shows that nothing beats 1, then makes the convergence precise, and finds the two conditions under which it is actually true. We then dig into two famous examples: gambler's ruin, which is not ergodic, and PageRank, which is.

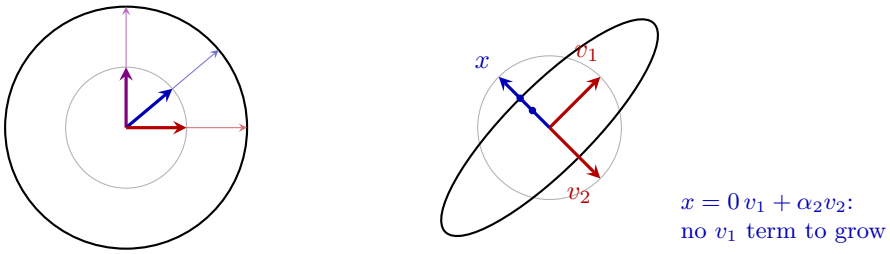
6.4.1 When the Power Method Works

The power method of Section 6.3 turns xM^k toward the top eigenvector v_1, but only if two conditions hold (Figure 6.16).

1. *The largest eigenvalue must be strictly bigger in size than all the others.* Take $M = \left(\begin{smallmatrix} 2 & 0 \\ 0 & 2 \end{smallmatrix}\right)$. Every vector is an eigenvector with eigenvalue 2, so the unit circle maps to a circle twice as big and no direction ever changes: $xM^k / \left\|xM^k\right\| = x / \|x\|$ for every k. No direction beats any other, and you end up pointing wherever you started. A tie can also keep the vector moving forever without settling. The flip F has eigenvalues 1 and -1, so xF^k bounces between two directions. The quarter turn R has the complex eigenvalues $\pm i$, both of size 1, so xR^k goes around and around (Figure 6.15). The complex eigenvalues of a real matrix always come in pairs like this, with equal size, so a complex top eigenvalue always ties, and its rotation never settles.

2. *The starting vector must have a non-trivial amount of the top eigenvector.* If $x = 0\,v_1 + \alpha_2 v_2$, then $xM^k = \alpha_2 \lambda_2^k v_2$, and there is no v_1 term to grow. For example, when the eigenvectors are perpendicular, as in the figure, then $\alpha_1 = 0$ and there is no component to grow through power iterations of matrix multiplication.

Both requirements come back below for Markov chains. The first is where irreducibility and aperiodicity come in, and the second turns out to be automatic.



(a) $M = \left(\begin{smallmatrix} 2 & 0 \\ 0 & 2 \end{smallmatrix}\right)$ (b) a start orthogonal to v_1

Figure 6.16: The two ways the power method fails. (a) For $M = \left(\begin{smallmatrix} 2 & 0 \\ 0 & 2 \end{smallmatrix}\right)$ every vector is an eigenvector with the same eigenvalue, so the unit circle (gray) maps to a circle of radius 2 and every direction stays where it was. (b) M stretches the unit circle into the ellipse, long along v_1. A start x lying along the v_2 axis shrinks along that axis (dots) and never turns toward v_1.

6.4.2 Nothing Beats 1

Running the chain for k steps, $\pi^{(0)}P^k$, is the power method applied to a distribution, and every vector along the way is a probability distribution, with entries that add to 1. An eigenvalue bigger than 1 in size would make some component grow without bound, so we expect every eigenvalue to be at most 1 in size. And 1 itself is always an eigenvalue.

Proposition 6.4.1. *Let P be a stochastic matrix. Then $\lambda = 1$ is an eigenvalue of P, and every eigenvalue λ of P satisfies $|\lambda| \leq 1$.*

Proof. Let $\mathbb{1}$ be the column vector of all ones. The ith entry of $P\mathbb{1}$ is the sum of row i, which is 1, so $P\mathbb{1} = \mathbb{1}$ and 1 is an eigenvalue.

For the bound, suppose $Pv = \lambda v$ and let m be an index where $|v_m|$ is largest. Taking the mth entry and using that the entries of row m are nonnegative and sum to 1,

$$|\lambda|\,|v_m| = \left|\sum_j P_{mj} v_j\right| \leq \sum_j P_{mj}\,|v_j| \leq \sum_j P_{mj}\,|v_m| = |v_m|\,.$$

Dividing by $|v_m| > 0$ gives $|\lambda| \leq 1$. □

Remark 6.4.2 (Left and right eigenvectors). Notice which side the all-ones vector is on. It is a column, multiplied on the right, $P\mathbb{1} = \mathbb{1}$, so it is a *right* eigenvector. The vectors we push through the chain are rows, multiplied on the left, as in Section 6.3: the row eigenvector with eigenvalue 1 is the stationary distribution, $\pi P = \pi$. A detail from linear algebra that we will just take on faith for now[a] is that a matrix has the same eigenvalues from either side but usually different eigenvectors, so $\mathbb{1}$ and π are two views of the same eigenvalue 1. For the sticky chain they happen to point the same way, $(\frac{1}{2}, \frac{1}{2})$, because its P is symmetric; for $P = \left(\begin{smallmatrix} 1/2 & 1/2 \\ 1/4 & 3/4 \end{smallmatrix}\right)$ the rows still sum to 1, but $\pi = (\frac{1}{3}, \frac{2}{3})$.

[a] Don't worry if left vs. right eigenvectors and eigenvalues confuse you, as this point is minor

So the eigenvalues of a transition matrix sit in the unit disc, and 1 is always included (there is always a stationary distribution). Nothing can grow under repeated multiplication, which makes sense because the entries of $\pi^{(t)}$ are probabilities.

6.4.3 Stationary Distributions as Left Eigenvectors

Definition 6.4.3 (Stationary distribution)**.** A probability distribution π is *stationary* for P if $\pi P = \pi$.

A stationary distribution is a left eigenvector of P with eigenvalue 1, normalized to sum to 1. Proposition 6.4.1 guarantees that the eigenvalue 1 exists, and a separate argument, which we omit, gives a corresponding left eigenvector with nonnegative entries.

Example 6.4.4. For our two-state chain, solving $\pi P = \pi$ with $\pi_1 + \pi_2 = 1$ gives $\pi = (\frac{1}{2}, \frac{1}{2})$, which is the distribution the sticky chain approached in Section 6.2. Once the chain is in its stationary distribution it stays there. Note that this does not mean the chain stops moving between states: stationarity is a statement about the distribution, not about the chain sitting still.

Convergence needs conditions, though, as the next two chains show.

6.4.4 Reducible Chains

Definition 6.4.5. States i and j *communicate* if the chain can get from i to j and from j to i, each with positive probability in some number of steps. A chain is *irreducible* if every pair of states communicates. Otherwise it is *reducible*, and it splits into communicating classes.

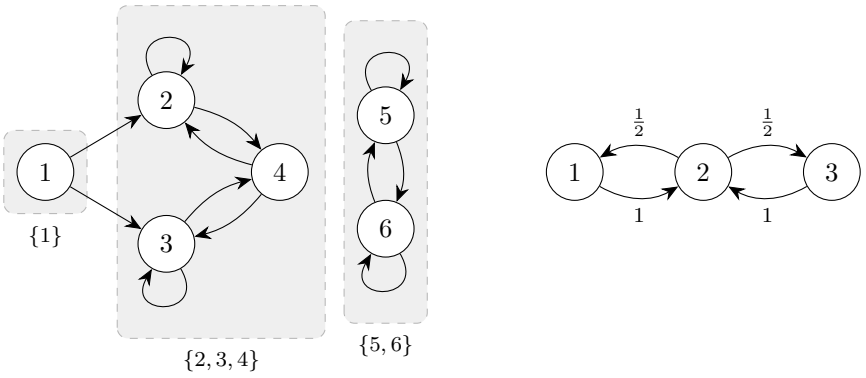


Figure 6.17: The two ways convergence fails. On the left, every arrow has probability $\frac{1}{2}$, and the dashed boxes are the three communicating classes. States 2 and 4 communicate, as do 4 and 3, so 2 and 3 communicate through 4. State 1 can be left but never re-entered, and $\{5, 6\}$ is cut off from everything else. On the right, the chain moves from state 2 to 1 or 3 with probability $\frac{1}{2}$ each and always comes straight back, so state 2 is occupied only every other step and the distribution oscillates instead of settling.

The left panel of Figure 6.17 is reducible. Once the chain enters $\{2, 3, 4\}$ it stays there forever, and the rest of the chain no longer matters; the same goes for $\{5, 6\}$. Each of these classes has its own stationary distribution, uniform in both cases,

$$(0, \tfrac{1}{3}, \tfrac{1}{3}, \tfrac{1}{3}, 0, 0) \qquad \text{and} \qquad (0, 0, 0, 0, \tfrac{1}{2}, \tfrac{1}{2}),$$

and every mixture of the two is stationary as well. A chain started at 1, 2, 3, or 4 approaches the first, and one started at 5 or 6 approaches the second, so there is no single answer to the long-run question. This is the first way the power method fails: the eigenvalue 1 appears twice, once for each closed class, just as every direction had eigenvalue 2 for $M = 2I$.

6.4.5 Periodic Chains

Definition 6.4.6. A state has *period* d if the chain can return to it only at times that are multiples of d. A chain is *aperiodic* if every state has period 1.

The right panel of Figure 6.17 has period 2: state 2 can only be reached every other step. Starting at state 1, the chain is at state 2 at every odd time and at state 1 or 3 at every even time. The distribution never settles, even though the chain is irreducible and has a perfectly good stationary distribution. The chain therefore always remembers the parity of the time step, stored by whether it is in states $\{1, 3\}$ or state 2.

In terms of eigenvalues, the transition matrix

$$P = \begin{pmatrix} 0 & 1 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 1 & 0 \end{pmatrix}$$

has stationary distribution $\pi = (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$, and it also has the eigenvalue -1, the flip of Section 6.3, with left eigenvector $v = (\frac{1}{4}, -\frac{1}{2}, \frac{1}{4})$. Starting from state 1,

$$\pi^{(t)} = \pi + (-1)^t v \qquad (t \geq 1),$$

which is $(0, 1, 0)$ at odd times and $(\frac{1}{2}, 0, \frac{1}{2})$ at even times. The term $(-1)^t v$ never decays, because $|-1| = 1$ ties the eigenvalue 1. This repeats the failure of the power method, but in a way that oscillates on a period.

Aperiodic chains can be thought of as having states that communicate *through time*. Unroll the chain, with a copy of every state at every time step (Figure 6.18). The (state, time) pairs split into two groups, red and blue, and probability that starts on one color stays on that color forever. A periodic chain is reducible in time, and an aperiodic chain is one whose states communicate through time as well as through the graph.

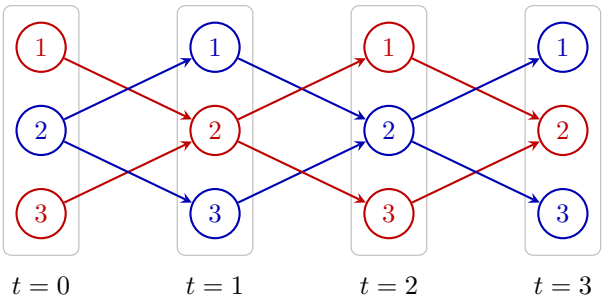


Figure 6.18: The periodic chain of Figure 6.17 unrolled in time, with a column of states for each time step. Arrows out of state 2 have probability $\frac{1}{2}$ and the rest have probability 1. Red marks states 1 and 3 at even times and state 2 at odd times, and blue marks the rest. Every arrow stays within its color, so probability that starts on red stays on red: the two colors are communicating classes in time, and they never mix.

Adding a self-loop anywhere destroys periodicity, which is why the chains we care about in practice are usually aperiodic without any effort.

6.4.6 Ergodicity

Definition 6.4.7 (Ergodic)**.** A Markov chain is *ergodic* if it is irreducible and aperiodic.

Theorem 6.4.8 (Convergence to stationarity)**.** *An ergodic Markov chain on finitely many states has a unique stationary distribution π, and for every starting distribution $\pi^{(0)}$,*

$$\pi^{(0)} P^t \longrightarrow \pi \qquad \text{as } t \to \infty.$$

Aperiodicity and irreducibility let probability swish around the whole chain and stabilize, instead of being trapped in one class or sloshing back and forth with the clock. To see how, let's go back to the power method's two requirements.

Sketch. Assume P has a full set of left eigenvectors, and write the starting distribution in terms of them:

$$s = \alpha_1 \pi + (\alpha_2 v_2 + \alpha_3 v_3 + \cdots).$$

1. *P conserves probability.* The rows of P sum to 1, so the entries of sP add up to the same total as the entries of s.

2. *Every other eigenvector sums to zero.* If $vP = \lambda v$ with $\lambda \neq 1$, add up the entries on both sides. The left side has the same total as v, and the right side has λ times that total, so the total must be 0. These eigenvectors carry no probability; they just move it around.

3. *The start has a full share of π.* Adding up the entries of s gives $1 = \alpha_1 \cdot 1 + 0$, so $\alpha_1 = 1$. The power method's second requirement holds automatically: every distribution has a π component to grow, whatever the chain.

4. *Everything else dies out.* After t steps,

$$sP^t = \pi + \alpha_2 \lambda_2^t v_2 + \alpha_3 \lambda_3^t v_3 + \cdots .$$

No eigenvalue is bigger than 1 in size (Proposition 6.4.1), and ergodicity makes every eigenvalue other than the one for π strictly smaller than 1 in size. So every term after π goes to zero, and $sP^t \to \pi$. □

Only step 4 uses the hypotheses, which is the power method's first requirement: the largest eigenvalue must win outright. Loosely, irreducibility ensures that π is unique, so the eigenvalue 1 is not repeated. In the reducible chain of Figure 6.17, step 3 would have two terms whose entries sum to more than zero, one for each stationary distribution. Step 3 then says the start puts total weight 1 on the two stationary distributions, but not how that weight splits between them. The extreme case is the identity matrix, the transition matrix of a chain that never moves: every distribution is stationary, exactly the $M = 2I$ picture. Aperiodicity rules out the other eigenvalues of size 1: the -1 of the period-2 chain, whose eigenvector sums to zero as step 2 says it must but never shrinks, or the complex eigenvalues of a longer cycle, which rotate forever as in Section 6.3. The resulting gap of $|\lambda_2| < 1$ ensures that the π term wins across many iterations.

An ergodic chain *mixes*: after enough time it forgets where it started. A second proof strategy, called coupling, runs two copies of the chain from different starting states and argues that they meet; from the moment they meet they are indistinguishable, so the starting state cannot have mattered. This type of argument is very similar to synchronizing words on DFAs, which you have studied in the problem sets.

6.4.7 A Non-Ergodic Example: Gambler's Ruin

Example 6.4.9 (Gambler's ruin)**.** A gambler with i dollars bets one dollar at a time, winning each bet with probability p, and stops on reaching zero or a target of N dollars. The fortune is a Markov chain on $0, 1, \ldots, N$ (Figure 6.19(a)). This closely mirrors the roulette walk of Figure 6.5 cut off by two absorbing ends, so the chain is reducible. Write $h(i)$ for the probability of reaching N first. First-step analysis, as in Section 6.2, gives $h(0) = 0$, $h(N) = 1$, and

$$h(i) = p\,h(i+1) + (1-p)\,h(i-1) \qquad \text{for } 0 < i < N.$$

To solve it, rewrite the equation as

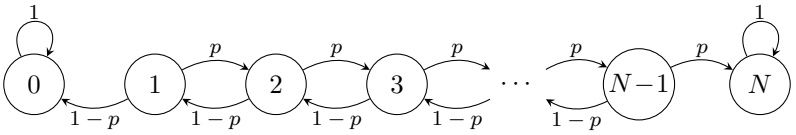
$$p\,\big[h(i+1) - h(i)\big] = (1-p)\,\big[h(i) - h(i-1)\big],$$

so each step up, the difference $h(i+1) - h(i)$ is multiplied by $r = (1-p)/p$. So $h(i) = h(1)\,(1 + r + \cdots + r^{i-1})$, and choosing $h(1)$ so that $h(N) = 1$ gives

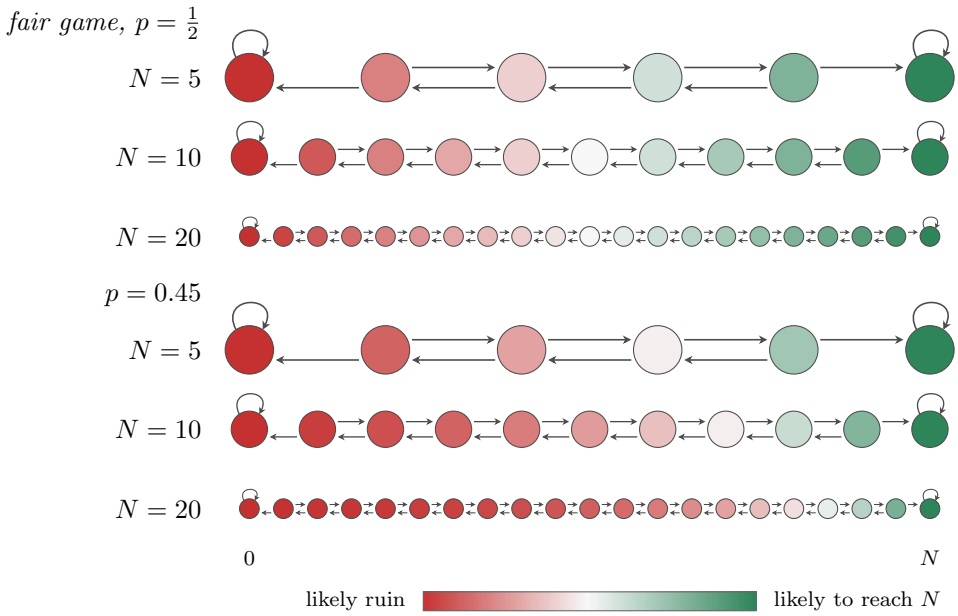
$$h(i) = \frac{1 - r^i}{1 - r^N}.$$

In a fair game $r = 1$, every difference is the same, meaning that the probability of winning grows linearly, $h(i) = i/N$, and is favorable once you start above $N/2$. The chain does settle, at 0 or at N forever, but where it settles depends on where it started.

A small advantage or disadvantage makes a big difference: with $N = 100$ and 50 dollars to start, a fair game reaches the winning target with probability 0.5, but at $p = 0.49$ that drops to 0.119. This is because the house's advantage has a long time to compound into a systematic loss rate for the player. In any unfavorable game, $p < \frac{1}{2}$, so $r > 1$, and the crossover where $h(i) = \frac{1}{2}$ climbs toward N as N grows. At $p = 0.45$, $r = 11/9 \approx 1.22$. With $N = 4$ and 2 dollars to start you still win about 40% of the time, but with $N = 100$ and 50 dollars the chance is about 0.00004. Figure 6.19(b) shows how the crossover moves with the target.



(a) the chain



(b) where the game ends, by starting state

Figure 6.19: Gambler's ruin. (a) As a Markov chain: from each fortune between 0 and N, the gambler wins a dollar with probability p or loses one with probability $1 - p$, and the ends are absorbing. (b) The same chain for larger targets, with each state colored by the chance of reaching N before 0 from there, from red for likely ruin to green for likely reaching N. In a fair game the colors run evenly from one end to the other for every N. At $p = 0.45$, the halfway point stays about 3.5 states below N, so as N grows, almost every starting state is red.

> *Remark* 6.4.10 (Betting strategies). Notice that the $N = 4$ and 2 dollar case can be made comparable to the $N = 100$ and 50 dollar case by simply making 25-dollar bets, starting with 50 dollars and retiring at 100. That raises your chance of winning from 0.00004 to 0.40, and a single all-in bet of 50 dollars does better still, winning with probability 0.45.

> This calculation therefore yields some unexpected gambling advice: if the odds are against you . . . go all in![a] This is because you make fewer bets for the house's advantage to take full effect. Of course, if the odds are with you (as they are for investing), you should diversify and let the law of large numbers work its magic.[b]
>
> [a]This is not official gambling advice, but it is a celebrated result by Dubins and Savage from 1965.
> [b]Still not financial advice.

6.4.8 An Ergodic Example: PageRank

What if we opened up the internet and clicked links at random? Build the directed graph whose vertices are pages and whose edges are links, and let A be its adjacency matrix and d_i the out-degree of page i. A surfer who follows a uniformly random link on the current page moves according to

$$P_{ij} = \frac{A_{ij}}{d_i},$$

the row-normalized adjacency matrix of Section 6.1.

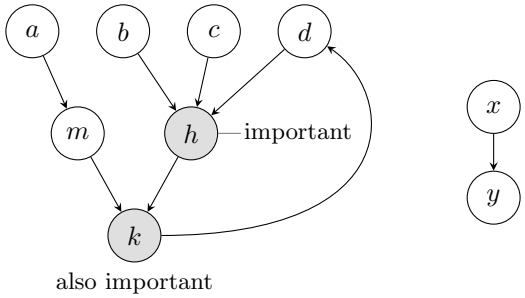


Figure 6.20: Random surfing on a small web, as drawn in lecture. A page with many incoming links is important, like h, which has three. But a link from an important page is worth more than a link from an obscure one: k has only two incoming links and is also important, because one of them comes from h. The cleanest comparison is d and m, which each have exactly one incoming link, d's from k and m's from the obscure page a. With teleportation at $\alpha = 0.85$ the PageRanks are about 0.29 for k, 0.28 for h, 0.27 for d, and 0.04 for m. Without teleportation the surfer is not ergodic in three ways: a, b, c, and m can be left but never revisited, $d \to h \to k \to d$ is a cycle with period 3, and $\{x, y\}$ is cut off from the rest, with y dangling: it has no links out at all.

In 1996, two Ph.D. students at Stanford named Sergey Brin and Larry Page had the idea of using this model to understand the importance of search results. The idea was that links carried importance, and links from important pages were even better (Figure 6.20). A random surfer simulated this, because webpages collected probability from their incoming links, and more important/probable links could then send out more probability to the websites they pointed to. The two students dropped out of school and founded a company called Google, which some of you have likely heard of.

On its own this random surfer is not ergodic and won't give us a good stationary distribution that aligns with importance. Instead, the naive random surfer would just give us a list of sinks that don't link anywhere. To fix this, we can add a small probability of jumping to a random page. In Google's matrix, the surfer follows a random link with probability α, and jumps to a page chosen uniformly at random with probability $1 - \alpha$. This gives

$$G = \alpha P + (1 - \alpha)\frac{1}{n}J,$$

where J is the matrix of all ones and n is the number of pages. The jump is called *teleportation* and α the *damping factor*, usually about 0.85. A "sink" page with no links is therefore treated as if it linked to everything, making it lose its advantage.

Every entry of G is now strictly positive, which settles the two hypotheses of the convergence theorem. The chain can reach any page from any other in one step, so it is irreducible. Every page has a positive probability of teleporting to itself, which is a self-loop, so no state has a period and the chain is aperiodic. The chain is therefore ergodic, and by the convergence theorem it has a unique stationary distribution which it approaches from any starting point. That distribution is now called the PageRank. It is computed with the power method, which parallelizes well across many machines (hold tight until Section 7.3 if you want to hear more about how!).

> *Remark* 6.4.11 (Other uses). Nothing in the argument is specific to the internet. Any situation with directed endorsements has the same structure: ranking sports teams or chess players by who beat whom; recommending who to follow on a social network; prioritizing which pages a crawler should visit; and studying traffic flow through a road network.

6.5 Reversible Markov Chains and MCMC

Finding a stationary distribution means solving $\pi P = \pi$, which can take a long time if we have a lot of states. When the state space is as large as, say, the foldings of an RNA molecule, we cannot even write the transition matrix P down without running out of memory. This lecture gives a shortcut that works for a wide class of chains, and then flips the shortcut around: instead of finding the stationary distribution of a given chain, we design a chain whose stationary distribution is one we want to sample from, first for physics and then for Bayesian statistics. Section 7.3 uses the shortcut again, for queues.

6.5.1 Reversibility and Detailed Balance

Definition 6.5.1 (Detailed balance)**.** A distribution π satisfies *detailed balance* for P if

$$\pi_i P_{ij} = \pi_j P_{ji} \qquad \text{for all } i, j.$$

A chain whose stationary distribution satisfies detailed balance is called *reversible*.

It's helpful to think about this as a statement about probability flow. Recall that multiplying a vector of probabilities by a transition matrix "moves probability around" the system. In equilibrium $\pi_i P_{ij} = \Pr[X_t = i,\ X_{t+1} = j]$, so detailed balance says

$$\Pr[X_t = i,\ X_{t+1} = j] = \Pr[X_t = j,\ X_{t+1} = i]\,:$$

a jump from i to j is exactly as common as the jump back. That is where the name comes from. A reversible chain in equilibrium looks the same run backward in time.

Another way to think about this is as imports and exports. If we are in a stationary distribution, then the probability that is sent into a state must be equal to the probability that it sends out. One way to achieve this is with detailed balance, where the "probability trade deficit" is balanced with all of the state's neighbors, with each neighboring state sending as much probability as it receives.

Proposition 6.5.2. *If π satisfies detailed balance for P then π is stationary.*

Proof. Sum the detailed balance equations over i and use that the rows of P sum to one:

$$(\pi P)_j = \sum_i \pi_i P_{ij} = \sum_i \pi_j P_{ji} = \pi_j \sum_i P_{ji} = \pi_j.$$

□

Detailed balance is a set of equations about pairs of states, which can be solved locally, edge by edge, whereas $\pi P = \pi$ couples everything together. Figure 6.21 carries out an example of this computation. Each edge gives one equation relating two neighboring values of π, the chain of equations determines π up to a constant, and the requirement that the entries sum to 1 fixes the constant.

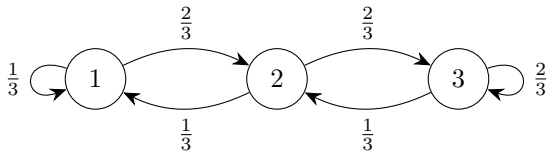


Figure 6.21: A reversible chain. Detailed balance across the edge $\{1, 2\}$ reads $\pi_1 \cdot \frac{2}{3} = \pi_2 \cdot \frac{1}{3}$, and across $\{2, 3\}$ it reads $\pi_2 \cdot \frac{2}{3} = \pi_3 \cdot \frac{1}{3}$. So $\pi_2 = 2\pi_1$ and $\pi_3 = 2\pi_2$, and normalizing gives $\pi = (\frac{1}{7}, \frac{2}{7}, \frac{4}{7})$ with no eigenvector computation at all.

If we return to thinking about stationarity as a balanced trade deficit, we realize that detailed balance is not necessarily needed to balance imports and exports overall, since we may carry a trade deficit for some state neighbors and a trade surplus for other neighbors. If you don't like my metaphor, I'm essentially just saying that some Markov chains may not be reversible, but can still have a stationary distribution where some states send the state extra probability to compensate for the states that don't send back enough probability. The next example shows this.

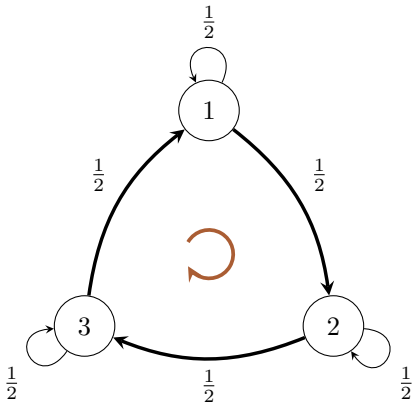


Figure 6.22: A probability vortex. Each state stays put with probability $\frac{1}{2}$ and steps clockwise with probability $\frac{1}{2}$. The uniform distribution is stationary: each state keeps $\frac{1}{3} \cdot \frac{1}{2}$ and takes in $\frac{1}{3} \cdot \frac{1}{2}$ from its other neighbor, exactly the $\frac{1}{3}$ it started with. But nothing ever flows back, so in the long run every step carries $\frac{1}{6}$ of probability clockwise across every edge (orange). The probability that leaves 1 for 2 is replaced by probability coming in from 3.

Example 6.5.3 (A probability vortex)**.** Put three states on a triangle. From each one, stay put with probability $\frac{1}{2}$ or step clockwise with probability $\frac{1}{2}$ (Figure 6.22). Each state

keeps half of what it has and receives half of its counterclockwise neighbor's, so the uniform distribution $\pi = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ is stationary: $(\pi P)_j = \frac{1}{3} \cdot \frac{1}{2} + \frac{1}{3} \cdot \frac{1}{2} = \frac{1}{3}$. The chain is irreducible, since it can go all the way around, and aperiodic, since every state has a self-loop, so it is ergodic and converges to π. But detailed balance fails on every edge:

$$\pi_1 P_{12} = \tfrac{1}{3} \cdot \tfrac{1}{2} = \tfrac{1}{6} \qquad \text{but} \qquad \pi_2 P_{21} = \tfrac{1}{3} \cdot 0 = 0.$$

Each state receives exactly as much probability as it loses, which is all that is needed for stationarity, but the probability going out of 1 to 2 is coming in from 3. A net $\frac{1}{6}$ of probability flows clockwise around the triangle at every step, forever.

It's fairly easy to see why the probability vortex isn't reversible: forwards in time the states move clockwise (even if they don't do so periodically), and backward in time the states move counterclockwise. Unlike a reversible chain, the vortex shows you which way time is running.

> *Remark* 6.5.4. Reversibility and ergodicity are independent conditions. The vortex is ergodic but not reversible. In the other direction, the period-2 chain of Figure 6.17 is reversible, with $\pi = (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$ and $\pi_1 P_{12} = \frac{1}{4} = \pi_2 P_{21}$, but it is periodic, and two reversible chains with no edges between them form a reversible chain that is reducible.

6.5.2 The Ehrenfest Diffusion Model

Tatyana and Paul Ehrenfest[1] introduced the following chain in 1907 to study diffusion and the second law of thermodynamics. A container is divided into two halves, A and B, holding N molecules between them. At each step, with probability q a uniformly chosen molecule moves to the other side, and with probability $1 - q$ nothing happens.

Let the state be n, the number of molecules in A. Moving a molecule out of A requires choosing one of the n in A, and moving one in requires choosing one of the $N - n$ in B, so

$$P_{n,n-1} = q\,\frac{n}{N}, \qquad P_{n,n+1} = q\,\frac{N-n}{N}.$$

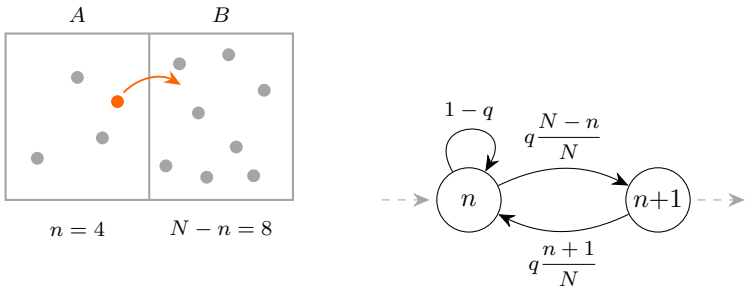


Figure 6.23: The Ehrenfest model with $N = 12$ molecules, currently in state $n = 4$. At each step one molecule is chosen at random and changes sides, so the state moves up or down by one. A move into A requires picking one of the $N - n$ molecules in B, and a move out requires picking one of the n in A, which makes the chain a birth-death chain whose rates pull it toward $n = N/2$.

Figure 6.23 shows the process and the resulting chain. The rates determine the physics: when A is nearly empty almost every move puts a molecule into it, and when A is nearly full almost every move takes one out, so the chain trends toward an even split.

[1] Coincidentally, these two are the parents of the other Tatyana Ehrenfest in this course, who generalized de Bruijn graphs.

Detailed balance across the edge $\{n, n+1\}$ gives

$$\pi_n \, q \, \frac{N-n}{N} = \pi_{n+1} \, q \, \frac{n+1}{N} \qquad \Longrightarrow \qquad \pi_{n+1} = \pi_n \frac{N-n}{n+1}.$$

Applying this repeatedly from π_0,

$$\pi_n = \pi_0 \cdot \frac{N}{1} \cdot \frac{N-1}{2} \cdots \frac{N-n+1}{n} = \pi_0 \binom{N}{n},$$

and normalizing, since $\sum_n \binom{N}{n} = 2^N$ counts the subsets of an N-element set by size, gives

$$\pi_n = \binom{N}{n} 2^{-N},$$

which is the Binomial$(N, \frac{1}{2})$ distribution of Section 2.4.

The equilibrium of a diffusing gas is therefore the same distribution as the number of heads in N coin flips. For large N that distribution is sharply concentrated near $N/2$ by Section 2.5, which is why the two halves of a container are found to hold equal amounts of gas, and why they are never found to hold anything else.

6.5.3 Metropolis–Hastings

We now reverse the problem: instead of finding a stationary distribution for a Markov chain, we start with a distribution we want to sample from and set up a Markov chain that converges to it. This is useful when the sample spaces are very large.

Example 6.5.5. An RNA molecule is a sequence of tens of thousands of nucleic acids that folds by forming bonds between pairs. A protein is a sequence of up to a few thousand amino acids, and about 2^{2000} subsets can bind together. We would like to know the probability that a molecule takes a particular shape, which means summing over an absurd number of configurations. The same problem shows up in finance, where the chance that a portfolio of a thousand assets loses more than ten per cent of its value is a thousand-dimensional integral, and in weather forecasting.

In these situations we usually cannot compute the probability of a configuration, but we can compute the *ratio* of two probabilities. In physics, for instance, the relative probability of two states with energies E_A and E_B is

$$\frac{\Pr[E_B]}{\Pr[E_A]} = \frac{e^{-E_B/kT}}{e^{-E_A/kT}} = e^{-(E_B - E_A)/kT},$$

which needs no knowledge of the normalizing constant, whereas computing either probability by itself would require summing $e^{-E/kT}$ over every state. Physicists call that sum the *partition function*. Sampling directly from this distribution is also intractable, since drawing a state with the right probability means knowing the probability of every state.

Instead, we sample using a Markov chain: because detailed balance is a statement about ratios, we only need the probability ratios to build a Markov chain that stabilizes on the distribution we want to sample from.

> **Metropolis–Hastings.** Let f be any function proportional to the target distribution π. Choose a *proposal* rule $g(x' \mid x)$ that suggests a next state and is symmetric, $g(x' \mid x) = g(x \mid x')$. From the current state x_t:

1. Propose a candidate x' from $g(\cdot \mid x_t)$.
2. Compute $\alpha = f(x')/f(x_t)$.
3. With probability $\min(1, \alpha)$ accept, setting $x_{t+1} = x'$. Otherwise reject, setting $x_{t+1} = x_t$.

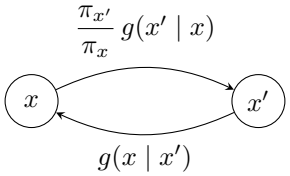
The proposed Markov chain need not resemble the actual dynamics of the system we are studying, though it sometimes does. For example, for RNA folding we can have $g(\cdot \mid x_t)$ pick two nucleic acids at random and add a bond between them if there is none, or remove it if there is. Since we are equally likely to undo this move by selecting those two bases again, this $g(\cdot \mid x_t)$ satisfies the required symmetry.

Proposition 6.5.6. *The chain produced by Metropolis–Hastings satisfies detailed balance with respect to π.*

Proof. Take two states x and x' with $\pi_x \geq \pi_{x'}$, so that the move from x to x' is downhill and is accepted with probability $\pi_{x'}/\pi_x$, while the reverse is uphill and always accepted. Then

$$\pi_x \, P_{xx'} = \pi_x \, g(x' \mid x) \frac{\pi_{x'}}{\pi_x} = \pi_{x'} \, g(x' \mid x) = \pi_{x'} \, g(x \mid x') = \pi_{x'} \, P_{x'x},$$

using the symmetry of g in the third step. The two transitions look like this:



Although α is computed from f rather than π, the two are related by some scaling factor that cancels in the ratio. □

By the proposition of the first subsection, π is therefore stationary, and provided the proposal makes the chain ergodic, the chain converges to it. So we can sample from a distribution we cannot even evaluate, by running a walk that only ever compares two states at a time.

> *Remark* 6.5.7 (Where the algorithm came from). The method first appeared in a 1953 paper, "Equation of State Calculations by Fast Computing Machines," by Nicholas Metropolis, Arianna Rosenbluth, Marshall Rosenbluth, Augusta Teller and Edward Teller. It began as a problem in materials science: Arianna and Marshall Rosenbluth proposed using MANIAC, one of the first computers at Los Alamos, to study how solids melt. Augusta Teller wrote an early version of the program, and Arianna Rosenbluth wrote the final code entirely, which made her the first person to implement Markov chain Monte Carlo. Neither of them received the credit they deserved. The algorithm carries Metropolis's name, although Marshall Rosenbluth later said that Metropolis contributed little beyond computer time, and a circulating rumor even posited that the two women's names had been added to the paper as a courtesy (which was wrong). W. Keith Hastings generalized the method in 1970, which is the second name in Metropolis–Hastings.

6.5.4 Bayesian Inference by Sampling

Having an unnormalized quantity that is *proportional* to a probability distribution is also a common problem in Bayesian statistics. Recall Bayes' rule from Section 2.2, read as a statement about models and data:

$$\Pr[\text{model} \mid \text{data}] = \frac{\Pr[\text{data} \mid \text{model}]\ \Pr[\text{model}]}{\Pr[\text{data}]}.$$

The numerator, likelihood times prior, is usually easy to write down. The evidence in the denominator is not: by the law of total probability it is a sum of the numerator over *every* model. When a model has many unknown quantities, that sum runs over every combination of their values, which gives combinatorics that can easily become hopeless. Fortunately, we can solve this with the exact same hammer we used for physical systems.

Example 6.5.8 (Rating sports teams)**.** A league has 30 teams, each playing 10 games a season, and team i has an unknown skill s_i, one of the levels $1, 2, \ldots, 10$. Take a uniform prior, with every combination of skills equally likely, and model a game by

$$\Pr[i \text{ beats } j] = \frac{s_i}{s_i + s_j},$$

which is the *Bradley–Terry* model of 1952. Treating the games of a season as independent given the skills, the posterior is proportional to the likelihood,

$$\Pr[s \mid \text{season}] \ \propto \prod_{\text{games}} \frac{s_{\text{winner}}}{s_{\text{winner}} + s_{\text{loser}}},$$

since the uniform prior is a constant. Normalizing would mean summing this product over all 10^{30} skill vectors. A computer checking a billion per second would need about 10^{21} seconds, a few thousand times the age of the universe.

Metropolis–Hastings can help us converge on a ranking before the human race ceases to exist. The state is a skill vector s. To propose a move, pick a team at random and move its skill up or down one level, each with probability $\frac{1}{2}$. Reject any move off the 1-10 scale. The proposal is symmetric, since every move is exactly as likely as the move that undoes it. In the ratio α, every game not involving the chosen team contributes the same factor before and after, and cancels. Suppose team A, at skill 5, beat team B at skill 4 and team C at skill 7. Proposing skill 6 gives

$$\alpha = \frac{6/10}{5/9} \cdot \frac{6/13}{5/12} \approx 1.20,$$

so the move is accepted: a higher skill explains two wins better. Proposing skill 4 instead gives

$$\alpha = \frac{4/8}{5/9} \cdot \frac{4/11}{5/12} \approx 0.79,$$

which is accepted with probability 0.79. This manufactured chain drifts toward skills that explain the season well, but keeps visiting the others in proportion to how plausible they are.

Run the chain, throw away the early samples taken before it has converged, and keep track of some useful statistics. The posterior probability that team A is the best is approximately the fraction of samples in which s_A is the largest skill, its posterior expected skill is about the average of s_A over the samples, and the middle 95% of the sampled values of s_A give a 95% *credible interval* for its skill. All of these can be computed before the next game!

Summarizing Metropolis-Hastings. Physics and statistics both hand us a distribution known only up to a constant, physics giving us the partition function and statistics giving us the evidence. In both cases, the constant in question is a sum over too many states to compute in our lifetime. Metropolis–Hastings skips that step: it compares two states at a time, and the convergence theorem of Section 6.4 turns the walk into samples. When the unknowns are continuous, as most parameters are, the sum becomes an integral (Chapter 7), and the same trick applies.

Chapter 7

Continuous Distributions and Monte Carlo

7.1 Poisson and Exponential Distributions

Up until now everything has been discrete. This lecture introduces continuous random variables, then finds the continuous-time versions of two distributions we already know.

7.1.1 Continuous Random Variables and Densities

A discrete random variable is described by its probability mass function, which gives the probability of each value. A continuous random variable can take any value in an interval, so the probability of any single exact value is zero, and the mass function says nothing. A description that works for both asks for the probability of everything up to x instead.

Definition 7.1.1 (Cumulative distribution function)**.** The *cumulative distribution function*, or CDF, of a random variable X is

$$F(x) = \Pr[X \leq x].$$

The CDF never decreases, starting from 0 far to the left and rising (or staying level) to 1 far to the right, and the probability of an interval is a difference: $\Pr[a < X \leq b] = F(b) - F(a)$. For a discrete random variable, it is a staircase that jumps by $\Pr[X = k]$ at each value k (Figure 7.1). A continuous random variable has no jumps, since no single value carries any probability, so its CDF rises smoothly, and the density describes how quickly this happens.

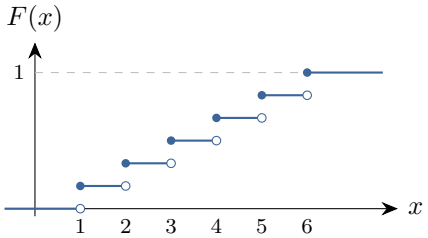


Figure 7.1: The CDF of a fair die roll is a staircase. It jumps by $\Pr[X = k] = \frac{1}{6}$ at each value k and is flat in between, rising from 0 to 1.

Definition 7.1.2 (Density)**.** Let X be a continuous random variable with CDF F. Its *probability density function*, or pdf, is

$$f(x) = \frac{dF(x)}{dx}, \qquad \text{so that} \qquad F(x') = \int_{-\infty}^{x'} f(x)\,dx.$$

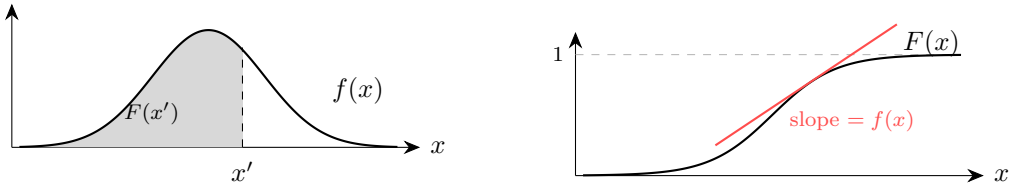


Figure 7.2: A density and its cumulative distribution function. The shaded area under f up to x' equals the height of F at x', and the slope of F at any point equals the density there. Probabilities are areas, which is why no single point carries any probability.

Figure 7.2 shows how the two descriptions fit together. Everything from Chapter 2 carries over, with sums replaced by integrals. In particular, expectation is now an integral:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x f(x)\,dx.$$

The simplest continuous distribution is the *uniform* on $[0, 1]$, with density 1 on $[0, 1]$ and 0 elsewhere, so that $\Pr[U \le u] = u$ for every u between 0 and 1.

Example 7.1.3 (P-values are uniform)**.** A statistical test asks whether an observation is surprising. Suppose a test statistic X has a continuous CDF F when nothing interesting is going on, the *null hypothesis*, and that large values are evidence against it. The *p-value* of an observed value x is the probability of a value at least that large under the null, the area in the tail beyond x:

$$p = \Pr[X \ge x] = 1 - F(x),$$

and a small p-value suggests something is going on, since it means we just saw something that would be very surprising if the null hypothesis were true.

What is the distribution of p-values when the null hypothesis is true? The p-value is then itself random, $p = 1 - F(X)$. First, $F(X)$ is uniform on $[0, 1]$: when F is increasing,

$$\Pr[F(X) \le u] = \Pr[X \le F^{-1}(u)] = F(F^{-1}(u)) = u.$$

And if $F(X)$ is uniform then so is $1 - F(X)$, by symmetry. So under the null hypothesis every p-value is equally likely, and $p < 0.05$ happens exactly 5% of the time. Test twenty colors of jelly bean for a link to acne, as in Figure 7.3, and on average one will come out "significant" by chance alone. This is the problem of *multiple comparisons*: one small p-value found by searching through many tests means essentially nothing.

7.1.2 From Coin Flips to Continuous Time

For coin flips, the geometric distribution says how long until heads, and the binomial says how many heads. Today we find the continuous-time pair: for events that can happen at any moment, the *exponential* distribution says how long until an event, and the *Poisson* distribution says how many events (Figure 7.4).

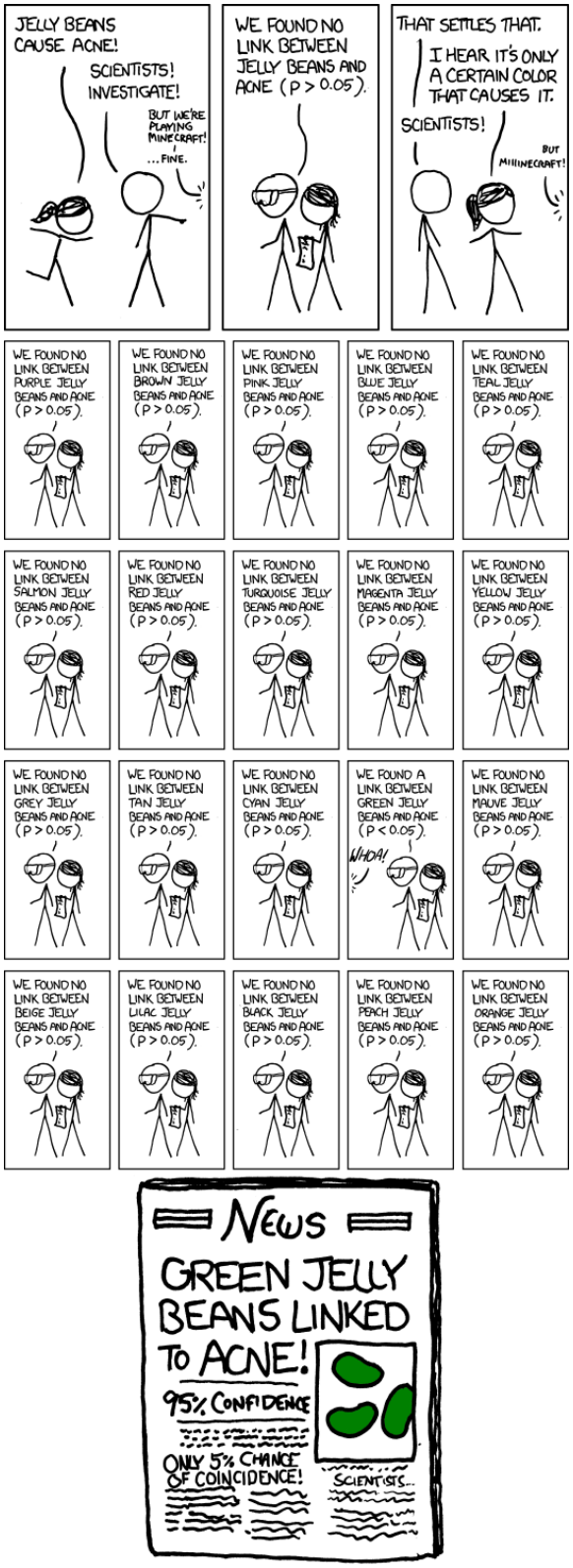


Figure 7.3: A comic about p-values. Credit to Randall Munroe and XKCD.

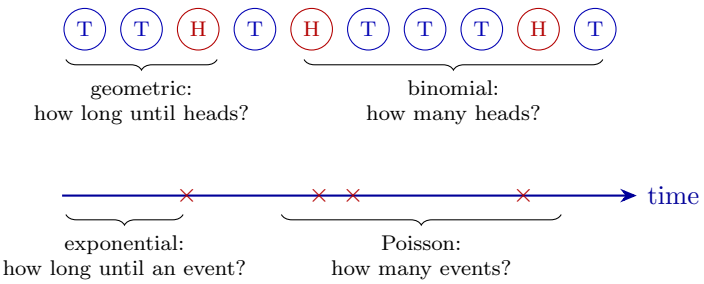


Figure 7.4: The discrete pair and the continuous pair. Coin flips come one at a time; events in continuous time, such as radioactive decays, can happen at any moment.

7.1.3 The Poisson Distribution

Start with the probability mass function of a binomial, and let the number of trials n grow while the success probability shrinks, $p = \lambda/n$, so that the mean $np = \lambda$ stays the same:

$$\Pr[X = k] = \binom{n}{k}\left(\frac{\lambda}{n}\right)^k\left(1-\frac{\lambda}{n}\right)^{n-k} = \frac{\lambda^k}{k!}\cdot\frac{n!}{(n-k)!\,n^k}\cdot\left(1-\frac{\lambda}{n}\right)^n\left(1-\frac{\lambda}{n}\right)^{-k}.$$

As $n\to\infty$, $n!/\big((n-k)!\,n^k\big)\to 1$ and $(1-\lambda/n)^{-k}\to 1$. For the remaining factor, take logarithms and write the limit as a fraction, so that l'Hôpital's rule can differentiate the numerator and the denominator:

$$\lim_{n\to\infty} n\log\left(1-\frac{\lambda}{n}\right) = \lim_{n\to\infty}\frac{\log\left(1-\frac{\lambda}{n}\right)}{1/n} = \lim_{n\to\infty}\frac{\dfrac{\lambda/n^2}{1-\lambda/n}}{-1/n^2} = \lim_{n\to\infty}\frac{-\lambda}{1-\lambda/n} = -\lambda,$$

so $(1-\lambda/n)^n\to e^{-\lambda}$. That last limit shows that our earlier approximation becomes exact as n gets large.

Definition 7.1.4 (Poisson distribution)**.** $X\sim\text{Poisson}(\lambda)$ if

$$\Pr[X=k] = \frac{\lambda^k}{k!}\,e^{-\lambda}, \qquad k = 0,1,2,\ldots$$

Its mean is λ, and so is its variance.

The parameter is the mean number of events, and it scales with time the way the mean of a binomial scales with the number of flips. If events happen at a *rate* of λ per unit time, the count in the interval $(0,1)$ is Poisson(λ). Think of n flips per unit time, each with probability λ/n: the interval $(0,t)$ holds nt flips, with mean λt, so the count there is Poisson(λt),

$$\Pr[k \text{ events in } (0,t)] = \frac{(\lambda t)^k}{k!}\,e^{-\lambda t}.$$

In general, it's often helpful to start a Poisson calculation by asking what the mean count is.

The limit needs only many independent chances for something unlikely, not time. The car of Example 2.2.8 had $n = 10{,}000$ parts, each defective with probability $p = 10^{-4}$, and the chance of no defect came out to $(1-10^{-4})^{10000} = 0.368$. That is the limit above: $\lambda = np = 1$, and 0.368 is $e^{-1} = \Pr[\text{Poisson}(1) = 0]$. The genome coverage calculation of Section 2.4 also demonstrated this, where a base is missed with probability e^{-c}: a Poisson count of zero reads, with mean c. The number of reads from each gene would likewise be Poisson if every sample expressed the gene equally. Expression varies from sample to sample,

though, and a Poisson count whose rate itself varies is more spread out than a Poisson, with variance larger than its mean. Mixing Poissons over a varying rate in this way gives the negative binomial of Section 2.4, which is what is usually used by computational biologists to model the distribution of genomic reads.

7.1.4 The Exponential Distribution

We derive the exponential as the "infinite coin" limit of a geometric. Flip a coin with success probability λ/n every $1/n$ of a time unit, scaling as before, so that successes come at rate λ. The number of flips until the first success is $T_n \sim \text{Geom}(\lambda/n)$, which takes time T_n/n. It comes after time t exactly when the first nt flips are all tails:

$$\Pr\Big[\frac{T_n}{n} > t\Big] = \Big(1 - \frac{\lambda}{n}\Big)^{nt} = \Big[\Big(1 - \frac{\lambda}{n}\Big)^{n}\Big]^{t} \longrightarrow e^{-\lambda t},$$

using the limit from the Poisson derivation.

The pattern of successes these flips converge to is called a *Poisson process* with rate λ. It has two properties: the count in any interval of length t is Poisson(λt), and counts in non-overlapping intervals are independent, since they are the limits of different coin flips.

Definition 7.1.5 (Exponential distribution)**.** $T \sim \text{Exp}(\lambda)$, the wait between events of a Poisson process with rate λ, has

$$\Pr[T > t] = e^{-\lambda t}, \qquad \text{so} \qquad \Pr[T \le t] = 1 - e^{-\lambda t}.$$

We can also derive the exponential in the following way: the first event comes after time t exactly when there are no events in $[0, t]$, a Poisson count with mean λt equal to 0, which has probability $\frac{(\lambda t)^0}{0!} e^{-\lambda t} = e^{-\lambda t}$.

The exponential is continuous, with density $f(t) = \lambda e^{-\lambda t}$ for $t \ge 0$, and integrating by parts gives $\mathbb{E}[T] = 1/\lambda$: events at rate λ come, on average, $1/\lambda$ apart. Software is not consistent about the parameter: R's `rexp` takes the rate λ, but NumPy's `exponential` and SciPy's `expon` take the mean $1/\lambda$ (their *scale*). Watch out for coding bugs that mix these two conventions up!

7.1.5 Exponentials Are Memoryless

Just like the geometric (Section 2.4), the exponential forgets how long you have already waited:

$$\Pr[T > t + s \mid T > s] = \frac{\Pr[T > t + s]}{\Pr[T > s]} = \frac{e^{-\lambda(t+s)}}{e^{-\lambda s}} = e^{-\lambda t} = \Pr[T > t].$$

Example 7.1.6 (Radioactive decay)**.** A Geiger counter clicks each time an atom in a radioactive sample emits a beta particle. Count the clicks in many separate intervals of the same length Δt, and the counts follow a Poisson distribution (Figure 7.5); say the clicks come at a rate of λ per Δt.

- *How many clicks do we expect in* $2\Delta t$*?* The mean count scales with time: 2λ.
- *How is the time between clicks distributed?* Exponentially, Exp(λ), measuring time in units of Δt.
- *After a long silence, do we expect a burst of clicks to make up for it?* No: by memorylessness, the wait from now on is the same as it always was.

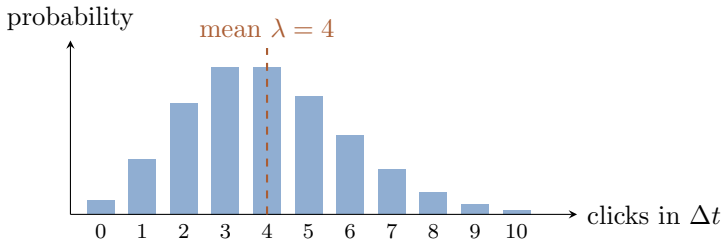


Figure 7.5: Count the clicks of a Geiger counter in many separate intervals of the same length Δt, and the counts follow a Poisson distribution, here with mean $\lambda = 4$ clicks per interval.

The same model describes photons arriving at a detector, which causes *shot noise* in electronics, and mutations along a stretch of DNA.

Example 7.1.7 (The 100-year flood)**.** Civil engineers size levees, bridges and storm drains for the 100-*year flood*, a flood so large that it has probability 1% in any given year. Model these floods as arriving at rate $\lambda = 0.01$ per year. Over a 30-year mortgage, the number of them is Poisson with mean $\lambda t = 0.3$, so

$$\Pr[\text{at least one}] = 1 - e^{-0.3} \approx 0.26.$$

So there is more than a one-in-four chance that a bank giving out a 30-year mortgage will at some point have to deal with a 100-year flood. This is why they make you get flood insurance... And just as with the Geiger counter, last year's flood buys no time: by memorylessness, the wait for the next one is still Exp(0.01), averaging 100 years from today.

Now look backward as well. A Poisson process run backward in time is again a Poisson process (it looks the same in reverse, like the reversible chains of Section 6.5), so the time since the *last* 100-year flood is also Exp(0.01), with mean 100 years. Therefore, the stretch between the flood before today and the flood after it is, on average 200 years, even though the floods come 100 years apart on average. This is the *inspection paradox*. Long gaps cover more of the timeline, so a random moment such as today is more likely to land in one; it is the same reason the bus always seems to take a long time.

This flood argument assumes the rate has held steady for a long time with the same λ. Because of climate change and rapid development, this has often not been the case: Houston saw three floods described as 500-year events between 2015 and 2017.

The queues of Section 7.3 run on the sliced clock itself, with a customer arriving with a small probability at each step; letting the steps shrink would make the arrivals a Poisson process.

> *Remark* 7.1.8 (Why is the night sky dark?). The Poisson limit needs only many unlikely, independent chances, which can happen in time or space (or both). Suppose the universe were infinite and eternal, with stars scattered at random, n per unit volume on average. Each star gives off power L, and its disk has area σ. This argument, *Olbers' paradox*, shows the night sky should then be blindingly bright.
>
> *Step 1: every line of sight ends on a star.* Follow one line of sight, an infinitely thin ray. A star blocks it exactly when the star's center lies in a cylinder around the ray with cross-section σ, whatever the star's distance. Out to distance R that cylinder has volume

σR, so the number of stars in it is Poisson with mean $n\sigma R$, and

$$\Pr[\text{the ray hits a star within distance } R] = 1 - e^{-n\sigma R} \longrightarrow 1.$$

Step 2: a star looks equally bright at any distance. Measure the brightness of a patch of sky as the light received from it divided by its solid angle. A star at distance r delivers $L/(4\pi r^2)$ and covers a solid angle of σ/r^2, so its brightness is

$$\frac{L/(4\pi r^2)}{\sigma/r^2} = \frac{L}{4\pi\sigma} = S,$$

the brightness of a star's surface, with no r left in it. A far star is fainter, but that is only because it looks smaller — it's still as bright as the sun.

Step 3: the sky is as bright as a star. A patch of sky is a bundle of rays. A distant star covers only a few of them, but by Step 1 a fraction $1 - e^{-n\sigma R}$ of them end on some star, and each of those sees brightness S. So the patch has brightness $S(1 - e^{-n\sigma R})$, which approaches S: every patch of sky should look like the surface of the Sun. If you're having trouble imagining this, go look through a large forest and observe that every line of sight eventually hits a tree trunk (however thin and far away), giving what is effectively one solid wall of bark.

The sky is dark because the universe is not eternal. It is about 14 billion years old, so light reaches us from only about $R = 10^{10}$ light-years away, while the mean distance to the first star, $1/(n\sigma)$, is around 10^{23} light-years. So $n\sigma R \approx 10^{-13}$, and the sky has about 10^{-13} of the Sun's brightness. Edgar Allan Poe was the first to propose this answer, in his 1848 prose poem *Eureka*.

7.2 Random Walks and Normal Distributions

7.2.1 Zooming In on a Random Walk

In Section 7.1 we took a limit of the binomial, letting the number of trials n grow while holding the mean $np = \lambda$ fixed. The success probability shrank, the events became rare, and the limit was the Poisson. Today we take the binomial's other limit: hold p fixed and let n grow.

Consider a random walk that starts at $Z_0 = 0$ and at each step moves up by h or down by h, each with probability $\frac{1}{2}$. Its increments $X_i = Z_i - Z_{i-1}$ are $X_i = h\,b_i$, where each b_i is $+1$ or -1 with probability $\frac{1}{2}$. This is a Bernoulli, shifted and scaled, $b_i = 2B_i - 1$ for a fair coin B_i. After n steps the walker is at

$$Z_n = X_1 + X_2 + \cdots + X_n = h\sum_{i=1}^{n} b_i,$$

a shifted and scaled binomial. Choose h so that $\mathrm{Var}(Z_n) = 1$. Each b_i has $\mathrm{Var}(b_i) = \frac{1}{2}(1)^2 + \frac{1}{2}(-1)^2 = 1$, so by Section 2.5

$$\mathrm{Var}(Z_n) = h^2\,\mathrm{Var}\Big(\sum_{i=1}^{n} b_i\Big) = h^2\, n\,\mathrm{Var}(b_i) = nh^2,$$

and $h = 1/\sqrt{n}$. As n grows, the steps shrink; the walker's possible positions crowd together into a continuous random variable.

7.2.2 Sums Smooth to Bells

In the limit, the walker's position becomes a *continuous* random variable, described by a density as in Section 7.1. Most sums of random variables start to look bell-shaped. To see why, we need to introduce a new way of thinking about a convolution.

Recall the convolution of two dice from Section 4.3. To find $\Pr[D_1 + D_2 = 7]$, write the first die's outcomes $1, \ldots, 6$ in a row, and under them the second die's outcomes in reverse, $6, \ldots, 1$. Every column then adds up to 7, the probability of each column is the product of its two probabilities, and adding the columns gives $\Pr[D_1 + D_2 = 7] = \frac{6}{36}$. For $\Pr[D_1 + D_2 = 8]$, slide the reversed row one place over: now only five columns line up, giving $\frac{5}{36}$ (Figure 7.6). This interpretation of convolution is the reason we call neural networks with sliding windows (which you may see in a later class) "convolutional" neural networks.

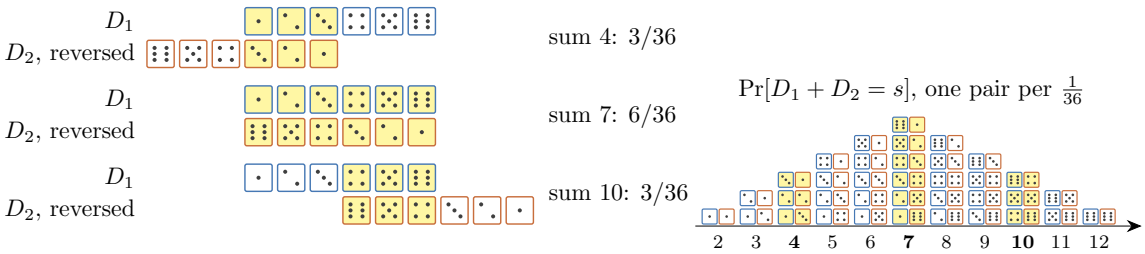


Figure 7.6: Convolution as flip and slide. Reverse the probabilities of the second die, in orange, and slide them along those of the first, in blue, so that every column adds up to the target sum. The shaded columns are the ways of making the sum, each with probability $\frac{1}{6} \cdot \frac{1}{6}$: they are the microstates stacked over that sum in Figure 2.1. Sliding one step at a time traces out the whole distribution, on the right, where each sum's column stacks the dice pairs that make it, as in Figure 2.1, and the three sums worked on the left are highlighted.

Writing f and g for the pmfs of independent X and Y, the convolution is

$$(f * g)(x) = \sum_y f(y)\, g(x - y),$$

and for densities the sum becomes an integral, $(f * g)(x) = \int f(y)\, g(x - y)\, dy$. In words: reverse g, slide it to x, multiply it by f, and add up. For densities the sliding works the same way. Take the simplest continuous distribution, the uniform on $[0, 1]$, whose density is a box (and a box reversed is still a box). To find the density of $X_1 + X_2$ at x, slide a box to x and measure the area where it overlaps the first box (since the product of the overlap is just multiplying times the uniform density, which is 1 everywhere). As it slides in, the overlap grows steadily, and as it slides out, it shrinks steadily: the sum of two uniforms has a tent-shaped density. Sliding a box over the tent gives a curve made of three smooth pieces, which already looks like a bell (Figure 7.7). Each time we add another copy, the density gets averaged over the window, which smooths it out.

The same thing happens whatever distribution we start from, as long as its variance is finite: after rescaling, the sums always approach the same bell shape. The next subsection shows why the starting distribution is forgotten.

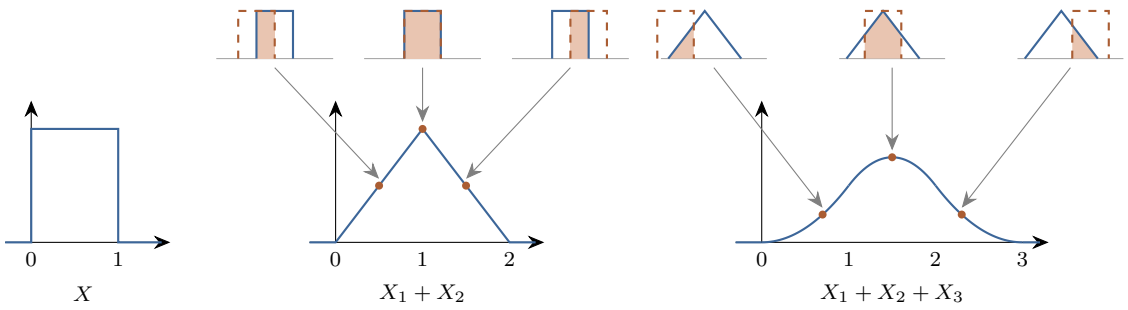


Figure 7.7: Sums smooth out, computed by sliding. Each small picture shows the density so far (solid) with a box (dashed) slid to one position x; the area where they overlap (shaded) is the height of the new density at that x, where the arrow points. Alternatively, you can think of this area as an average over the box's window. Sliding a box over a box, the overlap grows steadily and then shrinks steadily, giving the tent. Sliding a box over the tent gives a curve made of three smooth pieces, $x^2/2$, then $(-2x^2 + 6x - 3)/2$, then $(3 - x)^2/2$, which already looks like a bell.

7.2.3 Only the Variance Survives

Use the moments of Section 2.5. Let $X_1, X_2, \dots$ be independent and identically distributed with mean 0 and variance σ^2, and scale their sum so that its variance stays fixed:

$$Z_n = \frac{X_1 + \cdots + X_n}{\sqrt{n}}, \qquad \mathbb{E}[Z_n] = 0, \quad \operatorname{Var}(Z_n) = \sigma^2.$$

Expanding $(X_1 + \cdots + X_n)^k$ gives products of the X_i. A product in which some X_i appears exactly once has expectation 0, since that X_i is independent of the rest and has mean 0. So only the products in which every X_i appears at least twice survive.

For the third moment, look first at $n = 2$:

$$(X_1 + X_2)^3 = X_1^3 + 3X_1^2X_2 + 3X_1X_2^2 + X_2^3,$$

and the middle terms vanish in expectation, since for instance $\mathbb{E}[3X_1^2X_2] = 3\,\mathbb{E}[X_1^2]\,\mathbb{E}[X_2] = 0$. In general only the n terms X_i^3 survive:

$$\mathbb{E}[Z_n^3] = \frac{n\,\mathbb{E}[X^3]}{n^{3/2}} = \frac{\mathbb{E}[X^3]}{\sqrt{n}} \longrightarrow 0.$$

For the fourth, write $(X_1 + \cdots + X_n)^4$ as the sum of all products $X_aX_bX_cX_d$, one for each choice of four indices. Products like $X_iX_j^2X_k$ or $X_iX_j^3$ contain some X exactly once, so they vanish. Two kinds survive:

- all four indices equal, giving the n terms X_i^4, each with expectation $\mathbb{E}[X^4]$;
- two different indices, each used twice, giving terms $X_i^2X_j^2$ with $i \neq j$. There are $\binom{n}{2}$ pairs of indices, and each pair appears in $\binom{4}{2} = 6$ orderings (which two of the four positions get i), so $6\binom{n}{2} = 3n(n-1)$ such terms. By independence each has expectation $\mathbb{E}[X_i^2]\,\mathbb{E}[X_j^2] = \sigma^2 \cdot \sigma^2 = \sigma^4$.

Dividing by $(\sqrt{n})^4 = n^2$,

$$\mathbb{E}[Z_n^4] = \frac{n\,\mathbb{E}[X^4] + 3n(n-1)\,\sigma^4}{n^2} \longrightarrow 3\sigma^4.$$

The starting distribution's own $\mathbb{E}[X^3]$ and $\mathbb{E}[X^4]$ have washed out, and only σ^2 is left. The same counting works for every k: in the limit, every odd moment goes to 0, and every even moment becomes a fixed multiple of σ^k. The limit's moments are the same whatever distribution we started with, and they pin down its shape, so the limit is one universal shape that depends only on the variance. That shape has a name that you've probably seen before.

Definition 7.2.1 (Normal distribution)**.** $X \sim N(\mu, \sigma^2)$ if its density is

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-(x-\mu)^2/2\sigma^2}.$$

Here $\mu = \mathbb{E}[X]$ is the mean and $\sigma^2 = \mathrm{Var}(X)$ the variance.

The normal, or *Gaussian*, has two parameters, and they are exactly the mean and variance of Chapter 2. Stripped of its constants the shape is e^{-x^2}; everything else positions it, scales it, and makes the area equal one. But why should this particular formula be the universal shape? Because adding two independent normals gives another normal: the Gaussian is a *fixed point* of adding copies. Seeing that takes a second look at convolution.

7.2.4 Convolution as a Table

There is another way to see the convolution of two dice. Make a table with the outcomes of one die along the bottom and the other up the side, like the axes of a graph. Each cell holds the probability of that pair of rolls, the product of the two probabilities, and the cells whose rolls add up to 7 lie along a diagonal. Adding up that diagonal gives $\Pr[D_1 + D_2 = 7]$, and each of the other diagonals gives one of the other sums (Figure 7.8). It works the same way for a loaded die: the cells are no longer all equal, but each diagonal still collects exactly the products in the convolution formula.

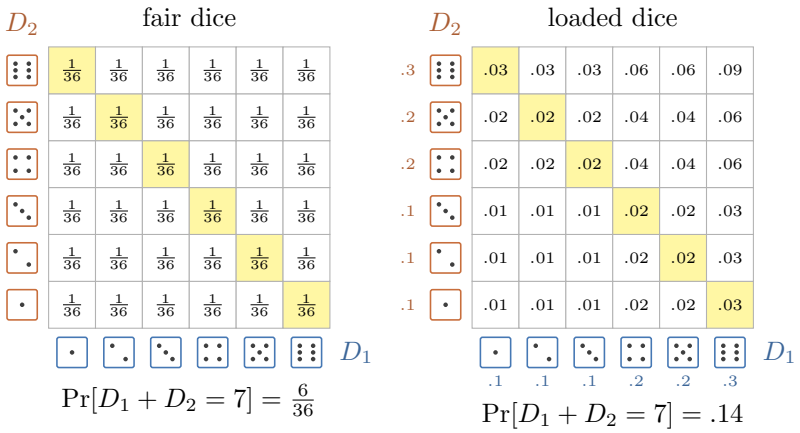
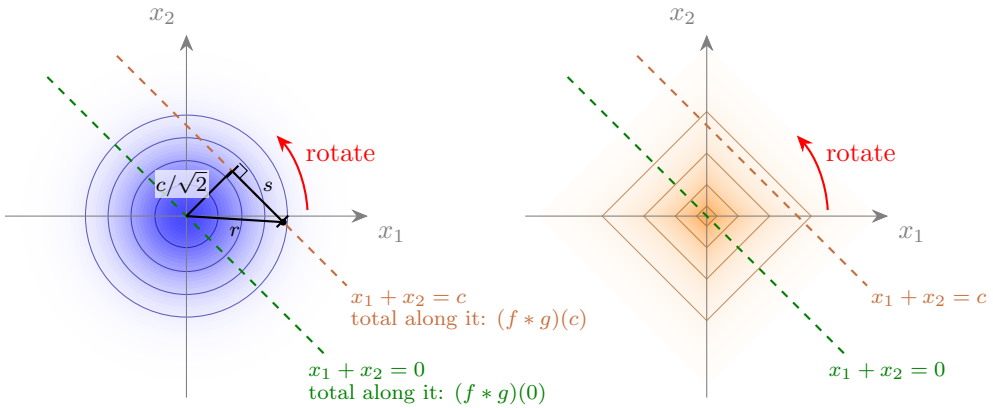


Figure 7.8: Convolution as a table. The first die runs along the bottom, in blue, and the second up the side, in orange, like the axes x_1 and x_2 of Figure 7.9. Each cell holds the probability of that pair of rolls, the product of the two dice's probabilities, and the pairs that add up to 7 lie on the highlighted diagonal, running the same way as the line $x_1 + x_2 = c$ there, so adding up the diagonal gives $\Pr[D_1 + D_2 = 7]$. Left: fair dice, where every cell is $\frac{1}{36}$. Right: a loaded die rolled twice, with probabilities .1, .1, .1, .2, .2, .3 for the faces 1 to 6, so the cells differ, but the same diagonal still gives the answer, $.03 + .02 + .02 + .02 + .02 + .03 = .14$.

7.2.5 Gaussians Are a Fixed Point

For continuous random variables the table becomes a surface over the plane, the joint density $f(x_1)\,g(x_2)$, and the convolution at c is the total along the diagonal line $x_1 + x_2 = c$. Seen from above, the surface looks like the table, with the same axes and the same diagonal (Figure 7.9).



Gaussian: circular level sets
every slice has the same shape

Laplacian: diamond level sets
slices depend on the direction

Figure 7.9: The joint density of two independent random variables, seen from above like a map: darker means higher, and the thin lines are contour lines, each joining points of equal density. On the left, two independent Gaussians give $e^{-x_1^2}e^{-x_2^2} = e^{-r^2}$, which depends only on the distance from the origin, so the level sets are circles and rotating the picture changes nothing. Every straight slice therefore has the same shape, including the two dashed diagonals, through the origin ($c = 0$, green) and off to the side (orange), the same slices as in Figure 7.10. Adding up the density along a diagonal gives the density of $X_1 + X_2$ there, $(f * g)(0)$ along the green line and $(f * g)(c)$ along the orange one. The black right triangle is the one in the text: the closest point of the orange line is at distance $c/\sqrt{2}$ from the origin, and a point a further distance s along the line is at distance r, with $r^2 = c^2/2 + s^2$. On the right, two independent Laplacians give a density proportional to $e^{-|x_1|-|x_2|}$, whose level sets are diamonds aligned with the axes. Here the two diagonals differ: the green one, through the origin, cuts straight across the diamonds, peaking sharply at the center, while the orange one runs along a diamond's edge, where the density is flat. So the slices do not all have the same shape. Independence alone does not produce the symmetry: rotating this picture deforms it, and the sum of two Laplacians is not Laplacian.

Take two independent normals, keeping only the shape e^{-x^2}. Their table is

$$e^{-x_1^2} \cdot e^{-x_2^2} = e^{-(x_1^2+x_2^2)} = e^{-r^2},$$

where r is the distance from the origin, by the Pythagorean theorem. The surface depends only on that distance, so it is rotationally symmetric: a hill whose contour lines are circles (Figure 7.9, left).

Now slice it along the diagonal $x_1 + x_2 = c$ (Figure 7.10). The point of that line closest to the origin is at distance $c/\sqrt{2}$, so a point a further distance s along the line has $r^2 = c^2/2 + s^2$, by Pythagoras again. Along the line the hill is therefore

$$e^{-c^2/2} \cdot e^{-s^2}.$$

In s this is a Gaussian, the same shape on every diagonal. Adding it up along the line gives a constant times $e^{-c^2/2}$, so the total on each diagonal, as a function of c, is again a

Gaussian. The sum of two independent normals is normal: the Gaussian distribution is a fixed point of $X \to X + X$ (Figure 7.10).

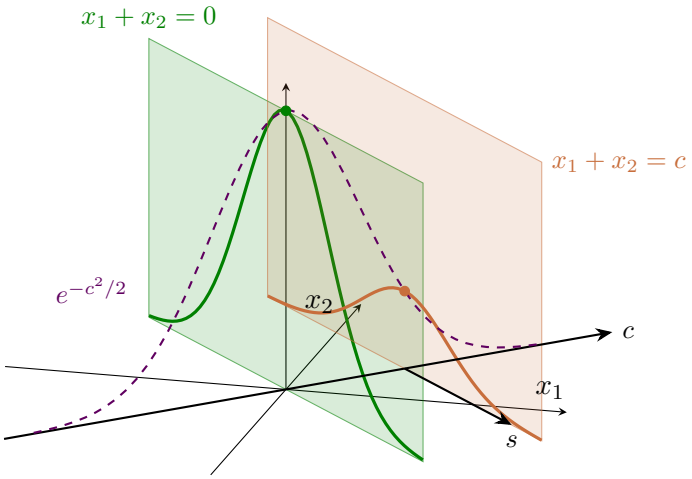


Figure 7.10: Slicing the hill $e^{-x_1^2}e^{-x_2^2} = e^{-r^2}$. Each vertical plane stands on a diagonal $x_1 + x_2 = c$, and where it cuts the hill is the slice whose total is the convolution at c. The slice through the peak, in green, is a bell curve. The slice further out, in orange, is the same bell curve, only lower, by the factor $e^{-c^2/2}$. Along a slice the position is s, and moving from one slice to the next changes c (black arrows on the floor). The dashed purple curve runs across the slices through their peaks (dots): it is the Gaussian $e^{-c^2/2}$ that sets each slice's height. So every slice is a Gaussian, and the slices' totals, as c varies, form a Gaussian too.

Only the Gaussian has this property: among densities whose table factors into two independent pieces, it is the only one that is rotationally symmetric (Figure 7.9, right). The same rotational symmetry is why π appears in the Gaussian's formula: making the area equal one means measuring a hill with circular contours.

7.2.6 The Central Limit Theorem

Theorem 7.2.2 (Central limit theorem)**.** *Let $X_1, X_2, \ldots$ be independent and identically distributed with mean μ and finite variance σ^2. Then the standardized sum*

$$\frac{X_1 + \cdots + X_n - n\mu}{\sigma\sqrt{n}}$$

converges in distribution to $N(0,1)$ as $n \to \infty$.

The two halves of this lecture fit together. Start the sums from normal X_i: since the Gaussian is a fixed point, every Z_n is then exactly $N(0, \sigma^2)$, so for normals the limit is the Gaussian. The moments show that the limit is the same whatever distribution we start from, so it is the Gaussian for all of them.

The random walk we started with is a special case: its position after n steps is the standardized sum of n fair ± 1 steps, so it approaches $N(0,1)$. For the binomial $B \sim \text{Bin}(n,p)$ this says

$$\frac{B - np}{\sqrt{np(1-p)}} \longrightarrow N(0,1),$$

which Abraham de Moivre found for coin flips in 1733, which was the first version of the central limit theorem.

This completes the story begun in Section 2.5. There we showed that the sample mean has variance σ^2/n and used Chebyshev to conclude that it concentrates. The central limit theorem says more: not only does the error shrink like $\sigma/\sqrt{n}$, its distribution is asymptotically normal, so the loose Chebyshev bound can be replaced by an actual confidence interval. A common rule of thumb is that $n > 30$ is enough for the approximation to be usable.

7.2.7 Why Gaussians Are Everywhere

Anything that is a sum of many small, independent contributions is close to normal.

- *Height.* A person's height is the sum of the lengths of many bones, each varying a little from person to person, so heights are close to normal.
- *Noise.* The voltage across a resistor jitters because of the tiny, independent pushes of countless electrons, so electrical noise is normal, which is why engineers model noise as Gaussian.
- *Products.* If a quantity is a *product* of many independent factors, $Z = Y_1 Y_2 \cdots Y_n$, then $\log Z$ is a sum, so $\log Z$ is close to normal and Z is *log-normal*. Multiplicative growth processes, including many financial ones, are log-normal.
- *Brownian motion.* Go back to the walk with steps of size $1/\sqrt{n}$, one molecular kick per time step, and follow its whole path rather than only where it ends. As n grows, the path becomes a continuous random curve, and its position at time t is $N(0, t)$: the variance grows in proportion to time, the number of steps. This is *Brownian motion*. Robert Brown saw it in 1827 as the jittering of tiny particles in water, and Einstein explained it in 1905 as the sum of countless molecular kicks.
- *Least squares.* Fit a line $y = \alpha x$ to data, with each point off the line by a residual r_i (Figure 7.11). If the residuals are independent $N(0, \sigma^2)$ errors, the likelihood of the data is
$$\Pr[\text{data} \mid \text{model}] \propto e^{-r_1^2/2\sigma^2} \cdots e^{-r_n^2/2\sigma^2} = e^{-\sum_i r_i^2/2\sigma^2},$$
which is largest when $\sum_i r_i^2$ is smallest. Under a uniform prior, Bayes' rule (Section 2.2) makes the posterior $\Pr[\text{model} \mid \text{data}]$ proportional to this likelihood, so the most probable line is the least-squares line.

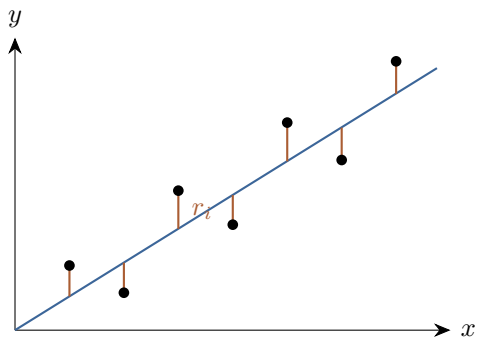


Figure 7.11: Least squares. Each residual r_i is the vertical distance from a data point to the line. If the residuals are independent and normal, the line that makes the data most likely is the one that makes $\sum_i r_i^2$ smallest.